GameVote: Manual Rebuild Guide

Every command and every file, phase by phase

Chuck

November 2, 2026

Table of contents

Rebuild guide **4**
Overview . . . 5
Hand-made files and edits . . . 5
New files with no artisan generator . . . 5
Starter-kit files to edit . . . 6
Build order . . . 7
Phase 1: Setup . . . 7
Phase 2: Username accounts . . . 8
Phase 3: One-column UI . . . 16
Phase 4: Data structure . . . 17
Phase 5: Backend processing . . . 18
Phase 6: Refining the UI . . . 21
Phase 7: Security hardening . . . 21
Phase 8: Tests . . . 22
Phase 9: Docker, deployment and docs . . . 22
Files to delete from the starter kit . . . 24
Verification . . . 25

Backend code **25**
`.editorconfig` (edit) . . . 26
`.env.example` (edit) . . . 26
`.github/workflows/tests.yml` (edit) . . . 26
`.gitignore` (edit) . . . 27
`app/Actions/Fortify/CreateNewUser.php` (edit) . . . 27
`app/Actions/Games/AddGame.php` (new file) . . . 27
`app/Actions/Games/CastVote.php` (new file) . . . 28
`app/Actions/Games/RemoveGame.php` (new file) . . . 29
`app/Actions/Games/RemoveVote.php` (new file) . . . 30
`app/Actions/Games/ResetLocalLibrary.php` (new file) . . . 31
`app/Concerns/ProfileValidationRules.php` (edit) . . . 31
`app/Console/Commands/SeedGames.php` (new file) . . . 32
`app/Enums/DailyActionType.php` (new file) . . . 34
`app/Exceptions/GameActionException.php` (new file) . . . 34
`app/Http/Controllers/GameController.php` (new file) . . . 35
`app/Http/Controllers/GameVoteController.php` (new file) . . . 37
`app/Http/Controllers/Settings/ProfileController.php` (edit) . . . 37
`app/Http/Middleware/AddContentSecurityPolicy.php` (new file) . . . 38
`app/Http/Middleware/HandleAppearance.php` (edit) . . . 39
`app/Http/Middleware/HandleInertiaRequests.php` (edit) . . . 39
`app/Http/Requests/Settings/ProfileUpdateRequest.php` (edit) . . . 40
`app/Http/Requests/StoreGameRequest.php` (new file) . . . 40
`app/Models/ApiCredential.php` (new file) . . . 41
`app/Models/DailyAction.php` (new file) . . . 42
`app/Models/User.php` (edit) . . . 43
`app/Models/Vote.php` (new file) . . . 44
`app/Providers/AppServiceProvider.php` (edit) . . . 44
`app/Providers/FortifyServiceProvider.php` (edit) . . . 45
`app/Services/DailyAllowance.php` (new file) . . . 46
`app/Services/Essense/ApiKeyProvider.php` (new file) . . . 49
`app/Services/Essense/EssenseApiException.php` (new file) . . . 51
`app/Services/Essense/EssenseClient.php` (new file) . . . 51
`app/Services/Essense/GameData.php` (new file) . . . 54
`bootstrap/app.php` (edit) . . . 55

composer.json (edit) . . . 55
config/app.php (edit) . . . 56
config/fortify.php (edit) . . . 56
config/games.php (new file) . . . 56
config/inertia.php (edit) . . . 57
config/services.php (edit) . . . 57
database/factories/DailyActionFactory.php (new file) . . . 57
database/factories/UserFactory.php (edit) . . . 58
database/factories/VoteFactory.php (new file) . . . 58
database/migrations/0001_01_01_000000_create_users_table.php (edit) . . . 59
database/migrations/2026_10_20_013125_create_votes_table.php (new file) . . . 60
database/migrations/2026_10_20_013126_create_daily_actions_table.php (new file) 60
database/migrations/2026_10_20_013445_add_timezone_to_users_table.php (new file) 61
database/migrations/2026_10_20_014928_create_api_credentials_table.php (new file) . . . 61
database/seeders/DatabaseSeeder.php (edit) . . . 62
phpunit.xml (edit) . . . 62
routes/settings.php (edit) . . . 62
routes/web.php (edit) . . . 62
package.json (edit) . . . 63
resources/css/app.css (edit) . . . 63
resources/js/app.tsx (edit) . . . 65
resources/js/components/add-game-form.tsx (new file) . . . 66
resources/js/components/app-logo-icon.tsx (edit) . . . 67
resources/js/components/app-top-bar.tsx (new file) . . . 67
resources/js/components/daily-allowance-notice.tsx (new file) . . . 68
resources/js/components/game-list-item.tsx (new file) . . . 69
resources/js/components/meeple-votes.tsx (new file) . . . 73
resources/js/components/meeple.tsx (new file) . . . 74
resources/js/components/spray-painted-title.tsx (new file) . . . 74
resources/js/hooks/use-flip-animation.ts (new file) . . . 75
resources/js/layouts/app-layout.tsx (edit) . . . 76
resources/js/pages/auth/login.tsx (edit) . . . 76
resources/js/pages/auth/register.tsx (edit) . . . 78
resources/js/pages/dashboard.tsx (edit) . . . 79
resources/js/pages/settings/appearance.tsx (edit) . . . 81
resources/js/pages/settings/profile.tsx (edit) . . . 81
resources/js/pages/settings/security.tsx (edit) . . . 83
resources/js/pages/welcome.tsx (edit) . . . 83
resources/js/types/auth.ts (edit) . . . 85
resources/js/types/games.ts (new file) . . . 85
resources/js/types/global.d.ts (edit) . . . 85
resources/js/types/index.ts (edit) . . . 85
resources/js/types/navigation.ts (edit) . . . 85
resources/js/types/ui.ts (edit) . . . 86
resources/views/app.blade.php (edit) . . . 86
vite.config.ts (edit) . . . 86

Tests code **87**
tests/FakeEssenseApi.php (new file) . . . 88
tests/Feature/Auth/AuthenticationTest.php (edit) . . . 90
tests/Feature/Auth/RegistrationTest.php (edit) . . . 92
tests/Feature/DashboardTest.php (edit) . . . 94
tests/Feature/Games/AddGameTest.php (new file) . . . 95
tests/Feature/Games/ApiKeyTest.php (new file) . . . 96
tests/Feature/Games/EssenseClientTest.php (new file) . . . 98

tests/Feature/Games/GameLibraryTest.php (new file) . . . 100
tests/Feature/Games/RemoveGameTest.php (new file) . . . 101
tests/Feature/Games/RemoveLimitsTest.php (new file) . . . 102
tests/Feature/Games/SeedGamesCommandTest.php (new file) . . . 103
tests/Feature/Games/VoteTest.php (new file) . . . 104
tests/Feature/SecurityHardeningTest.php (new file) . . . 106
tests/Feature/Settings/ProfileUpdateTest.php (edit) . . . 107
tests/Feature/TrustedProxiesTest.php (new file) . . . 110
tests/Pest.php (check) . . . 110
tests/TestCase.php (edit) . . . 111

Docker code **112**
.dockerignore (new file) . . . 113
.railway/railway.ts (new file) . . . 113
Dockerfile (new file) . . . 114
docker-compose.yml (new file) . . . 116
docker/dev-entrypoint.sh (new file) . . . 117
docker/entrypoint.sh (new file) . . . 117
docker/nginx-http-auth.sh (new file) . . . 118
docker/nginx.conf (new file) . . . 119
docker/railway-start.sh (new file) . . . 120

Rebuild guide

Overview

GameVote is the official Laravel React starter kit plus 70 new files and edits to 56 kit files; artisan can generate about 30 of the new ones, and everything else is written or edited by hand. This guide lists every one of those, compared file by file against a fresh clone of laravel/react-starter-kit, then gives the build order.

- **Prerequisites:** macOS, Linux, or Windows with WSL2 (Ubuntu; keep projects under ~/, not /mnt/c). PHP 8.5, Composer, Node 22.12+ with npm 10+ (install Node with nvm; Ubuntu's apt npm 9 is too old for the kit), and the Laravel installer (`composer global require laravel/installer`).
- **PHP extensions and tools:** Composer needs `zip`, `curl`, `xml`/`dom` and `mbstring` to install the kit; the app needs `sqlite3` (SQLite database and tests), `intl` and `bcmath`. Install them with `unzip` and `git` before running `laravel new` (change `8.5` if your PHP version differs):

  ```
  sudo apt install php8.5-{cli,zip,curl,xml,mbstring,sqlite3,intl,bcmath} unzip git
  php -m | grep -iE 'zip|curl|xml|dom|mbstring|sqlite|intl|bcmath'
  ```

  If `laravel new` fails with `Command failed: composer create-project laravel/react-starter-kit ...`, a missing extension is the usual cause: delete the half-made folder, run `composer create-project laravel/react-starter-kit test-app --stability=dev -vvv` to see the full error, then install what it names.
- **Laravel installer on PATH:** Composer puts `laravel` in its global bin folder. Add it once:

  ```
  echo 'export PATH="$(composer global config bin-dir --absolute --quiet):$PATH"' >> ~/.bashrc
  source ~/.bashrc
  laravel --version
  ```
- **Reference copy:** clone webanoid/gamevote beside your rebuild. Its code matches the code sections except where the repo keeps two-factor and passkeys; there, use the registration-only versions in this guide.
- **Rules and reasoning** live in the repo's `docs/` folder: `daily-rule.md`, `api-integration.md`, `data-model.md`, `frontend.md`, `deployment.md`, `decisions.md`.
- **Kit version caveat:** the comparison used the kit's current main branch, which may be a few commits ahead of the version GameVote started from. A file flagged as changed may differ only by those upstream updates.

The exact text for every file is in the code sections that follow: Backend code, Frontend code, Tests code and Docker code. New files show their full content. Edited kit files show a diff against the kit (`-` remove, `+` add), or the full file when most of it changed.

Hand-made files and edits

Yes: the earlier walkthrough relied on many files with no artisan generator, and it skipped several starter-kit edits entirely. Below is the complete list. The exact text for each file is in the code sections, A to Z by path.

New files with no artisan generator

File	What it holds
`config/games.php`	`remove_limits` flag read from `REMOVE_LIMITS`
`tests/FakeEssenseApi.php`	In-memory fake of the API, including its quirks, `restart()` and `unreachable()`
`resources/js/components/add-game-form.tsx`	Add-game form with inline errors

File	What it holds
resources/js/components/app-top-bar.tsx	Sticky bar: logo, @username, settings icon, Log out
resources/js/components/daily-allowance-notice.tsx	Today's allowance or the limits-off notice
resources/js/components/game-list-item.tsx	One ranked row: title, badge, vote bar, buttons (memoised)
resources/js/components/meeple-votes.tsx	Vote bar packed with meeples
resources/js/components/meeple.tsx	Meeple SVG
resources/js/components/spray-painted-title.tsx	Spray-painted ”Game Library” heading on bricks
resources/js/hooks/use-flip-animation.ts	FLIP re-sort animation
resources/js/types/games.ts	Game and page prop types
Dockerfile, .dockerignore, docker-compose.yml	Production, dev and Railway images
docker/entrypoint.sh, docker/dev-entrypoint.sh, docker/railway-start.sh	Container start scripts (key, migrate, optimize)
docker/nginx.conf, docker/nginx-http-auth.sh	nginx config and optional basic auth
.railway/railway.ts	Railway infrastructure-as-code
readme.md, docs/*.md	Project documentation

composer.lock and package-lock.json are produced by Composer and npm, not written by hand.

Starter-kit files to edit

File	Change
database/migrations/0001_01_01_000000_create_users_table.php	Drop email, email_verified_at and password_reset_tokens; add unique username
app/Models/User.php	Username instead of email; add votes(), dailyActions(), localToday(); timezone fillable
database/factories/UserFactory.php	Generate username instead of email
database/seeders/DatabaseSeeder.php	Test user gets 'username' => 'test'
app/Actions/Fortify/CreateNewUser.php	Username rules, timezone:all validation, save timezone
app/Concerns/ProfileValidationRules.php	Username rules (3–30, alpha_dash:ascii, unique, lowercase)
app/Http/Requests/Settings/ProfileUpdateRequest.php	Validate username instead of email
app/Http/Controllers/Settings/ProfileController.php	Drop mustVerifyEmail, status and the email-reset logic; just update($request->validated())
app/Providers/FortifyServiceProvider.php	Remove reset-password views; add the 20/min per-IP login limit
config/fortify.php	'username' => 'username', 'lowercase_usernames' => true; drop reset and verification features
app/Providers/AppServiceProvider.php	Bind ApiKeyProvider and EssenseClient singletons; TrustProxies::at() from config
app/Http/Middleware/HandleAppearance.php	Accept only light, dark, system from the cookie
app/Http/Middleware/HandleInertiaRequests.php	Remove the sidebarOpen shared prop
bootstrap/app.php	Append the CSP middleware; trustHosts() when proxies are trusted
config/app.php	Add trusted_proxies and content_security_policy
config/services.php	Add the essense block (url, key)
config/inertia.php	'enabled' => (bool) env('INERTIA_SSR_ENABLED', true)
.env.example	Add ESSENSE_API_URL, API_KEY_ESSENCE, REMOVE_LIMITS, HTTP_AUTH_*, TRUSTED_PROXIES
routes/web.php	Dashboard via GameController, the four game routes, throttle
routes/settings.php	Drop the verified middleware group

File	Change
`phpunit.xml`	Add `INERTIA_SSR_ENABLED=(false)`, `ESSENSE_API_URL=https://essense.test`, `API_KEY_ESSENCE=test-key`, `REMOVE_LIMITS=(false)`
`tests/TestCase.php`	`Http::preventStrayRequests()` in `setUp()`
`tests/Pest.php`	Pest bootstrap (generated by the kit; edit to match the repo)
`tests/Feature/Auth/*`, `DashboardTest.php`, `Settings/*Test.php`	Username instead of email in existing tests
`resources/views/app.blade.php`	`color-scheme` meta and CSS; `nonce` on the inline theme script
`resources/js/app.tsx`	App name GameVote; toasts for `networkError` and `httpException`
`resources/js/layouts/app-layout.tsx`	Single column under `AppTopBar`
`resources/js/pages/dashboard.tsx`	The game library page
`resources/js/pages/welcome.tsx`	GameVote landing page
`resources/js/pages/auth/register.tsx`, `login.tsx`	Username field; hidden timezone field on register
`resources/js/pages/settings/profile.tsx`, `security.tsx`, `appearance.tsx`	Username instead of email; new layout
`resources/js/components/app-logo-icon.tsx`	Meeple logo
`resources/js/types/*.ts`	`User` has `username`, no `email`; drop sidebar and breadcrumb types
`resources/css/app.css`	Spray paint colour (`--spray-rgb`), brick backgrounds, row glow and dimming
`vite.config.ts`	`bunny('Rubik Spray Paint')` font; exclude `docs/ai-transcript.md` from formatting
`package.json`, `composer.json`	Real `vite@^8.0.0` (no `overrides` block), scripts call `vite`, PHP `^8.5`
`.github/workflows/tests.yml`	PHP 8.5
`.gitignore`, `.editorconfig`	Minor housekeeping

Some kit files also carry `@chisel` marker comments that `laravel new` strips automatically, so those lines are not edits you make.

Build order

Nine phases, each ending with something you can run and see. You build the app the way you'd explain it: accounts first, then the page frame, then the data it stores, then the logic, then the polish.

Phase	You build	You can see
1. Setup	A fresh starter-kit app	Default kit login and sign-up
2. Username accounts	Username login, timezone at sign-up	Register and log in with a username
3. One-column UI	Top bar, single-column layout, placeholder list	GameVote's page frame with sample games
4. Data structure	Votes, daily ledger and API key tables	Tables in the database
5. Backend processing	API client, daily rule, actions, routes	Real games: add, vote, remove
6. Refining the UI	Meeple vote bars, spray-paint title, animations	The finished library page
7. Security hardening	CSP, trusted proxies, cookie checks	Same app, locked down
8. Tests	Fake API and the full test suite	`php artisan test` passing
9. Docker and docs	Containers, Railway, readme	`docker compose up`

The code sections hold the final text of every file. Each phase says which files to create or copy, and gives inline code for the two temporary versions of the dashboard used in phases 3 and 5.

Phase 1: Setup

```
laravel new gamevote --react --database=sqlite --pest --no-boost
cd gamevote
git checkout -b laravel-react-setup
```

Run this in a macOS, Linux or WSL terminal. The installer's prompts work properly there; on native Windows they often don't.

When the installer asks **Which authentication features would you like to enable?**, use the arrow keys and Space so only **Registration** is checked, then press Enter:

```
◻ Email verification
◼ Registration
◻ Two-factor authentication
◻ Passkeys
◻ Password confirmation
```

That gives plain username-and-password accounts: sign-up, login and logout, and a settings page for changing your password. The installer deletes the code for the four other features: pages, components, tests, the two-factor and passkey migrations, and the matching traits on `User`.

This differs from the GameVote repo, which also keeps two-factor authentication, passkeys and password confirmation. The guide follows the simpler version: phase 2 and the code sections show the registration-only versions of the files that differ, so copying those files straight from the repo won't match.

If `npm install` fails with "Invalid comparator: npm:@voidzero-dev/vite-plus-core", your npm is too old: check `npm -v` is 10 or higher (see Prerequisites), delete the half-built folder, and run `laravel new` again.

`--no-boost` skips the installer's own Boost setup, so you add Boost as a separate step instead. That keeps `laravel new` to fewer prompts and lets you run Boost's agent picker in whichever terminal handles it best:

```
composer require laravel/boost --dev
php artisan boost:install
```

To do it in one go instead, drop `--no-boost` from `laravel new`, let the installer set up Boost, and skip those two commands.

- ☐ `package.json`: real `vite@^8.0.0` instead of the `vite-plus-core` alias, delete the `overrides` block, dev/build call `vite` (`check`/`check:fix` stay on `vp`), remove `@rollup/*` optional deps; keep the kit's `vite-plus` version. Then run `npm install` so the real Vite replaces the alias (without it, `npm run build` fails with *vite: not found*), and check with `npx vite --version`
- ☐ Delete `pnpm-workspace.yaml`; run `npm install` and confirm no peer warnings
- ☐ `composer.json`: change `"php": "^8.3"` to `"php": "^8.5"`. That's the only hand edit: `--pest` and the Boost command add their packages, and the `scripts` block already matches the repo (the installer deletes its own `install:features` hooks once the prompts are answered)
- ☐ `.github/workflows/tests.yml`: `php-version: '8.5'` in the Setup PHP step. Newer copies of the kit already say 8.5; only change it if yours says 8.3. Do the same check on `composer.json`'s `"php"` line

Phase 2: Username accounts

```
php artisan make:migration add_timezone_to_users_table --table=users
```

Why this phase exists: GameVote identifies people by a username, and nothing in the app ever sends email. So email comes out everywhere: the database column, the login and sign-up forms, validation, and the two features that only work with email (password reset and email verification). The browser's timezone is also saved at sign-up, because the daily rule (phase 5) needs each user's local midnight.

These edits assume you checked only **Registration** in phase 1. The installer already removed everything for email verification, two-factor authentication, passkeys and password confirmation, including the verify-email, two-factor and confirm-password pages, the passkey components, their tests and migrations, and the `MustVerifyEmail`, two-factor and passkey traits on `User`. The

sections below cover only what's left: swapping email for username, adding the timezone, and removing password reset. Work through them in order, then run the check at the end.

2a. Users table: `database/migrations/0001_01_01_000000_create_users_table.php`

Why: the `users` table needs a unique `username` instead of `email`, and `password_reset_tokens` only exists to support password reset by email.

In `up()`, find:

```
$table->string('email')->unique();
$table->timestamp('email_verified_at')->nullable();
```

Replace both lines with:

```
$table->string('username')->unique();
```

Still in `up()`, delete this whole block:

```
Schema::create('password_reset_tokens', function (Blueprint $table) {
    $table->string('email')->primary();
    $table->string('token');
    $table->timestamp('created_at')->nullable();
});
```

In `down()`, delete:

```
Schema::dropIfExists('password_reset_tokens');
```

2b. Timezone column: the migration you just generated

Why: the daily rule resets at each user's local midnight, so every user needs a stored timezone. It's a separate migration because it was added after the users table existed; see the repo's history.

Open `database/migrations/<timestamp>_add_timezone_to_users_table.php`. Inside `up()`, in the `Schema::table('users', ...)` closure, add:

```
$table->string('timezone')->default('UTC')->after('username');
```

Inside `down()`, in its closure, add:

```
$table->dropColumn('timezone');
```

2c. Fake users: `database/factories/UserFactory.php`

Why: tests and the seeder create users through this factory, so it must fill `username` instead of the deleted email columns.

In `definition()`, find:

```
'email' => fake()->unique()->safeEmail(),
'email_verified_at' => now(),
```

Replace both lines with:

```
'username' => fake()->unique()->lexify('user_????????'),
```

Then delete the whole `unverified()` method (its doc comment included). It sets `email_verified_at`, which no longer exists.

2d. Seed user: `database/seeders/DatabaseSeeder.php`

Why: `php artisan db:seed` creates a test user, and it can't set an email any more.

Find:

```
'email' => 'test@example.com',
```

Replace with:

```
'username' => 'test',
```

2e. Fortify settings: `config/fortify.php`

Why: Fortify is the package behind login and registration. This tells it which form field identifies a user, and turns off the email-only features so their routes stop existing.

Find:

```
'username' => 'email',
```

Replace with:

```
'username' => 'username',
```

Check that this line is already present (it is in the current kit). It makes Chuck and chuck the same account:

```
'lowercase_usernames' => true,
```

In the `'features'` array, delete this line (the installer already removed `Features::emailVerification()`):

```
Features::resetPasswords(),
```

That leaves `Features::registration()` as the only feature.

2f. Fortify wiring: `app/Providers/FortifyServiceProvider.php`

Why: this file connects Fortify to your actions and React pages. It still points at the password-reset action and the reset and verify pages you're about to delete. It's also where the login rate limit lives.

Delete these two `use` lines at the top:

```
use App\Actions\Fortify\ResetUserPassword;
use Laravel\Fortify\Features;
```

Keep `use Illuminate\Validation\Rules\Password;`. The sign-up view still calls `Password::defaults()`. If your IDE's "optimize imports" removes it after you delete the reset views below, the sign-up page fails with *Class "App\Providers\Password" not found*.

In `configureActions()`, delete:

```
Fortify::resetUserPasswordsUsing(ResetUserPassword::class);
```

In `configureViews()`, the login view passes a `canResetPassword` flag, and two views after it serve the password-reset pages. (The installer already removed the `verifyEmailView`.) Delete the `canResetPassword` line and both reset views, so this:

```
Fortify::loginView(fn (Request $request) => Inertia::render('auth/login', [
    'canResetPassword' => Features::enabled(Features::resetPasswords()),
    'status' => $request->session()->get('status'),
]));

Fortify::resetPasswordView(fn (Request $request) => Inertia::render('auth/reset-password', [
    'email' => $request->email,
    'token' => $request->route('token'),
    'passwordRules' => Password::defaults()->toPasswordRulesString(),
]));

Fortify::requestPasswordResetLinkView(fn (Request $request) => Inertia::render('auth/forgot-password', [
```

```
    'status' => $request->session()->get('status'),
]));
```

becomes:

```
Fortify::loginView(fn (Request $request) => Inertia::render('auth/login', [
    'status' => $request->session()->get('status'),
]));
```

In `configureRateLimiting()`, inside the `'login'` limiter, find:

```
return Limit::perMinute(5)->by($throttleKey);
```

Replace with:

```
return [
    Limit::perMinute(5)->by($throttleKey),
    // Also cap each IP across usernames, so one address can't try a password against many accounts.
    Limit::perMinute(20)->by('ip:'.$request->ip()),
];
```

The kit only limits each username-plus-IP pair, so one address could try a password against every account. The second limit caps each IP at 20 attempts a minute across all usernames.

2g. Delete the email-only files

Why: with password reset turned off, nothing uses these, and leaving them would break the type check and test run. (The installer already deleted the verify-email page and the two verification tests.)

- `app/Actions/Fortify/ResetUserPassword.php`
- `resources/js/pages/auth/forgot-password.tsx`
- `resources/js/pages/auth/reset-password.tsx`
- `tests/Feature/Auth/PasswordResetTest.php`

2h. User model: `app/Models/User.php`

Why: the model must match the new columns. (The installer already removed `MustVerifyEmail` and the two-factor and passkey traits, but it leaves the two-factor columns in the doc comment and the `Hidden` list.)

In the doc comment above the class, find:

```
 * @property string $email
 * @property Carbon|null $email_verified_at
```

Replace with:

```
 * @property string $username
 * @property string $timezone
```

Find:

```
#[Fillable(['name', 'email', 'password'])]
```

Replace with:

```
#[Fillable(['name', 'username', 'timezone', 'password'])]
```

In the same doc comment, delete the three two-factor lines, since those columns no longer exist:

```
 * @property string|null $two_factor_secret
 * @property string|null $two_factor_recovery_codes
 * @property Carbon|null $two_factor_confirmed_at
```

Find:

```
#[Hidden(['password', 'two_factor_secret', 'two_factor_recovery_codes', 'remember_token'])]
```

Replace with:

```
#[Hidden(['password', 'remember_token'])]
```

In `casts()`, delete:

```
'email_verified_at' => 'datetime',
```

The `votes()`, `dailyActions()` and `localToday()` methods come in phase 4, once the `Vote` and `DailyAction` models exist.

2i. Sign-up: app/Actions/Fortify/CreateNewUser.php

Why: Fortify calls this when someone registers. It must save the username, and validate and save the timezone the sign-up form sends.

In the `Validator::make(...)` rules, after the `'password'` line, add:

```
'timezone' => ['nullable', 'string', 'timezone:all'],
```

`timezone:all` rejects anything that isn't a real timezone name, so nobody can post junk.

In `User::create([...])`, find:

```
'email' => $input['email'],
```

Replace with:

```
'username' => $input['username'],
'timezone' => $input['timezone'] ?? config('app.timezone'),
```

If the browser sends no timezone, the user gets the app's timezone (UTC).

2j. Username rules: app/Concerns/ProfileValidationRules.php

Why: sign-up and the profile page share these rules. Usernames need their own rules: 3–30 characters, only letters, numbers, dashes and underscores, and unique.

In `profileRules()`, find:

```
'email' => $this->emailRules($userId),
```

Replace with:

```
'username' => $this->usernameRules($userId),
```

Then replace the whole `emailRules()` method, doc comment included, with:

```
/**
 * Get the validation rules used to validate usernames.
 *
 * @return array<int, ValidationRule|array<mixed>|string>
 */
protected function usernameRules(?int $userId = null): array
{
    return [
        'required',
        'string',
        'min:3',
        'max:30',
        'alpha_dash:ascii',
        $userId === null
            ? Rule::unique(User::class)
            : Rule::unique(User::class)->ignore($userId),
    ];
}
```

The `ignore($userId)` part lets someone save their profile without changing their username: their own row doesn't count as a duplicate.

2k. Profile update: app/Http/Requests/Settings/ProfileUpdateRequest.php

Why: Fortify lowercases usernames at sign-up and login, but the profile page doesn't go through Fortify. Without this, someone could rename themselves Chuck while another user holds chuck.

Add two use lines after the existing ones:

```
use Illuminate\Support\Str;
use Laravel\Fortify\Fortify;
```

Add this method after `rules()`, inside the class:

```
/**
 * Lowercase the username to match how Fortify stores it at registration and looks it up at login.
 */
protected function prepareForValidation(): void
{
    if (config('fortify.lowercase_usernames') && is_string($this->input(Fortify::username()))) {
        $this->merge([
            Fortify::username() => Str::lower($this->input(Fortify::username())),
        ]);
    }
}
```

2l. Profile controller: app/Http/Controllers/Settings/ProfileController.php

Why: the kit's profile page checks verification status and clears `email_verified_at` when the email changes. None of that applies now.

Delete these two use lines:

```
use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Http\Request;
```

Replace the `edit()` method with:

```
public function edit(): Response
{
    return Inertia::render('settings/profile');
}
```

In `update()`, find:

```
$request->user()->fill($request->validated());

if ($request->user()->isDirty('email')) {
    $request->user()->email_verified_at = null;
}

$request->user()->save();
```

Replace with:

```
$request->user()->update($request->validated());
```

Leave `destroy()` as it is.

2m. Settings routes: routes/settings.php

Why: the kit puts account deletion and security settings behind a `verified` group, meaning "email verified". With verification gone, that group would lock everyone out, so merge it into the plain `auth` group above it.

Find these three lines:

```
});

Route::middleware(['auth', 'verified'])->group(function () {
```

Delete them. The routes that were in the second group now sit in the first group.

2n. User type: `resources/js/types/auth.ts`

Why: TypeScript's `User` type must match what the server sends, or every `auth.user.username` is an error.

In `export type User`, replace `email: string;` with `username: string;`, and delete `email_verified_at: string | null;`.

One kit component still reads `user.email`: `resources/js/components/user-info.tsx`. It's deleted in phase 3 with the old layout. Until then, change `{user.email}` to `{user.username}` there so `npm run types:check` passes.

2o. Login page: `resources/js/pages/auth/login.tsx`

Why: the form must post a field named `username`, because that's what Fortify now expects. The "Forgot your password?" link goes, since reset is gone.

Delete this import:

```
import { request } from '@/routes/password';
```

In `type Props`, delete `canResetPassword: boolean;`. Change the function line to:

```
export default function Login({ status }: Props) {
```

Replace the email field (the `<Label htmlFor="email">` through `<InputError message={errors.email} />`) with:

```
<Label htmlFor="username">Username</Label>
<Input
    id="username"
    type="text"
    name="username"
    required
    autoFocus
    tabIndex={1}
    autoComplete="username"
    autoCapitalize="none"
    spellCheck={false}
    placeholder="Username"
/>
<InputError message={errors.username} />
```

Replace the password label row (the `<div className="flex items-center">` that holds the label and the "Forgot your password?" link) with just:

```
<Label htmlFor="password">Password</Label>
```

At the bottom, change the description to:

```
description: 'Enter your username and password below to log in',
```

`autoCapitalize="none"` and `spellCheck={false}` stop phones from capitalising or autocorrecting usernames.

2p. Sign-up page: `resources/js/pages/auth/register.tsx`

Why: same username field swap, plus a hidden field that sends the browser's timezone to `CreateNewUser`.

Right after the opening `<>` inside the `<Form>` render function, add:

```
<input
    type="hidden"
    name="timezone"
    value={
        Intl.DateTimeFormat().resolvedOptions().timeZone
    }
/>
```

Replace the email field (`<Label htmlFor="email">` through `<InputError message={errors.email} />`) with:

```
<Label htmlFor="username">Username</Label>
<Input
    id="username"
    type="text"
    required
    tabIndex={2}
    autoComplete="username"
    autoCapitalize="none"
    spellCheck={false}
    name="username"
    placeholder="Username"
/>
<InputError message={errors.username} />
```

2q. Profile page: `resources/js/pages/settings/profile.tsx`

Why: the page still shows an email field. (The installer already removed the verification props, the `Link` and `send` imports, and the unverified-email message.)

Delete this import:

```
import { edit } from '@/routes/profile';
```

If the function signature was left as `export default function Profile(` with `) {` on the next line, tidy it to:

```
export default function Profile() {
```

In the `<Heading>`, change the description to `"Update your name and username"`.

Replace the email field (`<Label htmlFor="email">` through its `<InputError ... message={errors.email} />`) with:

```
<Label htmlFor="username">Username</Label>

<Input
    id="username"
    className="mt-1 block w-full"
    defaultValue={auth.user.username}
    name="username"
    required
    autoComplete="username"
    autoCapitalize="none"
    spellCheck={false}
    placeholder="Username"
/>

<InputError
    className="mt-2"
    message={errors.username}
/>
```

Finally, delete the `Profile.layout = { breadcrumbs: [...] };` block at the bottom; breadcrumbs go away with the new layout in phase 3.

2r. Existing tests

Why: the kit's tests log in and register with `email`, so they fail until they send `username`.

- `tests/Feature/Auth/AuthenticationTest.php`: replace every `'email' => $user->email` with `'username' => $user->username`, and in the rate-limit test change `[$user->email, '127.0.0.1']` to `[$user->username, '127.0.0.1']`. If your copy still has a two-factor test (one that calls `withTwoFactor()`), delete it and the `use Laravel\Fortify\Features;` line; newer kits remove it during install, so it may not be there.
- `tests/Feature/Auth/RegistrationTest.php`: replace `'email' => 'test@example.com'` with `'username' => 'test_user'`.
- `tests/Feature/Settings/ProfileUpdateTest.php`: in `test_profile_information_can_be_updated`, send `'username' => 'test_user'` instead of an email, assert `$this->assertSame('test_user', $user->username);`, and delete the `assertNull($user->email_verified_at)` line. Delete the whole `test_email_verification_status_is_unchanged_when_the_email_address_is_unchanged` test.

`tests/Feature/Settings/SecurityTest.php` still has two two-factor tests the installer left in. They skip themselves when two-factor is off, so leave them.

Those are the minimum changes to get green. GameVote's versions also add tests for the new rules: lowercase usernames, invalid usernames, timezone capture, and email routes returning 404. The full files are on the Tests code section; copy them in to match the repo.

2s. Check it works

```
php artisan migrate:fresh --seed
php artisan test
npm run types:check
```

`migrate:fresh` rebuilds the database from the edited migrations, `--seed` proves the factory and seeder work, and the two checks confirm nothing still refers to email. Then run `composer dev`, open the app, and register an account with a username.

Phase 3: One-column UI

Goal: replace the kit's sidebar dashboard with GameVote's page frame (a sticky top bar over one centred column) and a placeholder game list. No backend yet; the list is hard-coded so you can judge the layout on its own.

3a. Delete the sidebar layout. These 16 kit files only serve the sidebar and header layout:

- `resources/js/components/`: `app-content.tsx`, `app-header.tsx`, `app-logo.tsx`, `app-shell.tsx`, `app-sidebar-header.tsx`, `app-sidebar.tsx`, `breadcrumbs.tsx`, `nav-footer.tsx`, `nav-main.tsx`, `nav-user.tsx`, `user-info.tsx`, `user-menu-content.tsx`
- `resources/js/hooks/`: `use-initials.tsx`, `use-mobile-navigation.ts`
- `resources/js/layouts/app/`: `app-header-layout.tsx`, `app-sidebar-layout.tsx`

Deleting `user-info.tsx` also retires the temporary `{user.username}` fix from phase 2r.

3b. Add the new frame. Copy these from the Frontend code section:

- `resources/js/components/meeple.tsx` (new): the meeple SVG, used as the logo now and in the vote bars later
- `resources/js/components/app-logo-icon.tsx` (edit): the logo becomes a meeple
- `resources/js/components/app-top-bar.tsx` (new): logo, `@username`, settings icon, Log out
- `resources/js/layouts/app-layout.tsx` (edit): top bar over a `max-w-3xl` column
- `resources/js/pages/welcome.tsx` (edit): the GameVote landing page
- `resources/js/pages/settings/appearance.tsx` and `security.tsx` (edit): drop their breadcrumbs, which the new layout doesn't show

3c. Tidy the types. From the Frontend code section: `resources/js/types/games.ts` (new: the `Game` and `DailyAllowance` shapes the server will send), plus edits to `types/index.ts`, `types/ui.ts`, `types/navigation.ts` and `types/global.d.ts` (sidebar and breadcrumb types go). On the server, remove the `sidebarOpen` line from `app/Http/Middleware/HandleInertiaRequests.php` (Backend code section).

3d. Placeholder library page. Replace `resources/js/pages/dashboard.tsx` with this temporary version. Phase 5 swaps it for one fed by real data:

```
import { Head } from '@inertiajs/react';
import type { Game } from '@/types';

// Sample data until the backend exists (phase 5).
const sampleGames: Game[] = [
    { id: 1, name: 'Catan', votes: 5, voted: false },
    { id: 2, name: 'Codenames', votes: 3, voted: true },
    { id: 3, name: 'Mario Kart 8 Deluxe', votes: 0, voted: false },
];

export default function Dashboard() {
    return (
        <>
            <Head title="Game library" />
            <div className="flex flex-col gap-6">
                <h1 className="text-2xl font-semibold">Game Library</h1>
                <ol className="divide-y rounded-xl border">
                    {sampleGames.map((game, index) => (
                        <li key={game.id} className="flex items-center gap-3 p-4">
                            <span className="w-6 text-right text-sm text-muted-foreground tabular-nums">
                                {index + 1}
                            </span>
                            <span className="flex-1 truncate font-medium">
                                {game.name}
                            </span>
                            <span className="text-sm tabular-nums">
                                {game.votes} votes
                            </span>
                        </li>
                    ))}
                </ol>
            </div>
        </>
    );
}
```

Check it:

```
npm run types:check
composer dev
```

Log in: you should see the top bar, one centred column and three sample games. Open Settings to confirm the profile, security and appearance pages still work.

Phase 4: Data structure

Goal: create the tables for what the external API can't store: who voted for what, the one-action-per-day ledger, and the API key. No behaviour yet.

Why these tables: the API owns games and vote totals. Locally you need `votes` (so people can only remove their own vote), `daily_actions` (the daily rule) and a one-row `api_credentials` table. The unique indexes carry the rules: one vote per game per user, one action per user per day, enforced by the database even when two requests race.

```
php artisan make:enum Enums/DailyActionType --string
php artisan make:model Vote -mf
php artisan make:model DailyAction -mf
php artisan make:model ApiCredential -m
```

Fill each generated file from the Backend code section:

- `app/Enums/DailyActionType.php`: the `Vote` and `AddGame` cases
- The three migrations (your timestamps will differ from the tab's file names): `create_votes_table`, `create_daily_actions_table`, `create_api_credentials_table`
- Models: `app/Models/Vote.php`, `DailyAction.php`, `ApiCredential.php`
- Factories: `database/factories/VoteFactory.php`, `DailyActionFactory.php`

Then add the relationships to `app/Models/User.php`. Add two `use` lines:

```
use Carbon\CarbonImmutable;
use Illuminate\Database\Eloquent\Relations\HasMany;
```

and these methods after `casts()`:

```
/**
 * Get the votes the user has cast.
 *
 * @return HasMany<Vote, $this>
 */
public function votes(): HasMany
{
    return $this->hasMany(Vote::class);
}

/**
 * Get the daily actions the user has taken.
 *
 * @return HasMany<DailyAction, $this>
 */
public function dailyActions(): HasMany
{
    return $this->hasMany(DailyAction::class);
}

/**
 * Get the current calendar date in the user's timezone.
 */
public function localToday(): CarbonImmutable
{
    return now($this->timezone)->startOfDay();
}
```

Check it:

```
php artisan migrate
php artisan tinker --execute="dump(App\Models\User::first()?->localToday())"
```

The migration should create `votes`, `daily_actions` and `api_credentials`, and tinker should print today's date in your account's timezone.

Phase 5: Backend processing

Goal: talk to the real API, enforce the daily rule, and wire the routes, so the library page shows real games you can add, vote for and remove.

5a. Configuration. From the Backend code section:

- `config/games.php` (new): the `REMOVE_LIMITS` testing switch
- `config/services.php` (edit): the `essense` block (API URL and optional key)
- `.env.example` (edit): copy the new keys into your own `.env` too

5b. API client. Everything that talks HTTP to the Essense API lives in four classes:

```
php artisan make:class Services/Essense/EssenseApiException
php artisan make:class Services/Essense/GameData
php artisan make:class Services/Essense/ApiKeyProvider
php artisan make:class Services/Essense/EssenseClient
```

Fill them from the Backend code section. Then register the client in `app/Providers/AppServiceProvider.php`, which builds both from config. Add three `use` lines:

```
use App\Actions\Games\ResetLocalLibrary;
use App\Services\Essense\ApiKeyProvider;
use App\Services\Essense\EssenseClient;
```

and make `register()` read:

```
public function register(): void
{
    $this->app->singleton(ApiKeyProvider::class, fn () => new ApiKeyProvider(
        baseUrl: config('services.essense.url'),
        configuredKey: config('services.essense.key'),
        resetLocalLibrary: $this->app->make(ResetLocalLibrary::class),
    ));

    $this->app->singleton(EssenseClient::class, fn () => new EssenseClient(
        baseUrl: config('services.essense.url'),
        keys: $this->app->make(ApiKeyProvider::class),
    ));
}
```

5c. Daily rule and use cases. One class owns the rule; one action class per use case:

```
php artisan make:class Services/DailyAllowance
php artisan make:exception GameActionException
php artisan make:class Actions/Games/AddGame
php artisan make:class Actions/Games/CastVote
php artisan make:class Actions/Games/RemoveVote
php artisan make:class Actions/Games/RemoveGame
php artisan make:class Actions/Games/ResetLocalLibrary
```

Fill all seven from the Backend code section. Now the client can run:

```
php artisan tinker --execute="dump(app(App\Services\Essense\EssenseClient::class)->games())"
```

That should print an empty list. The first call also generates an API key and stores it, encrypted, in `api_credentials`.

5d. HTTP layer.

```
php artisan make:request StoreGameRequest
php artisan make:controller GameController
php artisan make:controller GameVoteController
php artisan make:command SeedGames --command=games:seed
```

Fill them from the Backend code section, then replace `routes/web.php` with the tab's version: the dashboard now comes from `GameController@index`, plus the add, remove, vote and remove-vote routes. Check the routes and add some sample games:

```
php artisan route:list --except-vendor
php artisan games:seed --count=5
```

5e. Wire the page to real data. Copy `resources/js/components/add-game-form.tsx` (new) from the Frontend code section, then replace the placeholder `dashboard.tsx` with this working, unstyled version. Phase 6 replaces it again with the finished one:

```
import { Form, Head, router } from '@inertiajs/react';
import GameController from '@/actions/App/Http/Controllers/GameController';
import GameVoteController from '@/actions/App/Http/Controllers/GameVoteController';
import AddGameForm from '@/components/add-game-form';
import { Button } from '@/components/ui/button';
import type { DailyAllowance, Game } from '@/types';

type Props = {
    games: Game[] | null;
```

```
    libraryError: string | null;
    allowance: DailyAllowance;
    timezone: string;
};

export default function Dashboard({ games, libraryError, allowance }: Props) {
    return (
        <>
            <Head title="Game library" />
            <div className="flex flex-col gap-6">
                <h1 className="text-2xl font-semibold">Game Library</h1>

                <p className="text-sm text-muted-foreground">
                    {!allowance.enforced
                        ? 'Daily limits are off.'
                        : allowance.available
                          ? 'You have one action today: vote or add a game.'
                          : 'You have used today’s action.'}
                </p>

                <AddGameForm
                    disabled={!allowance.available}
                    limited={allowance.enforced}
                />

                {libraryError !== null || games === null ? (
                    <div className="rounded-xl border border-destructive p-4 text-sm">
                        <p>{libraryError ?? 'The game library could not be loaded.'}</p>
                        <Button
                            variant="outline"
                            size="sm"
                            className="mt-2"
                            onClick={() => router.reload()}
                        >
                            Try again
                        </Button>
                    </div>
                ) : games.length === 0 ? (
                    <p className="text-sm text-muted-foreground">
                        No games yet. Suggest the first one above.
                    </p>
                ) : (
                    <ol className="divide-y rounded-xl border">
                        {games.map((game, index) => (
                            <li key={game.id} className="flex items-center gap-3 p-4">
                                <span className="w-6 text-right text-sm text-muted-foreground
↪  tabular-nums">
                                    {index + 1}
                                </span>
                                <span className="flex-1 truncate font-medium">
                                    {game.name}
                                </span>
                                <span className="text-sm tabular-nums">
                                    {game.votes} votes
                                </span>
                                {game.voted ? (
                                    <Form
                                        {...GameVoteController.destroy.form(game.id)}
                                        options={{ preserveScroll: true }}
                                    >
                                        {({ processing }) => (
                                            <Button type="submit" variant="outline" size="sm"
↪  disabled={processing}>
                                                Remove vote
                                            </Button>
                                        )}
                                    </Form>
                                ) : (
                                    <Form
                                        {...GameVoteController.store.form(game.id)}
```

```
                                        options={{ preserveScroll: true }}
                                    >
                                        {({ processing }) => (
                                            <Button
                                                type="submit"
                                                size="sm"
                                                disabled={!allowance.available || processing}
                                            >
                                                Vote
                                            </Button>
                                        )}
                                    </Form>
                                )}
                                <Form
                                    {...GameController.destroy.form(game.id)}
                                    options={{ preserveScroll: true }}
                                >
                                    {({ processing }) => (
                                        <Button type="submit" variant="ghost" size="sm"
↪  disabled={processing}>
                                            Remove
                                        </Button>
                                    )}
                                </Form>
                            </li>
                        ))}
                    </ol>
                )}
            </div>
        </>
    );
}
```

Check it: run `composer dev` and log in. The seeded games appear, sorted by votes. Vote for one, and the Vote buttons disable for the rest of the day. Remove that vote the same day, and you get the action back. A second add on the same day is refused under the field. To try things repeatedly, set `REMOVE_LIMITS=true` in `.env` (ignored in production).

Phase 6: Refining the UI

Goal: turn the working page into the finished one. The backend doesn't change.

Copy from the Frontend code section:

- New components: `meeple-votes.tsx` (each row's bar, packed with meeples), `spray-painted-title.tsx` (the brick-wall heading), `daily-allowance-notice.tsx` (today's allowance as an alert), `game-list-item.tsx` (one ranked row, with the remove-game confirmation dialog)
- New hook: `resources/js/hooks/use-flip-animation.ts` (slides rows to their new rank after a vote)
- `resources/js/pages/dashboard.tsx`: the final version, replacing phase 5's
- `resources/css/app.css`: spray-paint colours, brick backgrounds, the row glow, dimming while a request is pending
- `vite.config.ts`: loads the Rubik Spray Paint font
- `resources/js/app.tsx`: toasts when the server can't be reached or returns an error
- `resources/views/app.blade.php`: the `color-scheme` meta tag and CSS. The diff also adds a nonce to the inline script, which phase 7's security policy uses; it's harmless before then.

Check it: `composer dev`, then vote and watch the row slide to its new position and glow. Try light and dark mode under Settings, then Appearance, and narrow the window to phone width.

Phase 7: Security hardening

Goal: production protections that don't change what users see.

```
php artisan make:middleware AddContentSecurityPolicy
```

From the Backend code section:

- `app/Http/Middleware/AddContentSecurityPolicy.php`: a nonce-based Content-Security-Policy, so the browser only runs your own scripts
- `config/app.php` (edit): the `trusted_proxies` and `content_security_policy` settings
- `bootstrap/app.php` (edit): adds the CSP middleware, and only accepts your own host when proxies are trusted
- `app/Http/Middleware/HandleAppearance.php` (edit): only accepts `light`, `dark` or `system` from the unencrypted theme cookie
- `app/Providers/AppServiceProvider.php`: add `use Illuminate\Http\Middleware\TrustProxies;` and, in `configureDefaults()` after `Date::use(...)`, the trusted-proxies block:

```
$trustedProxies = config('app.trusted_proxies');

if (is_string($trustedProxies) && $trustedProxies !== '') {
    TrustProxies::at($trustedProxies === '*' ? '*' : array_map('trim', explode(',', $trustedProxies)));
}
```

Check it: the policy is off in local development (Vite's dev server would trip it). To see it, set `CONTENT_SECURITY_POLICY=true` in `.env`, run `npm run build` and `php artisan serve`, and look for a `Content-Security-Policy` header and no console errors. Phase 8's tests also cover it.

Phase 8: Tests

Goal: prove the rules hold, without touching the real API.

From the Backend code section: `phpunit.xml` (points tests at a fake API host and pins `REMOVE_LIMITS` off) and `config/inertia.php` (lets tests turn off SSR). From the Tests code section: `tests/TestCase.php` (blocks real HTTP calls), `tests/Pest.php` (check it matches), `tests/Feature/DashboardTest.php` (edit), and `tests/FakeEssenseApi.php` (create it by hand: an in-memory copy of the API, quirks included).

Generate the test files, then fill each from the Tests code section:

```
php artisan make:test Games/AddGameTest
php artisan make:test Games/VoteTest
php artisan make:test Games/RemoveGameTest
php artisan make:test Games/GameLibraryTest
php artisan make:test Games/ApiKeyTest
php artisan make:test Games/EssenseClientTest
php artisan make:test Games/RemoveLimitsTest
php artisan make:test Games/SeedGamesCommandTest
php artisan make:test SecurityHardeningTest
php artisan make:test TrustedProxiesTest
```

Check it:

```
php artisan test
composer ci:check
```

`ci:check` runs formatting, lint, TypeScript, Pint, PHPStan and Pest together; it's the bar for every commit. If the newer Vite+ formatter flags files you copied, run `npm run check:fix`.

Phase 9: Docker, deployment and docs

Goal: one-command startup, and the write-up.

How the pieces fit. Locally, docker-compose runs nginx and PHP as two containers. On Railway they share one container, because Railway runs one container per service.

```
docker compose up:  browser → web (nginx, localhost:8000) → app (PHP-FPM, app:9000) → /data volume
 ↪  (SQLite, app.key)
Railway:            browser → Railway proxy (HTTPS) → one container: nginx :8080 → PHP-FPM
 ↪  127.0.0.1:9000 → /data volume
```

nginx serves files from `public/` itself (built JS and CSS, favicon) and hands every other request to PHP-FPM, which runs Laravel. SQLite and the generated APP_KEY live on a volume mounted at `/data`, so they survive rebuilds and restarts.

9a. Copy the files. From the Docker code tab: `Dockerfile`, `.dockerignore`, `docker-compose.yml`, everything in `docker/` (`entrypoint.sh`, `dev-entrypoint.sh`, `railway-start.sh`, `nginx-http-auth.sh`, `nginx.conf`), and `.railway/railway.ts` if you deploy to Railway. Make the scripts executable:

```
chmod +x docker/*.sh
```

9b. What each file does.

- `Dockerfile`: one multi-stage file that builds every image. Each stage starts `FROM` an earlier one; docker-compose picks a stage with `target:`, and Railway always builds the last one.
  - `node`: the official Node 22 image, used only to copy the `node` and `npm` binaries from.
  - `php-base`: PHP 8.5 FPM plus the `intl`, `zip` and `pcntl` extensions (SQLite and OPcache are built in) and Composer.
  - `toolchain`: `php-base` plus Node, because Wayfinder's Vite plugin runs `php artisan` while the front end builds.
  - `dev`: `toolchain` with PHP's development settings. Runs `php artisan serve` and `npm run dev`; your source is mounted in, not copied.
  - `build`: installs production Composer packages and `npm ci`, copies the code, runs `npm run build`, then deletes `node_modules`. The dependency files are copied before the code, so Docker reuses the cached install steps when only code changes.
  - `production`: `php-base` plus the built app from `build`, with PHP's production settings. No Node, no dev packages. Creates `/data`, runs `entrypoint.sh`, then starts PHP-FPM on port 9000.
  - `web`: nginx (Alpine) with `nginx.conf`, the basic-auth script, and only the `public/` folder from `build`.
  - `railway`: `production` plus nginx in the same container. Points nginx at `127.0.0.1:9000` instead of `app:9000`, sends nginx's logs to the container output, and starts with `railway-start.sh`.
- `.dockerignore`: what Docker leaves out of the build. `.env` (secrets must never be baked into an image), `node_modules` and `vendor` (rebuilt inside), built assets, the SQLite database, logs and caches, and the generated Wayfinder files. Keeps builds fast and images clean.
- `docker-compose.yml`: three services.
  - `app`: the `production` stage with production settings (`APP_ENV=production`, `APP_DEBUG=false`, logs to stderr). SQLite at `/data/database.sqlite` on the `sqlite-data` volume. A health check confirms PHP-FPM is listening on port 9000.
  - `web`: nginx on host port 8000 (`APP_PORT`). Waits until `app` is healthy, and passes `HTTP_AUTH_USER` / `HTTP_AUTH_PASSWORD` for the optional site password.
  - `dev` (only with `--profile dev`): hot reload. Mounts your project folder, runs as your user ID so files it creates belong to you, keeps `node_modules` in its own volume so Linux binaries don't clash with your machine's, and binds ports 8000 and 5173 to `127.0.0.1` only, since it runs with debug on.
- `docker/entrypoint.sh`: runs every time the production container starts, before PHP-FPM.
  - Uses `APP_KEY` from the environment, or generates one on first boot and saves it to `/data/app.key`. Keeping it on the volume matters: the encrypted generated API key and sessions depend on it.
  - Creates the SQLite file if missing and hands `/data` to the `www-data` user PHP runs as.
  - Runs `php artisan migrate --force` (`--force` because production asks for confirmation) and `php artisan optimize` (caches config, routes and views).
  - Ends with `exec "$@"`, which replaces the script with the container's command (`php-fpm`).

- `docker/dev-entrypoint.sh`: the dev equivalent. Copies `.env.example` to `.env`, runs `composer install` and `npm ci`, generates `APP_KEY`, creates the SQLite file and migrates. Each step only runs if it hasn't been done, so restarts are quick.
- `docker/nginx.conf`: the web server config.
  - Serves real files from `public/`; everything else goes to `index.php` over FastCGI on `app:9000`.
  - Adds the security headers from Phase 7, and HSTS only when the proxy reports HTTPS.
  - Caches fingerprinted `/build/assets/` files for a year (their names change whenever their contents do).
  - Blocks dotfiles such as `.env` and `.git`.
  - Raises the FastCGI buffers: Laravel's headers (the CSP, the asset preload `Link` header, cookies) outgrow nginx's default, which it would answer with a 502.
  - Turns basic auth off for `/up`, so health checks never need the site password.
- `docker/nginx-http-auth.sh`: runs before nginx starts. If `HTTP_AUTH_PASSWORD` is set, it hashes the password (SHA-512, read from stdin so it never shows in a process list), writes `.htpasswd`, and writes a small nginx snippet that turns basic auth on. Otherwise the snippet is empty and the site stays open.
- `docker/railway-start.sh`: the Railway container's start command. Runs `entrypoint.sh` and the auth script, switches nginx to Railway's `$PORT`, then starts PHP-FPM and nginx side by side. On shutdown it stops both gracefully; if either one crashes, it stops the other and exits, so Railway restarts the container.
- `.railway/railway.ts`: Railway's infrastructure as code. One `gamevote` service built from the Dockerfile's last stage, a 512 MB volume at `/data`, the `/up` health check, one replica (SQLite on a volume can't be shared), sleep when idle, and the environment: `APP_ENV`, `APP_URL` from Railway's domain, `TRUSTED_PROXIES=*`, `SESSION_SECURE_COOKIE=true`. `preserve()` leaves `API_KEY_ESSENCE` and the site password alone if you set them in Railway's dashboard. Preview with `railway config plan`, apply with `railway config apply`.

9c. Try it.

```
docker compose up --build                 # production-like: http://localhost:8000
docker compose --profile dev up dev       # hot reload: http://localhost:8000, Vite on 5173
docker compose down                       # stop (add -v to also delete the SQLite volume)
```

The production-like stack needs no setup. On first boot it generates and saves `APP_KEY`, creates and migrates the database, and, with no `API_KEY_ESSENCE`, the app gets its own API key. docker compose reads `${...}` values from the project's `.env`, so `API_KEY_ESSENCE`, `HTTP_AUTH_PASSWORD` or `APP_PORT` set there are passed through. `DOCKER_APP_KEY` is separate from your local `APP_KEY` on purpose, so the container never reuses your development key.

9d. Docs. Finally, add `readme.md` and the `docs/` folder from the repo, adjusted for the registration-only build.

Files to delete from the starter kit

These 25 kit files are gone from GameVote. Delete them in phases 2 and 3. Checking only Registration in phase 1 already removes verify-email.tsx, EmailVerificationTest.php and VerificationNotificationTest.php, along with all the two-factor, passkey and confirm-password files.

- **Email and password reset (phase 2):** `app/Actions/Fortify/ResetUserPassword.php`, `resources/js/pages/auth/forgot-password.tsx`, `reset-password.tsx`, `verify-email.tsx`, `tests/Feature/Auth/EmailVerificationTest.php`, `PasswordResetTest.php`, `VerificationNotificationTest.php`
- **Sidebar and header layout (phase 3):** `resources/js/components/app-content.tsx`, `app-header.tsx`, `app-logo.tsx`, `app-shell.tsx`, `app-sidebar-header.tsx`, `app-sidebar.tsx`, `breadcrumbs.tsx`, `nav-footer.tsx`, `nav-main.tsx`, `nav-user.tsx`, `user-info.tsx`, `user-menu-content.tsx`, `resources/js/hooks/use-initials.tsx`, `use-mobile-navigation.ts`, `resources/js/layouts/app/app-header-layout.tsx`, `app-sidebar-layout.tsx`

- **Tooling (phase 1):** `pnpm-workspace.yaml`, `README.md` (replaced by `readme.md`)

`chisel.php`, `chisel-paths.php` and `app/Console/Commands/InstallFeaturesCommand.php` exist only in the kit's source repo; `laravel new` removes them for you.

Verification

The rebuild is done when all of these pass.

```
npm ci                      # no peer warnings
npm audit                   # 0 vulnerabilities
php artisan migrate:fresh
php artisan route:list --except-vendor
php artisan test
composer ci:check           # oxlint/oxfmt, tsc, Pint, PHPStan, Pest
docker-compose up --build   # then open http://localhost:8000
docker-compose exec app php artisan games:seed --force
```

- ☐ Register with name, username and password
- ☐ Add a game; a second add the same day is refused
- ☐ Vote, then remove the vote the same day and get the action back
- ☐ Remove a game
- ☐ Light and dark themes both render the brick title correctly

Backend code

Every backend file, A to Z by path. New files show their full content; edited starter-kit files show a diff against the kit installed with only **Registration** checked, or the full file when most of it changed. Where this registration-only build differs from the GameVote repo (which also keeps two-factor and passkeys), the version here is the one to use.

`.editorconfig` (edit)

Replace the whole file with:

```
root = true

[*]
charset = utf-8
end_of_line = lf
indent_size = 4
indent_style = space
insert_final_newline = true
trim_trailing_whitespace = true

[*.md]
trim_trailing_whitespace = false
```

`.env.example` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -1,3 +1,3 @@
-APP_NAME=Laravel
+APP_NAME=GameVote
 APP_ENV=local
 APP_KEY=
@@ -64,2 +64,20 @@

 VITE_APP_NAME="${APP_NAME}"
+
+# Essense of Australia code challenge API.
+# Leave API_KEY_ESSENCE empty and the app generates its own key and stores it in the database.
+# Set it to use a specific key (from https://codechallenge.essensedesigns.info/api-key) instead.
+ESSENSE_API_URL=https://codechallenge.essensedesigns.info
+API_KEY_ESSENCE=
+
+# Testing only: set to true to turn off the one-vote-or-one-game-per-day rule (ignored in production).
+REMOVE_LIMITS=false
+
+# Production Docker stack only: protect the whole site with HTTP basic auth (off when empty).
+# Wrap the password in single quotes if it contains $, # or spaces.
+HTTP_AUTH_USER=gamevote
+HTTP_AUTH_PASSWORD=
+
+# Behind a TLS-terminating proxy (e.g. Railway) set to * so URLs and cookies use https.
+# Leave empty when the app is exposed directly.
+TRUSTED_PROXIES=
```

`.github/workflows/tests.yml` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -23,5 +23,5 @@
         uses: shivammathur/setup-php@eb7c497e18156a6bbabfef1d3a82760b9eda3962 # v2
         with:
-          php-version: '8.3'
+          php-version: '8.5'
```

```
          tools: composer:v2
          coverage: none
```

`.gitignore` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -31,5 +31,4 @@
 /.amp
 /.claude
-/.codex
 /.cursor
 /.factory
```

`app/Actions/Fortify/CreateNewUser.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -23,9 +23,11 @@
             ...$this->profileRules(),
             'password' => $this->passwordRules(),
+            'timezone' => ['nullable', 'string', 'timezone:all'],
         ])->validate();

         return User::create([
             'name' => $input['name'],
-            'email' => $input['email'],
+            'username' => $input['username'],
+            'timezone' => $input['timezone'] ?? config('app.timezone'),
             'password' => $input['password'],
         ]);
```

`app/Actions/Games/AddGame.php` (new file)

Create it with this content:

```php
<?php

namespace App\Actions\Games;

use App\Enums\DailyActionType;
use App\Exceptions\GameActionException;
use App\Models\User;
use App\Services\DailyAllowance;
use App\Services\Essense\EssenseApiException;
use App\Services\Essense\EssenseClient;
use App\Services\Essense\GameData;
use Illuminate\Contracts\Cache\LockTimeoutException;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Str;

class AddGame
{
    public function __construct(
        private readonly EssenseClient $client,
        private readonly DailyAllowance $allowance,
    ) {}

    /**
     * Add a game to the library, spending the user's daily allowance, and return its ID.
     *
     * The API accepts duplicate names, so titles are compared case-insensitively against the
     * live list. A lock serialises adds so two people cannot add the same title at once.
     */
    public function handle(User $user, string $name): int
    {
        $name = Str::squish($name);
```

```
        $this->allowance->ensureAvailable($user);

        try {
            return Cache::lock('games:add', 15)->block(5, fn () => $this->addUnlessDuplicate($user,
            ↪  $name));
        } catch (LockTimeoutException) {
            throw GameActionException::busy();
        }
    }

    private function addUnlessDuplicate(User $user, string $name): int
    {
        try {
            $existing = collect($this->client->games())
                ->first(fn (GameData $game) => Str::lower(Str::squish($game->name)) ===
↪   Str::lower($name));
        } catch (EssenseApiException $exception) {
            report($exception);

            throw GameActionException::fromApiFailure($exception);
        }

        if ($existing !== null) {
            throw GameActionException::duplicateTitle($existing->name);
        }

        $action = $this->allowance->claim($user, DailyActionType::AddGame);

        try {
            $gameId = $this->client->add($name);
        } catch (EssenseApiException $exception) {
            $action?->delete();
            report($exception);

            throw GameActionException::fromApiFailure($exception);
        }

        $action?->update(['game_id' => $gameId]);

        return $gameId;
    }
}
```

app/Actions/Games/CastVote.php (new file)

Create it with this content:

```
<?php

namespace App\Actions\Games;

use App\Enums\DailyActionType;
use App\Exceptions\GameActionException;
use App\Models\User;
use App\Services\DailyAllowance;
use App\Services\Essense\EssenseApiException;
use App\Services\Essense\EssenseClient;
use Illuminate\Database\UniqueConstraintViolationException;
use Illuminate\Support\Facades\DB;

class CastVote
{
    public function __construct(
        private readonly EssenseClient $client,
        private readonly DailyAllowance $allowance,
    ) {}

    /**
```

```
     * Vote for a game, spending the user's daily allowance.
     *
     * The vote and allowance are recorded first so the unique indexes reject double votes and
     * concurrent requests; if the API call then fails, both records are rolled back.
     */
    public function handle(User $user, int $gameId): void
    {
        if ($user->votes()->where('game_id', $gameId)->exists()) {
            throw GameActionException::alreadyVoted();
        }

        $this->allowance->ensureAvailable($user);

        [$action, $vote] = DB::transaction(function () use ($user, $gameId) {
            $action = $this->allowance->claim($user, DailyActionType::Vote, $gameId);

            try {
                $vote = $user->votes()->create(['game_id' => $gameId]);
            } catch (UniqueConstraintViolationException) {
                throw GameActionException::alreadyVoted();
            }

            return [$action, $vote];
        });

        try {
            $counted = $this->client->vote($gameId);
        } catch (EssenseApiException $exception) {
            $vote->delete();
            $action?->delete();
            report($exception);

            throw GameActionException::fromApiFailure($exception);
        }

        if (! $counted) {
            $vote->delete();
            $action?->delete();

            throw GameActionException::gameNotFound();
        }
    }
}
```

app/Actions/Games/RemoveGame.php (new file)

Create it with this content:

```
<?php

namespace App\Actions\Games;

use App\Exceptions\GameActionException;
use App\Models\Vote;
use App\Services\DailyAllowance;
use App\Services\Essense\EssenseApiException;
use App\Services\Essense\EssenseClient;
use Illuminate\Support\Facades\DB;

class RemoveGame
{
    public function __construct(
        private readonly EssenseClient $client,
        private readonly DailyAllowance $allowance,
    ) {}

    /**
     * Remove a game from the library. Anyone may remove a game.
     *
```

```
     * Its votes are discarded, and anyone who spent today's allowance on it gets it back.
     */
    public function handle(int $gameId): void
    {
        try {
            $removed = $this->client->remove($gameId);
        } catch (EssenseApiException $exception) {
            report($exception);

            throw GameActionException::fromApiFailure($exception);
        }

        DB::transaction(function () use ($gameId) {
            $this->allowance->refundRemovedGame($gameId);
            Vote::query()->where('game_id', $gameId)->delete();
        });

        if (! $removed) {
            throw GameActionException::gameNotFound();
        }
    }
}
```

app/Actions/Games/RemoveVote.php (new file)

Create it with this content:

```
<?php

namespace App\Actions\Games;

use App\Enums\DailyActionType;
use App\Exceptions\GameActionException;
use App\Models\User;
use App\Services\DailyAllowance;
use App\Services\Essense\EssenseApiException;
use App\Services\Essense\EssenseClient;
use Illuminate\Support\Facades\DB;

class RemoveVote
{
    public function __construct(
        private readonly EssenseClient $client,
        private readonly DailyAllowance $allowance,
    ) {}

    /**
     * Remove the user's own vote from a game. A vote cast today gives today's allowance back.
     */
    public function handle(User $user, int $gameId): void
    {
        $vote = $user->votes()->where('game_id', $gameId)->first();

        if ($vote === null) {
            throw GameActionException::notVoted();
        }

        try {
            // false means the game is already gone from the API; the local vote is stale either way.
            $this->client->removeVote($gameId);
        } catch (EssenseApiException $exception) {
            report($exception);

            throw GameActionException::fromApiFailure($exception);
        }

        DB::transaction(function () use ($user, $vote, $gameId) {
            $vote->delete();
            $this->allowance->refundToday($user, DailyActionType::Vote, $gameId);
```

```
        });
    }
}
```

app/Actions/Games/ResetLocalLibrary.php (new file)

Create it with this content:

```php
<?php

namespace App\Actions\Games;

use App\Models\Vote;
use App\Services\DailyAllowance;
use Illuminate\Support\Facades\DB;

class ResetLocalLibrary
{
    public function __construct(private readonly DailyAllowance $allowance) {}

    /**
     * Discard local data that belonged to a game library which no longer applies.
     *
     * Each API key has its own library whose game ids start again at 1, so votes recorded
     * against another key would point at the wrong games. Today's actions on those games
     * are refunded; earlier days stay recorded.
     */
    public function handle(): void
    {
        DB::transaction(function () {
            $this->allowance->refundEveryoneToday();
            Vote::query()->delete();
        });
    }
}
```

app/Concerns/ProfileValidationRules.php (edit)

Replace the whole file with:

```php
<?php

namespace App\Concerns;

use App\Models\User;
use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Validation\Rule;

trait ProfileValidationRules
{
    /**
     * Get the validation rules used to validate user profiles.
     *
     * @return array<string, array<int, ValidationRule|array<mixed>|string>>
     */
    protected function profileRules(?int $userId = null): array
    {
        return [
            'name' => $this->nameRules(),
            'username' => $this->usernameRules($userId),
        ];
    }

    /**
     * Get the validation rules used to validate user names.
     *
     * @return array<int, ValidationRule|array<mixed>|string>
```

```
     */
    protected function nameRules(): array
    {
        return ['required', 'string', 'max:255'];
    }

    /**
     * Get the validation rules used to validate usernames.
     *
     * @return array<int, ValidationRule|array<mixed>|string>
     */
    protected function usernameRules(?int $userId = null): array
    {
        return [
            'required',
            'string',
            'min:3',
            'max:30',
            'alpha_dash:ascii',
            $userId === null
                ? Rule::unique(User::class)
                : Rule::unique(User::class)->ignore($userId),
        ];
    }
}
```

app/Console/Commands/SeedGames.php (new file)

Create it with this content:

```
<?php

namespace App\Console\Commands;

use App\Services\Essense\EssenseApiException;
use App\Services\Essense\EssenseClient;
use App\Services\Essense\GameData;
use Illuminate\Console\Attributes\Description;
use Illuminate\Console\Attributes\Signature;
use Illuminate\Console\Command;
use Illuminate\Console\ConfirmableTrait;
use Illuminate\Support\Str;

#[Signature('games:seed
    {--count=10 : How many games to add}
    {--max-votes=15 : The most votes a seeded game can get}
    {--force : Run in production without asking for confirmation}')]
#[Description('Add sample games with random vote counts to the game library')]
class SeedGames extends Command
{
    use ConfirmableTrait;

    /**
     * Titles to seed from, so the library looks like a real office wishlist.
     *
     * @var list<string>
     */
    public const TITLES = [
        '7 Wonders',
        'Azul',
        'Carcassonne',
        'Catan',
        'Codenames',
        'Dixit',
        'Exploding Kittens',
        'Jackbox Party Pack 9',
        'King of Tokyo',
        'Mario Kart 8 Deluxe',
        'Overcooked! 2',
```

```
        'Pandemic',
        'Rocket League',
        'Splendor',
        'Stardew Valley',
        'Super Smash Bros. Ultimate',
        'Ticket to Ride',
        'Wingspan',
    ];

    /**
     * Execute the console command.
     *
     * Games and votes go straight to the API, so the daily rule does not apply and the votes
     * are not attributed to any user. Titles already in the library are skipped.
     */
    public function handle(EssenseClient $client): int
    {
        $count = filter_var($this->option('count'), FILTER_VALIDATE_INT, ['options' => ['min_range' =>
        ↪  1, 'max_range' => count(self::TITLES)]]);
        $maxVotes = filter_var($this->option('max-votes'), FILTER_VALIDATE_INT, ['options' =>
        ↪  ['min_range' => 0, 'max_range' => 100]]);

        if ($count === false || $maxVotes === false) {
            $this->components->error('--count must be between 1 and '.count(self::TITLES).', and
            ↪  --max-votes between 0 and 100.');

            return self::INVALID;
        }

        if (! $this->confirmToProceed()) {
            return self::FAILURE;
        }

        $seeded = [];

        try {
            $existing = collect($client->games())->map(fn (GameData $game) =>
            ↪  Str::lower(Str::squish($game->name)));
            $titles = collect(self::TITLES)
                ->reject(fn (string $title) => $existing->contains(Str::lower($title)))
                ->shuffle()
                ->take($count);

            foreach ($titles as $title) {
                $gameId = $client->add($title);
                $votes = random_int(0, $maxVotes);

                for ($i = 0; $i < $votes; $i++) {
                    $client->vote($gameId);
                }

                $seeded[] = [$title, $votes];
            }
        } catch (EssenseApiException $exception) {
            $this->components->error("Seeding stopped: {$exception->getMessage()}");
            $this->reportSeeded($seeded);

            return self::FAILURE;
        }

        if (count($seeded) < $count) {
            $this->components->warn('Only '.count($seeded).' new titles were available; the others are
            ↪  already in the library.');
        }

        $this->reportSeeded($seeded);

        return self::SUCCESS;
    }
```

```
    /**
     * @param  list<array{string, int}>  $seeded
     */
    private function reportSeeded(array $seeded): void
    {
        if ($seeded === []) {
            $this->components->info('No games were added.');

            return;
        }

        $this->table(['Game', 'Votes'], $seeded);
        $this->components->info('Added '.count($seeded).' '.Str::plural('game', count($seeded)).' to the
        ↪  library.');
    }
}
```

app/Enums/DailyActionType.php (new file)

Create it with this content:

```
<?php

namespace App\Enums;

enum DailyActionType: string
{
    case Vote = 'vote';
    case AddGame = 'add_game';
}
```

app/Exceptions/GameActionException.php (new file)

Create it with this content:

```
<?php

namespace App\Exceptions;

use App\Services\Essense\EssenseApiException;
use Illuminate\Http\RedirectResponse;
use Inertia\Inertia;
use RuntimeException;

/**
 * A game library action that could not be completed, carrying a message safe to show the user.
 */
class GameActionException extends RuntimeException
{
    public static function dailyLimitReached(): self
    {
        return new self('You have already used today\'s vote or new game. You can take another action
        ↪  after midnight.');
    }

    public static function duplicateTitle(string $existingName): self
    {
        return new self("\"{$existingName}\" is already in the library.");
    }

    public static function alreadyVoted(): self
    {
        return new self('You have already voted for this game.');
    }

    public static function notVoted(): self
    {
```

```
        return new self('You have not voted for this game.');
    }

    public static function gameNotFound(): self
    {
        return new self('That game is no longer in the library. The list has been refreshed.');
    }

    public static function libraryUnavailable(): self
    {
        return new self('The game library is not responding right now. Please try again in a moment.');
    }

    public static function notConfigured(): self
    {
        return new self('The game library rejected the API key in API_KEY_ESSENCE. Update it in .env, or
        ↪  remove it to let the app get its own key, then restart the app.');
    }

    /**
     * Translate an API failure into the message the user should see.
     */
    public static function fromApiFailure(EssenseApiException $exception): self
    {
        return in_array($exception->errorCode, ['MISSING_KEY', 'INVALID_KEY'], true)
            ? self::notConfigured()
            : self::libraryUnavailable();
    }

    public static function busy(): self
    {
        return new self('Another game is being added right now. Please try again.');
    }

    /**
     * Send the user back with the message as an error toast.
     */
    public function render(): RedirectResponse
    {
        Inertia::flash('toast', ['type' => 'error', 'message' => $this->getMessage()]);

        return back();
    }
}
```

app/Http/Controllers/GameController.php (new file)

Create it with this content:

```
<?php

namespace App\Http\Controllers;

use App\Actions\Games\AddGame;
use App\Actions\Games\RemoveGame;
use App\Exceptions\GameActionException;
use App\Http\Requests\StoreGameRequest;
use App\Services\DailyAllowance;
use App\Services\Essense\EssenseApiException;
use App\Services\Essense\EssenseClient;
use App\Services\Essense\GameData;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;
use Inertia\Inertia;
use Inertia\Response;

class GameController extends Controller
```

```
{
    /**
     * Show the game library, most-voted first.
     */
    public function index(Request $request, EssenseClient $client, DailyAllowance $allowance): Response
    {
        $user = $request->user();

        $games = null;
        $libraryError = null;

        try {
            $games = $client->games();
        } catch (EssenseApiException $exception) {
            report($exception);
            $libraryError = GameActionException::fromApiFailure($exception)->getMessage();
        }

        $votedGameIds = $user->votes()->pluck('game_id')->flip();

        return Inertia::render('dashboard', [
            'games' => $games === null ? null : collect($games)
                ->sort(fn (GameData $a, GameData $b) => [$b->votes, Str::lower($a->name)] <=>
↪   [$a->votes, Str::lower($b->name)])
                ->map(fn (GameData $game) => [
                    'id' => $game->id,
                    'name' => $game->name,
                    'votes' => $game->votes,
                    'voted' => $votedGameIds->has($game->id),
                ])
                ->values(),
            'libraryError' => $libraryError,
            'allowance' => $allowance->status($user),
            'timezone' => $user->timezone,
        ]);
    }

    /**
     * Add a game to the library.
     */
    public function store(StoreGameRequest $request, AddGame $addGame): RedirectResponse
    {
        $name = $request->validated('name');

        try {
            $addGame->handle($request->user(), $name);
        } catch (GameActionException $exception) {
            throw ValidationException::withMessages(['name' => $exception->getMessage()]);
        }

       Inertia::flash('toast', ['type' => 'success', 'message' => "Added \"{$name}\" to the library."]);

        return to_route('dashboard');
    }

    /**
     * Remove a game from the library.
     */
    public function destroy(int $game, RemoveGame $removeGame): RedirectResponse
    {
        $removeGame->handle($game);

        Inertia::flash('toast', ['type' => 'success', 'message' => 'Game removed from the library.']);

        return to_route('dashboard');
    }
}
```

app/Http/Controllers/GameVoteController.php (new file)

Create it with this content:

```php
<?php

namespace App\Http\Controllers;

use App\Actions\Games\CastVote;
use App\Actions\Games\RemoveVote;
use App\Services\DailyAllowance;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Inertia\Inertia;

class GameVoteController extends Controller
{
    /**
     * Vote for a game.
     */
    public function store(Request $request, int $game, CastVote $castVote): RedirectResponse
    {
        $castVote->handle($request->user(), $game);

        Inertia::flash('toast', ['type' => 'success', 'message' => 'Your vote was counted.']);

        return to_route('dashboard');
    }

    /**
     * Remove the user's vote from a game.
     */
    public function destroy(Request $request, int $game, RemoveVote $removeVote, DailyAllowance
    ↪  $allowance): RedirectResponse
    {
        $removeVote->handle($request->user(), $game);

        $message = $allowance->isEnforced() && $allowance->usedToday($request->user()) === null
            ? 'Your vote was removed. You can vote or add a game again today.'
            : 'Your vote was removed.';

        Inertia::flash('toast', ['type' => 'success', 'message' => $message]);

        return to_route('dashboard');
    }
}
```

app/Http/Controllers/Settings/ProfileController.php (edit)

Replace the whole file with:

```php
<?php

namespace App\Http\Controllers\Settings;

use App\Http\Controllers\Controller;
use App\Http\Requests\Settings\ProfileDeleteRequest;
use App\Http\Requests\Settings\ProfileUpdateRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
use Inertia\Inertia;
use Inertia\Response;

class ProfileController extends Controller
{
    /**
     * Show the user's profile settings page.
     */
    public function edit(): Response
```

```
    {
        return Inertia::render('settings/profile');
    }

    /**
     * Update the user's profile information.
     */
    public function update(ProfileUpdateRequest $request): RedirectResponse
    {
        $request->user()->update($request->validated());

        Inertia::flash('toast', ['type' => 'success', 'message' => __('Profile updated.')]);

        return to_route('profile.edit');
    }

    /**
     * Delete the user's profile.
     */
    public function destroy(ProfileDeleteRequest $request): RedirectResponse
    {
        $user = $request->user();

        Auth::logout();

        $user->delete();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

app/Http/Middleware/AddContentSecurityPolicy.php (new file)

Create it with this content:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Vite;
use Symfony\Component\HttpFoundation\Response;

/**
 * Sends a Content-Security-Policy with a per-request nonce for the page's own scripts:
 * the inline theme script in app.blade.php and the tags Vite renders.
 *
 * On in production; off in local development, where the Vite dev server would need its
 * own origin and websocket allowed (see config/app.php content_security_policy).
 */
class AddContentSecurityPolicy
{
    /**
     * Handle an incoming request.
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        if (! config('app.content_security_policy')) {
            return $next($request);
        }

        $nonce = Vite::useCspNonce();
```

```
        $response = $next($request);

        $response->headers->set('Content-Security-Policy', implode('; ', [
            "default-src 'self'",
            "script-src 'self' 'nonce-{$nonce}'",
            // Inline style attributes (React style props, toasts) and the font-face styles.
            "style-src 'self' 'unsafe-inline'",
            // data: covers the brick wall SVG used as a CSS background.
            "img-src 'self' data:",
            "font-src 'self'",
            "connect-src 'self'",
            "object-src 'none'",
            "base-uri 'self'",
            "form-action 'self'",
            "frame-ancestors 'self'",
        ]));

        return $response;
    }
}
```

app/Http/Middleware/HandleAppearance.php (edit)

Replace the whole file with:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\View;
use Symfony\Component\HttpFoundation\Response;

class HandleAppearance
{
    /**
     * Handle an incoming request.
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        // The cookie is unencrypted and ends up in an inline script, so only accept known values.
        $appearance = $request->cookie('appearance');

        View::share('appearance', in_array($appearance, ['light', 'dark', 'system'], true) ? $appearance
        ↪  : 'system');

        return $next($request);
    }
}
```

app/Http/Middleware/HandleInertiaRequests.php (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -42,5 +42,4 @@
                'user' => $request->user(),
            ],
-            'sidebarOpen' => ! $request->hasCookie('sidebar_state') ||
↪  $request->cookie('sidebar_state') === 'true',
        ];
    }
```

app/Http/Requests/Settings/ProfileUpdateRequest.php (edit)

Replace the whole file with:

```php
<?php

namespace App\Http\Requests\Settings;

use App\Concerns\ProfileValidationRules;
use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Str;
use Laravel\Fortify\Fortify;

class ProfileUpdateRequest extends FormRequest
{
    use ProfileValidationRules;

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return $this->profileRules($this->user()->id);
    }

    /**
     * Lowercase the username to match how Fortify stores it at registration and looks it up at login.
     */
    protected function prepareForValidation(): void
    {
        if (config('fortify.lowercase_usernames') && is_string($this->input(Fortify::username()))) {
            $this->merge([
                Fortify::username() => Str::lower($this->input(Fortify::username())),
            ]);
        }
    }
}
```

app/Http/Requests/StoreGameRequest.php (new file)

Create it with this content:

```php
<?php

namespace App\Http\Requests;

use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Str;

class StoreGameRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, ValidationRule|array<mixed>|string>
     */
    public function rules(): array
```

```
    {
        return [
            'name' => ['required', 'string', 'max:100'],
        ];
    }

    /**
     * Get custom attributes for validator errors.
     *
     * @return array<string, string>
     */
    public function attributes(): array
    {
        return [
            'name' => 'game title',
        ];
    }

    /**
     * Collapse surrounding and repeated whitespace before validating the title.
     */
    protected function prepareForValidation(): void
    {
        if (is_string($this->input('name'))) {
            $this->merge(['name' => Str::squish($this->input('name'))]);
        }
    }
}
```

app/Models/ApiCredential.php (new file)

Create it with this content:

```
<?php

namespace App\Models;

use Carbon\CarbonImmutable;
use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Attributes\Hidden;
use Illuminate\Database\Eloquent\Model;

/**
 * The single row describing which Essense API key the app uses.
 *
 * `generated_key` is a key the app obtained for itself (encrypted at rest). It is null when
 * the key comes from API_KEY_ESSENCE. `library_fingerprint` identifies the key that the local
 * votes and daily actions belong to, so they can be reset when the active key changes.
 *
 * @property int $id
 * @property string|null $generated_key
 * @property string|null $library_fingerprint
 * @property CarbonImmutable|null $created_at
 * @property CarbonImmutable|null $updated_at
 */
#[Fillable(['generated_key', 'library_fingerprint'])]
#[Hidden(['generated_key'])]
class ApiCredential extends Model
{
    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'generated_key' => 'encrypted',
        ];
```

```
    }

    /**
     * Get the one credential row, creating it if needed.
     */
    public static function current(): self
    {
        return self::query()->firstOrCreate(['id' => 1]);
    }
}
```

app/Models/DailyAction.php (new file)

Create it with this content:

```
<?php

namespace App\Models;

use App\Enums\DailyActionType;
use Carbon\CarbonImmutable;
use Database\Factories\DailyActionFactory;
use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

/**
 * The single action (a vote or a new game) a user spent on one local calendar day.
 *
 * @property int $id
 * @property int $user_id
 * @property CarbonImmutable $action_date
 * @property DailyActionType $type
 * @property int|null $game_id
 * @property CarbonImmutable|null $created_at
 * @property CarbonImmutable|null $updated_at
 */
#[Fillable(['user_id', 'action_date', 'type', 'game_id'])]
class DailyAction extends Model
{
    /** @use HasFactory<DailyActionFactory> */
    use HasFactory;

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'action_date' => 'date',
            'type' => DailyActionType::class,
        ];
    }

    /**
     * Get the user who took the action.
     *
     * @return BelongsTo<User, $this>
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

app/Models/User.php (edit)

Replace the whole file with:

```php
<?php

namespace App\Models;

use Carbon\CarbonImmutable;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Attributes\Hidden;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Illuminate\Support\Carbon;

/**
 * @property int $id
 * @property string $name
 * @property string $username
 * @property string $timezone
 * @property string $password
 * @property string|null $remember_token
 * @property Carbon|null $created_at
 * @property Carbon|null $updated_at
 */
#[Fillable(['name', 'username', 'timezone', 'password'])]
#[Hidden(['password', 'remember_token'])]
class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'password' => 'hashed',
        ];
    }

    /**
     * Get the votes the user has cast.
     *
     * @return HasMany<Vote, $this>
     */
    public function votes(): HasMany
    {
        return $this->hasMany(Vote::class);
    }

    /**
     * Get the daily actions the user has taken.
     *
     * @return HasMany<DailyAction, $this>
     */
    public function dailyActions(): HasMany
    {
        return $this->hasMany(DailyAction::class);
    }

    /**
     * Get the current calendar date in the user's timezone.
     */
    public function localToday(): CarbonImmutable
```

```
    {
        return now($this->timezone)->startOfDay();
    }
}
```

app/Models/Vote.php (new file)

Create it with this content:

```php
<?php

namespace App\Models;

use Carbon\CarbonImmutable;
use Database\Factories\VoteFactory;
use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

/**
 * A user's vote for a game in the external library. Vote totals live in the API;
 * this records who cast each vote so it can only be removed by its owner.
 *
 * @property int $id
 * @property int $user_id
 * @property int $game_id
 * @property CarbonImmutable|null $created_at
 * @property CarbonImmutable|null $updated_at
 */
#[Fillable(['user_id', 'game_id'])]
class Vote extends Model
{
    /** @use HasFactory<VoteFactory> */
    use HasFactory;

    /**
     * Get the user who cast the vote.
     *
     * @return BelongsTo<User, $this>
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

app/Providers/AppServiceProvider.php (edit)

Replace the whole file with:

```php
<?php

namespace App\Providers;

use App\Actions\Games\ResetLocalLibrary;
use App\Services\Essense\ApiKeyProvider;
use App\Services\Essense\EssenseClient;
use Carbon\CarbonImmutable;
use Illuminate\Http\Middleware\TrustProxies;
use Illuminate\Support\Facades\Date;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\ServiceProvider;
use Illuminate\Validation\Rules\Password;

class AppServiceProvider extends ServiceProvider
{
```

```
    /**
     * Register any application services.
     */
    public function register(): void
    {
        $this->app->singleton(ApiKeyProvider::class, fn () => new ApiKeyProvider(
            baseUrl: config('services.essense.url'),
            configuredKey: config('services.essense.key'),
            resetLocalLibrary: $this->app->make(ResetLocalLibrary::class),
        ));

        $this->app->singleton(EssenseClient::class, fn () => new EssenseClient(
            baseUrl: config('services.essense.url'),
            keys: $this->app->make(ApiKeyProvider::class),
        ));
    }

    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        $this->configureDefaults();
    }

    /**
     * Configure default behaviors for production-ready applications.
     */
    protected function configureDefaults(): void
    {
        Date::use(CarbonImmutable::class);

        $trustedProxies = config('app.trusted_proxies');

        if (is_string($trustedProxies) && $trustedProxies !== '') {
            TrustProxies::at($trustedProxies === '*' ? '*' : array_map('trim', explode(',',
            ↪  $trustedProxies)));
        }

        DB::prohibitDestructiveCommands(
            app()->isProduction(),
        );

        Password::defaults(fn (): ?Password => app()->isProduction()
            ? Password::min(12)
                ->mixedCase()
                ->letters()
                ->numbers()
                ->symbols()
                ->uncompromised()
            : null,
        );
    }
}
```

app/Providers/FortifyServiceProvider.php (edit)

Replace the whole file with:

```
<?php

namespace App\Providers;

use App\Actions\Fortify\CreateNewUser;
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\ServiceProvider;
use Illuminate\Support\Str;
```

```php
use Illuminate\Validation\Rules\Password;
use Inertia\Inertia;
use Laravel\Fortify\Fortify;

class FortifyServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     */
    public function register(): void
    {
        //
    }

    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        $this->configureActions();
        $this->configureViews();
        $this->configureRateLimiting();
    }

    /**
     * Configure Fortify actions.
     */
    private function configureActions(): void
    {
        Fortify::createUsersUsing(CreateNewUser::class);
    }

    /**
     * Configure Fortify views.
     */
    private function configureViews(): void
    {
        Fortify::loginView(fn (Request $request) => Inertia::render('auth/login', [
            'status' => $request->session()->get('status'),
        ]));

        Fortify::registerView(fn () => Inertia::render('auth/register', [
            'passwordRules' => Password::defaults()->toPasswordRulesString(),
        ]));
    }

    /**
     * Configure rate limiting.
     */
    private function configureRateLimiting(): void
    {
        RateLimiter::for('login', function (Request $request) {
            $throttleKey = Str::transliterate(Str::lower($request->input(Fortify::username())).'|'.
            ↪  $request->ip());

            return [
                Limit::perMinute(5)->by($throttleKey),
                // Also cap each IP across usernames, so one address can't try a password against many
                ↪  accounts.
                Limit::perMinute(20)->by('ip:'.$request->ip()),
            ];
        });
    }
}
```

app/Services/DailyAllowance.php (new file)

Create it with this content:

```php
<?php

namespace App\Services;

use App\Enums\DailyActionType;
use App\Exceptions\GameActionException;
use App\Models\DailyAction;
use App\Models\User;
use Carbon\CarbonImmutable;
use Illuminate\Database\UniqueConstraintViolationException;

/**
 * Enforces the daily rule: each user may cast one vote OR add one game per local calendar day.
 *
 * The rule is backed by a unique index on (user_id, action_date), so two concurrent
 * requests cannot both spend the same day's allowance.
 *
 * Setting REMOVE_LIMITS=true turns the rule off for testing, except in production.
 */
class DailyAllowance
{
    /**
     * Whether the daily rule is being enforced.
     */
    public function isEnforced(): bool
    {
        return ! config('games.remove_limits') || app()->isProduction();
    }

    /**
     * Get the action the user has taken today, if any.
     */
    public function usedToday(User $user): ?DailyAction
    {
        return $user->dailyActions()
            ->whereDate('action_date', $user->localToday()->toDateString())
            ->first();
    }

    /**
     * Throw if the user has already used today's allowance.
     */
    public function ensureAvailable(User $user): void
    {
        if ($this->isEnforced() && $this->usedToday($user) !== null) {
            throw GameActionException::dailyLimitReached();
        }
    }

    /**
     * Spend today's allowance on the given action. Nothing is recorded while the rule is off.
     */
    public function claim(User $user, DailyActionType $type, ?int $gameId = null): ?DailyAction
    {
        if (! $this->isEnforced()) {
            return null;
        }

        try {
            return $user->dailyActions()->create([
                'action_date' => $user->localToday()->toDateString(),
                'type' => $type,
                'game_id' => $gameId,
            ]);
        } catch (UniqueConstraintViolationException) {
            throw GameActionException::dailyLimitReached();
        }
    }

    /**
```

```php
     * Give back today's allowance if it was spent on this action for this game.
     *
     * Actions from earlier days stay recorded, so undoing them never grants an extra action.
     */
    public function refundToday(User $user, DailyActionType $type, int $gameId): void
    {
        $user->dailyActions()
            ->whereDate('action_date', $user->localToday()->toDateString())
            ->where('type', $type)
            ->where('game_id', $gameId)
            ->delete();
    }

    /**
     * Refund every user who spent today's allowance on a game that has been removed.
     */
    public function refundRemovedGame(int $gameId): void
    {
        DailyAction::query()
            ->where('game_id', $gameId)
            ->with('user')
            ->get()
            ->filter(fn (DailyAction $action) => $action->action_date->toDateString() ===
↪   $action->user->localToday()->toDateString())
            ->each->delete();
    }

    /**
     * Refund every action taken today on a game, for every user.
     *
     * Actions without a game yet are adds still in progress; they keep their claim.
     */
    public function refundEveryoneToday(): void
    {
        DailyAction::query()
            ->whereNotNull('game_id')
            ->whereDate('action_date', '>=', now()->subDay()->toDateString())
            ->with('user')
            ->get()
            ->filter(fn (DailyAction $action) => $action->action_date->toDateString() ===
↪   $action->user->localToday()->toDateString())
            ->each->delete();
    }

    /**
     * Describe the user's allowance for today.
     *
     * @return array{enforced: bool, available: bool, used_for: string|null, game_id: int|null,
↪   resets_at: string}
     */
    public function status(User $user): array
    {
        $action = $this->isEnforced() ? $this->usedToday($user) : null;

        return [
            'enforced' => $this->isEnforced(),
            'available' => $action === null,
            'used_for' => $action?->type->value,
            'game_id' => $action?->game_id,
            'resets_at' => $this->resetsAt($user)->toIso8601String(),
        ];
    }

    /**
     * Get the moment the user's allowance next resets: their next local midnight.
     */
    public function resetsAt(User $user): CarbonImmutable
    {
        return $user->localToday()->addDay();
    }
```

```
}
```

app/Services/Essense/ApiKeyProvider.php (new file)

Create it with this content:

```php
<?php

namespace App\Services\Essense;

use App\Actions\Games\ResetLocalLibrary;
use App\Models\ApiCredential;
use Illuminate\Contracts\Cache\LockTimeoutException;
use Illuminate\Contracts\Encryption\DecryptException;
use Illuminate\Http\Client\ConnectionException;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Http;

/**
 * Supplies the Essense API key.
 *
 * API_KEY_ESSENCE wins when it is set. Otherwise the app generates its own key on first use
 * and stores it (encrypted) in the database, so a fresh `docker-compose up` needs no setup.
 *
 * Whichever key is active, the provider remembers which library the local votes belong to
 * and resets them when the key changes, because each key's game ids start again at 1.
 */
class ApiKeyProvider
{
    private ?string $resolvedKey = null;

    public function __construct(
        private readonly string $baseUrl,
        private readonly ?string $configuredKey,
        private readonly ResetLocalLibrary $resetLocalLibrary,
    ) {}

    /**
     * Whether the app manages its own key, rather than using API_KEY_ESSENCE.
     */
    public function isManaged(): bool
    {
        return blank($this->configuredKey);
    }

    /**
     * Get the key to send with API requests.
     *
     * @throws EssenseApiException
     */
    public function key(): string
    {
        if ($this->resolvedKey !== null) {
            return $this->resolvedKey;
        }

        $key = $this->isManaged() ? $this->storedOrGeneratedKey() : (string) $this->configuredKey;
        $this->adoptLibrary($key);

        return $this->resolvedKey = $key;
    }

    /**
     * Replace a generated key the API rejected. The API's keys reset when its server restarts.
     *
     * @throws EssenseApiException
     */
    public function replace(string $rejectedKey): string
    {
```

```
        $key = $this->withLock(function () use ($rejectedKey) {
            $credential = ApiCredential::current();
            $stored = $this->readStoredKey($credential);

            // Another request may already have replaced it while we waited for the lock.
            if ($stored !== null && $stored !== $rejectedKey) {
                return $stored;
            }

            $key = $this->generate();
            $credential->update(['generated_key' => $key]);

            return $key;
        });

        $this->adoptLibrary($key);

        return $this->resolvedKey = $key;
    }

    private function storedOrGeneratedKey(): string
    {
        return $this->readStoredKey(ApiCredential::current()) ?? $this->withLock(function () {
            $credential = ApiCredential::current();

            if (($stored = $this->readStoredKey($credential)) !== null) {
                return $stored;
            }

            $key = $this->generate();
            $credential->update(['generated_key' => $key]);

            return $key;
        });
    }

    /**
     * Reset local votes if they belong to a different key's library, then record this key's.
     */
    private function adoptLibrary(string $key): void
    {
        $fingerprint = hash('sha256', $key);
        $credential = ApiCredential::current();

        if ($credential->library_fingerprint === $fingerprint) {
            return;
        }

        if ($credential->library_fingerprint !== null) {
            $this->resetLocalLibrary->handle();
        }

        $credential->update(['library_fingerprint' => $fingerprint]);
    }

    /**
     * Read the stored key, treating one encrypted under a previous APP_KEY as missing.
     */
    private function readStoredKey(ApiCredential $credential): ?string
    {
        try {
            return filled($credential->generated_key) ? $credential->generated_key : null;
        } catch (DecryptException) {
            return null;
        }
    }

    private function generate(): string
    {
        try {
```

```
            $response = Http::baseUrl($this->baseUrl)
                ->acceptJson()
                ->connectTimeout(4)
                ->timeout(8)
                ->post('api-key/generate');
        } catch (ConnectionException $exception) {
           throw new EssenseApiException('The game library could not be reached.', previous: $exception);
        }

        $key = $response->json('key');

        if ($response->failed() || ! is_string($key) || $key === '') {
            throw new EssenseApiException('The game library did not issue an API key.',
             ↪  'KEY_GENERATION_FAILED');
        }

        return $key;
    }

    /**
     * @template TResult
     *
     * @param  callable(): TResult  $callback
     * @return TResult
     */
    private function withLock(callable $callback): mixed
    {
        try {
            return Cache::lock('essense:api-key', 20)->block(10, $callback);
        } catch (LockTimeoutException $exception) {
            throw new EssenseApiException('Timed out waiting to obtain an API key.', previous:
             ↪  $exception);
        }
    }
}
```

app/Services/Essense/EssenseApiException.php (new file)

Create it with this content:

```
<?php

namespace App\Services\Essense;

use RuntimeException;
use Throwable;

/**
 * Thrown when the Essense API cannot be reached or does not return a usable response.
 */
class EssenseApiException extends RuntimeException
{
    public function __construct(string $message, public readonly ?string $errorCode = null, ?Throwable
     ↪  $previous = null)
    {
        parent::__construct($message, 0, $previous);
    }
}
```

app/Services/Essense/EssenseClient.php (new file)

Create it with this content:

```
<?php

namespace App\Services\Essense;
```

```php
use Illuminate\Http\Client\ConnectionException;
use Illuminate\Http\Client\PendingRequest;
use Illuminate\Http\Client\Response;
use Illuminate\Support\Facades\Http;
use Throwable;

/**
 * Client for the Essense code challenge REST API.
 *
 * Every endpoint takes a form-encoded body that includes the API key. The API reports
 * most failures with HTTP 200 and a sentinel body (`{}`, `{"success": false}` or
 * `{"id": -1}`), so responses are checked by shape rather than status code alone.
 *
 * The API's keys reset when its server restarts. When a key the app generated is rejected,
 * the client gets a new one and retries once; the rejected request changed nothing.
 */
class EssenseClient
{
    public function __construct(
        private readonly string $baseUrl,
        private readonly ApiKeyProvider $keys,
        private readonly int $timeoutSeconds = 8,
    ) {}

    /**
     * List every game in the library.
     *
     * @return list<GameData>
     */
    public function games(): array
    {
        $body = $this->post('games/list', retry: true);

        if (! isset($body['games']) || ! is_array($body['games'])) {
            throw new EssenseApiException('The game library returned an unexpected response.');
        }

        return array_values(array_map(GameData::fromArray(...), $body['games']));
    }

    /**
     * Find a game by its ID, or null when no such game exists.
     */
    public function find(int $id): ?GameData
    {
        $body = $this->post('games/search', ['id' => $id], retry: true);

        return isset($body['id'], $body['name']) ? GameData::fromArray($body) : null;
    }

    /**
     * Add a game and return its new ID.
     */
    public function add(string $name): int
    {
        $body = $this->post('games/add', ['name' => $name]);
        $id = (int) ($body['id'] ?? -1);

        if ($id < 1) {
            throw new EssenseApiException('The game library could not add that game.');
        }

        return $id;
    }

    /**
     * Remove a game. Returns false when the game does not exist.
     */
    public function remove(int $id): bool
    {
```

```
        return $this->succeeded($this->post('games/remove', ['id' => $id]));
    }

    /**
     * Add one vote to a game. Returns false when the game does not exist.
     */
    public function vote(int $id): bool
    {
        return $this->succeeded($this->post('games/vote', ['id' => $id]));
    }

    /**
     * Remove one vote from a game. Returns false when the game does not exist.
     */
    public function removeVote(int $id): bool
    {
        return $this->succeeded($this->post('games/removeVote', ['id' => $id]));
    }

    /**
     * Send a request and return the decoded JSON body.
     *
     * Only idempotent reads are retried; retrying a vote could count it twice.
     *
     * @param  array<string, scalar>  $data
     * @return array<string, mixed>
     */
    private function post(string $endpoint, array $data = [], bool $retry = false): array
    {
        $key = $this->keys->key();

        try {
            return $this->send($endpoint, $key, $data, $retry);
        } catch (EssenseApiException $exception) {
            if ($exception->errorCode !== 'INVALID_KEY' || ! $this->keys->isManaged()) {
                throw $exception;
            }

            return $this->send($endpoint, $this->keys->replace($key), $data, $retry);
        }
    }

    /**
     * @param  array<string, scalar>  $data
     * @return array<string, mixed>
     */
    private function send(string $endpoint, string $key, array $data, bool $retry): array
    {
        try {
            $response = $this->request($retry)->post($endpoint, ['api_key' => $key, ...$data]);
        } catch (ConnectionException $exception) {
           throw new EssenseApiException('The game library could not be reached.', previous: $exception);
        }

        return $this->decode($response);
    }

    private function request(bool $retry): PendingRequest
    {
        $request = Http::baseUrl($this->baseUrl)
            ->asForm()
            ->acceptJson()
            ->connectTimeout(4)
            ->timeout($this->timeoutSeconds);

        return $retry
            ? $request->retry(2, 200, fn (Throwable $exception) => $exception instanceof
            ↪  ConnectionException, throw: false)
            : $request;
    }
```

```
    /**
     * @return array<string, mixed>
     */
    private function decode(Response $response): array
    {
        $body = $response->json();

        if (is_array($body) && is_string($body['error'] ?? null)) {
            throw new EssenseApiException("The game library rejected the request ({$body['error']}).",
             ↪  $body['error']);
        }

        if ($response->failed() || ! is_array($body)) {
           throw new EssenseApiException("The game library responded with HTTP {$response->status()}.");
        }

        return $body;
    }

    /**
     * @param  array<string, mixed>  $body
     */
    private function succeeded(array $body): bool
    {
        return ($body['success'] ?? false) === true;
    }
}
```

app/Services/Essense/GameData.php (new file)

Create it with this content:

```
<?php

namespace App\Services\Essense;

/**
 * A game as returned by the Essense code challenge API.
 */
readonly class GameData
{
    public function __construct(
        public int $id,
        public string $name,
        public int $votes,
    ) {}

    /**
     * Build a game from an API payload, rejecting anything that is not a well-formed game.
     *
     * @throws EssenseApiException
     */
    public static function fromArray(mixed $attributes): self
    {
        if (! is_array($attributes)
            || ! is_numeric($attributes['id'] ?? null)
            || ! is_string($attributes['name'] ?? null)
            || ! is_numeric($attributes['votes'] ?? null)) {
            throw new EssenseApiException('The game library returned a malformed game.');
        }

        return new self((int) $attributes['id'], $attributes['name'], (int) $attributes['votes']);
    }
}
```

`bootstrap/app.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -1,4 +1,5 @@
 <?php

+use App\Http\Middleware\AddContentSecurityPolicy;
 use App\Http\Middleware\HandleAppearance;
 use App\Http\Middleware\HandleInertiaRequests;
@@ -18,5 +19,18 @@
         $middleware->encryptCookies(except: ['appearance', 'sidebar_state']);

+        // Behind a trusted proxy the host can come from X-Forwarded-Host, so only accept the
+        // app's own host and Railway's health checker. Without trusted proxies, no restriction.
+        $middleware->trustHosts(at: function (): array {
+            $host = parse_url((string) config('app.url'), PHP_URL_HOST);
+
+            if (blank(config('app.trusted_proxies')) || ! is_string($host)) {
+                return [];
+            }
+
+            return ['^'.preg_quote($host).'$', '^healthcheck\.railway\.app$'];
+        }, subdomains: false);
+
         $middleware->web(append: [
+            AddContentSecurityPolicy::class,
             HandleAppearance::class,
             HandleInertiaRequests::class,
```

`composer.json` (edit)

Apply these changes (- remove, + add, other lines are context).:

```
@@ -10,5 +10,5 @@
     "license": "MIT",
     "require": {
-        "php": "^8.3",
+        "php": "^8.5",
         "inertiajs/inertia-laravel": "^3.0",
         "laravel/chisel": "^0.1.0",
@@ -21,4 +21,5 @@
         "fakerphp/faker": "^1.24",
         "larastan/larastan": "^3.9",
+        "laravel/boost": "^2.10",
         "laravel/pail": "^1.2.5",
         "laravel/pao": "^1.0.6",
@@ -27,5 +28,7 @@
         "mockery/mockery": "^1.6",
         "nunomaduro/collision": "^8.9.3",
-        "phpunit/phpunit": "^12.5.23"
+        "pestphp/pest": "^5.3",
+        "pestphp/pest-plugin-drift": "^5.0",
+        "pestphp/pest-plugin-laravel": "^5.0"
     },
     "autoload": {
@@ -80,5 +83,4 @@
         ],
         "post-update-cmd": [
-            "@php artisan install:features --ansi",
             "@php artisan vendor:publish --tag=laravel-assets --ansi --force"
         ],
@@ -100,5 +102,4 @@
             "installer": {
                 "post-create-project": [
-                    "@php artisan install:features --ansi"
                 ]
             }
```

The Pest packages arrive with `--pest` and Boost with `composer require laravel/boost --dev`;

only the PHP version needs a hand edit.

`config/app.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -57,4 +57,31 @@
     /*
     |--------------------------------------------------------------------------
+    | Trusted Proxies
+    |--------------------------------------------------------------------------
+    |
+    | Proxies whose X-Forwarded-* headers are trusted: "*" for any, or a comma
+    | separated list of IPs. Set this behind a TLS-terminating proxy (such as
+    | Railway's) so URLs and secure cookies use https. Leave it empty when the
+    | app is exposed directly, or clients could spoof their IP address.
+    |
+    */
+
+    'trusted_proxies' => env('TRUSTED_PROXIES'),
+
+    /*
+    |--------------------------------------------------------------------------
+    | Content Security Policy
+    |--------------------------------------------------------------------------
+    |
+    | Send a nonce-based Content-Security-Policy header with each response. On
+    | by default in production. It is off in local development because the
+    | Vite dev server would need its own origin and websocket allowed.
+    |
+    */
+
+    'content_security_policy' => (bool) env('CONTENT_SECURITY_POLICY', env('APP_ENV') ===
 ↪  'production'),
+
+    /*
+    |--------------------------------------------------------------------------
     | Application Timezone
     |--------------------------------------------------------------------------
```

`config/fortify.php` (edit)

Apply these changes (- remove, + add, other lines are context). Check that `'lowercase_usernames' => true` is set too:

```
@@ -46,5 +46,5 @@
     */

-    'username' => 'email',
+    'username' => 'username',

     'email' => 'email',
@@ -145,5 +145,4 @@
     'features' => [
         Features::registration(),
-        Features::resetPasswords(),
     ],
 ];
```

`config/games.php` (new file)

Create it with this content:

```
<?php

return [
```

```
    /*
    |--------------------------------------------------------------------------
    | Remove Limits
    |--------------------------------------------------------------------------
    |
    | For testing only: when true, the daily rule (one vote or one new game per
    | user per day) is not enforced. Duplicate titles and one vote per game are
    | still rejected. This is always ignored in production.
    |
    */

    'remove_limits' => (bool) env('REMOVE_LIMITS', false),

];
```

`config/inertia.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -17,5 +17,5 @@

     'ssr' => [
-        'enabled' => true,
+        'enabled' => (bool) env('INERTIA_SSR_ENABLED', true),
         'url' => 'http://127.0.0.1:13714',
         // 'bundle' => base_path('bootstrap/ssr/ssr.mjs'),
```

`config/services.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -36,3 +36,8 @@
     ],

+    'essense' => [
+        'url' => env('ESSENSE_API_URL', 'https://codechallenge.essensedesigns.info'),
+        'key' => env('API_KEY_ESSENCE'),
+    ],
+
 ];
```

`database/factories/DailyActionFactory.php` (new file)

Create it with this content:

```
<?php

namespace Database\Factories;

use App\Enums\DailyActionType;
use App\Models\DailyAction;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

/**
 * @extends Factory<DailyAction>
 */
class DailyActionFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
```

```
        return [
            'user_id' => User::factory(),
            'action_date' => now()->toDateString(),
            'type' => DailyActionType::Vote,
            'game_id' => fake()->unique()->numberBetween(1, 100000),
        ];
    }

    /**
     * Indicate that the action was adding a game.
     */
    public function addedGame(): static
    {
        return $this->state(fn (array $attributes) => [
            'type' => DailyActionType::AddGame,
        ]);
    }
}
```

database/factories/UserFactory.php (edit)

Replace the whole file with:

```
<?php

namespace Database\Factories;

use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;

/**
 * @extends Factory<User>
 */
class UserFactory extends Factory
{
    /**
     * The current password being used by the factory.
     */
    protected static ?string $password;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
            'username' => fake()->unique()->lexify('user_????????'),
            'password' => static::$password ??= Hash::make('password'),
            'remember_token' => Str::random(10),
        ];
    }
}
```

database/factories/VoteFactory.php (new file)

Create it with this content:

```
<?php

namespace Database\Factories;

use App\Models\User;
```

```php
use App\Models\Vote;
use Illuminate\Database\Eloquent\Factories\Factory;

/**
 * @extends Factory<Vote>
 */
class VoteFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'user_id' => User::factory(),
            'game_id' => fake()->unique()->numberBetween(1, 100000),
        ];
    }
}
```

database/migrations/0001_01_01_000000_create_users_table.php (edit)

Replace the whole file with:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('username')->unique();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('sessions');
    }
};
```

database/migrations/2026_10_20_013125_create_votes_table.php (new file)

Created by `php artisan make:model Vote -mf`; your timestamp will differ. Fill it with:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('votes', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->unsignedBigInteger('game_id')->index();
            $table->timestamps();

            $table->unique(['user_id', 'game_id']);
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('votes');
    }
};
```

database/migrations/2026_10_20_013126_create_daily_actions_table.php (new file)

Created by `php artisan make:model DailyAction -mf`. Fill it with:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('daily_actions', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->date('action_date');
            $table->string('type');
            $table->unsignedBigInteger('game_id')->nullable()->index();
            $table->timestamps();

            // The daily rule: at most one action per user per local calendar day.
            $table->unique(['user_id', 'action_date']);
        });
    }
```

```
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('daily_actions');
    }
};
```

database/migrations/2026_10_20_013445_add_timezone_to_users_table.php (new file)

Created by `php artisan make:migration add_timezone_to_users_table --table=users`. Fill it with:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('timezone')->default('UTC')->after('username');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('timezone');
        });
    }
};
```

database/migrations/2026_10_20_014928_create_api_credentials_table.php (new file)

Created by `php artisan make:model ApiCredential -m`. Fill it with:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('api_credentials', function (Blueprint $table) {
            $table->id();
            $table->text('generated_key')->nullable();
            $table->string('library_fingerprint', 64)->nullable();
```

```
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('api_credentials');
    }
};
```

`database/seeders/DatabaseSeeder.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -20,5 +20,5 @@
         User::factory()->create([
             'name' => 'Test User',
-            'email' => 'test@example.com',
+            'username' => 'test',
         ]);
     }
```

`phpunit.xml` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -33,4 +33,8 @@
         <env name="TELESCOPE_ENABLED" value="false"/>
         <env name="NIGHTWATCH_ENABLED" value="false"/>
+        <env name="INERTIA_SSR_ENABLED" value="(false)"/>
+        <env name="ESSENSE_API_URL" value="https://essense.test"/>
+        <env name="API_KEY_ESSENCE" value="test-key"/>
+        <env name="REMOVE_LIMITS" value="(false)"/>
     </php>
 </phpunit>
```

`routes/settings.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -10,7 +10,4 @@
     Route::get('settings/profile', [ProfileController::class, 'edit'])->name('profile.edit');
     Route::patch('settings/profile', [ProfileController::class, 'update'])->name('profile.update');
-});
-
-Route::middleware(['auth', 'verified'])->group(function () {
     Route::delete('settings/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
```

`routes/web.php` (edit)

Replace the whole file with:

```
<?php

use App\Http\Controllers\GameController;
use App\Http\Controllers\GameVoteController;
use Illuminate\Support\Facades\Route;

Route::inertia('/', 'welcome')->name('home');

Route::middleware(['auth'])->group(function () {
    Route::get('dashboard', [GameController::class, 'index'])->name('dashboard');
```

```
    Route::middleware('throttle:30,1')->group(function () {
        Route::post('games', [GameController::class, 'store'])->name('games.store');
        Route::delete('games/{game}', [GameController::class,
        ↪  'destroy'])->whereNumber('game')->name('games.destroy');

        Route::post('games/{game}/vote', [GameVoteController::class,
        ↪  'store'])->whereNumber('game')->name('games.vote.store');
        Route::delete('games/{game}/vote', [GameVoteController::class,
        ↪  'destroy'])->whereNumber('game')->name('games.vote.destroy');
    });
});

require __DIR__.'/settings.php';
```

`package.json` (edit)

Apply these changes (- remove, + add, other lines are context). Keep the kit's `vite-plus` version (1.4.0 at the time of writing): it bundles its own copy of Vite internally, so it works alongside the real `vite@^8` package and is still used for `npm run check`. GameVote's repo pins an older `^0.3.3`; that's only because of when it was built. If `npm run check` flags formatting in files copied from the repo, run `npm run check:fix`:

```
@@ -4,7 +4,7 @@
     "type": "module",
     "scripts": {
-        "build": "vp build",
-        "build:ssr": "vp build && vp build --ssr",
-        "dev": "vp dev",
+        "build": "vite build",
+        "build:ssr": "vite build && vite build --ssr",
+        "dev": "vite",
         "check": "vp check",
         "check:fix": "vp check --fix",
@@ -43,5 +43,5 @@
         "tw-animate-css": "^1.4.0",
         "typescript": "^5.7.2",
-        "vite": "npm:@voidzero-dev/vite-plus-core@1.4.0"
+        "vite": "^8.0.0"
     },
     "devDependencies": {
@@ -54,13 +54,8 @@
     "optionalDependencies": {
         "@laravel/multiplex": "^0.4.1",
-        "@rollup/rollup-linux-x64-gnu": "4.9.5",
-        "@rollup/rollup-win32-x64-msvc": "4.9.5",
         "@tailwindcss/oxide-linux-x64-gnu": "^4.0.1",
         "@tailwindcss/oxide-win32-x64-msvc": "^4.0.1",
         "lightningcss-linux-x64-gnu": "^1.29.1",
         "lightningcss-win32-x64-msvc": "^1.29.1"
-    },
-    "overrides": {
-        "vite": "npm:@voidzero-dev/vite-plus-core@1.4.0"
     }
 }
```

`resources/css/app.css` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -14,4 +14,7 @@
         'Noto Color Emoji';

+    --font-spray:
+        'Rubik Spray Paint', 'Instrument Sans', ui-sans-serif, sans-serif;
+
```

```
     --radius-lg: var(--radius);
     --radius-md: calc(var(--radius) - 2px);
@@ -142,2 +145,103 @@
     }
 }
+
+@layer components {
+    /* A painted brick wall for the spray-painted page title (running bond, offset rows). */
+    .graffiti-wall {
+        background-color: #c9b9ac;
+        background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
 ↪  width='64' height='32'%3E%3Crect width='64' height='32' fill='%23c9b9ac'/%3E%3Crect x='1' y='1'
 ↪  width='62' height='14' rx='1' fill='%23eee6de'/%3E%3Crect x='-31' y='17' width='62' height='14'
 ↪  rx='1' fill='%23eee6de'/%3E%3Crect x='33' y='17' width='62' height='14' rx='1'
 ↪  fill='%23eee6de'/%3E%3C/svg%3E");
+    }
+
+    .dark .graffiti-wall {
+        background-color: #110e0d;
+        background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
 ↪  width='64' height='32'%3E%3Crect width='64' height='32' fill='%23110e0d'/%3E%3Crect x='1' y='1'
 ↪  width='62' height='14' rx='1' fill='%233b322d'/%3E%3Crect x='-31' y='17' width='62' height='14'
 ↪  rx='1' fill='%233b322d'/%3E%3Crect x='33' y='17' width='62' height='14' rx='1'
 ↪  fill='%233b322d'/%3E%3C/svg%3E");
+    }
+
+    /* The spray paint colour, shared by the title and the vote bars. Kept as plain RGB:
+       transparent shades are written rgb(var(--spray-rgb) / alpha) rather than with
+       color-mix(), which older mobile browsers drop entirely when it contains a variable. */
+    :root {
+        --spray-rgb: 232 23 155;
+        --spray: rgb(var(--spray-rgb));
+        --spray-fill-from: 0.16;
+        --spray-fill-to: 0.38;
+    }
+
+    .dark {
+        --spray-rgb: 89 222 101;
+        --spray-fill-from: 0.22;
+        --spray-fill-to: 0.5;
+    }
+
+    .spray-paint {
+        font-family: var(--font-spray);
+        color: var(--spray);
+        text-shadow:
+            0 0 1px rgb(var(--spray-rgb)),
+            0 0 6px rgb(var(--spray-rgb) / 0.55),
+            0 0 18px rgb(var(--spray-rgb) / 0.3);
+    }
+
+    /* Let the browser skip layout and painting for rows scrolled out of view; a long list
+       otherwise relays out every row after each vote. */
+    .games-list > li {
+        content-visibility: auto;
+        contain-intrinsic-size: auto 92px;
+    }
+
+    /* While one game's request is pending, dim the other rows and ignore clicks on them. */
+    .games-list[data-busy] > li:not([data-pending]) {
+        opacity: 0.5;
+        pointer-events: none;
+    }
+
+    /* The filled part of a vote bar, sprayed in the title's colour. */
+    .vote-bar-fill {
+        background: linear-gradient(
+            90deg,
+            rgb(var(--spray-rgb) / var(--spray-fill-from)),
+            rgb(var(--spray-rgb) / var(--spray-fill-to))
+        );
```

```
+        box-shadow: inset -2px 0 0 rgb(var(--spray-rgb) / 0.8);
+    }
+
+    /* A run of paint from a letter; its length comes from --drip. */
+    .paint-drip {
+        position: absolute;
+        top: 70%;
+        width: 0.3rem;
+        height: var(--drip, 1.5rem);
+        border-radius: 0 0 999px 999px;
+        background: linear-gradient(
+            rgb(var(--spray-rgb)) 60%,
+            rgb(var(--spray-rgb) / 0.7)
+        );
+        filter: blur(0.3px);
+    }
+
+    .paint-drip::after {
+        content: '';
+        position: absolute;
+        bottom: -0.15rem;
+        left: 50%;
+        width: 0.45rem;
+        height: 0.5rem;
+        border-radius: 50% 50% 50% 50% / 40% 40% 60% 60%;
+        background: currentColor;
+        transform: translateX(-50%);
+    }
+}
+
+/* A brief glow on list rows whose votes just changed. Two identical keyframes let the
+   same row restart the animation on consecutive changes. */
+@keyframes row-changed-a {
+    from {
+        background-color: rgb(var(--spray-rgb) / 0.22);
+    }
+}
+
+@keyframes row-changed-b {
+    from {
+        background-color: rgb(var(--spray-rgb) / 0.22);
+    }
+}
```

`resources/js/app.tsx` (edit)

Replace the whole file with:

```
import { createInertiaApp, router } from '@inertiajs/react';
import { toast } from 'sonner';
import { Toaster } from '@/components/ui/sonner';
import { TooltipProvider } from '@/components/ui/tooltip';
import { initializeTheme } from '@/hooks/use-appearance';
import AppLayout from '@/layouts/app-layout';
import AuthLayout from '@/layouts/auth-layout';
import SettingsLayout from '@/layouts/settings/layout';

const appName = import.meta.env.VITE_APP_NAME || 'GameVote';

void createInertiaApp({
    title: (title) => (title ? `${title} - ${appName}` : appName),
    layout: (name) => {
        switch (true) {
            case name === 'welcome':
                return null;
            case name.startsWith('auth/'):
                return AuthLayout;
            case name.startsWith('settings/'):
                return [AppLayout, SettingsLayout];
            default:
```

```
                return AppLayout;
        }
    },
    strictMode: true,
    withApp(app) {
        return (
            <TooltipProvider delayDuration={0}>
                {app}
                <Toaster />
            </TooltipProvider>
        );
    },
    progress: {
        color: '#4B5563',
    },
});

// This will set light / dark mode on load...
initializeTheme();

const httpErrorMessages: Record<number, string> = {
    419: 'Your session has expired. Refresh the page and try again.',
    429: 'You’re doing that too often. Wait a moment and try again.',
    503: 'The app is down for maintenance. Please try again shortly.',
};

// Surface failures as toasts instead of failing silently or leaving the page.
router.on('networkError', () => {
    toast.error(
        'Couldn’t reach the server. Check your connection and try again.',
    );
});

router.on('httpException', (event) => {
    // Keep Laravel's detailed error page in development.
    if (import.meta.env.DEV) {
        return;
    }

    event.preventDefault();
    toast.error(
        httpErrorMessages[event.detail.response.status] ??
            'Something went wrong on our end. Please try again.',
    );
});
```

`resources/js/components/add-game-form.tsx` (new file)

Create it with this content:

```
import { Form } from '@inertiajs/react';
import { Plus } from 'lucide-react';
import GameController from '@/actions/App/Http/Controllers/GameController';
import InputError from '@/components/input-error';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Spinner } from '@/components/ui/spinner';

export default function AddGameForm({
    disabled,
    limited,
}: {
    disabled: boolean;
    limited: boolean;
}) {
    return (
        <Form
            {...GameController.store.form()}
```

```
            options={{ preserveScroll: true }}
            resetOnSuccess
            className="grid gap-2"
        >
            {({ processing, errors }) => (
                <>
                    <Label htmlFor="name">Suggest a game</Label>
                    <div className="flex flex-col gap-2 sm:flex-row">
                        <Input
                            id="name"
                            name="name"
                            required
                            maxLength={100}
                            autoComplete="off"
                            placeholder="e.g. Ticket to Ride"
                            aria-invalid={errors.name ? true : undefined}
                            aria-describedby="name-hint"
                            disabled={disabled}
                        />
                        <Button
                            type="submit"
                            disabled={disabled || processing}
                            className="sm:w-auto"
                            data-test="add-game-button"
                        >
                            {processing ? <Spinner /> : <Plus />}
                            Add game
                        </Button>
                    </div>
                    <InputError message={errors.name} />
                    <p id="name-hint" className="text-xs text-muted-foreground">
                        {disabled
                            ? 'You can suggest a game again after midnight.'
                            : limited
                              ? 'Adding a game uses your action for today.'
                              : 'Add as many games as you like while limits are off.'}
                    </p>
                </>
            )}
        </Form>
    );
}
```

`resources/js/components/app-logo-icon.tsx` (edit)

Replace the whole file with:

```
import type { SVGProps } from 'react';
import Meeple from '@/components/meeple';

/** The GameVote logo: a meeple, the piece used for votes. */
export default function AppLogoIcon(props: SVGProps<SVGSVGElement>) {
    return <Meeple {...props} />;
}
```

`resources/js/components/app-top-bar.tsx` (new file)

Create it with this content:

```
import { Link, router, usePage } from '@inertiajs/react';
import { LogOut, Settings } from 'lucide-react';
import AppLogoIcon from '@/components/app-logo-icon';
import { Button } from '@/components/ui/button';
import { dashboard, logout } from '@/routes';
import { edit } from '@/routes/profile';

export default function AppTopBar() {
```

```
    const { auth, name } = usePage().props;

    return (
        <header className="sticky top-0 z-20 border-b bg-background/85 backdrop-blur">
            <div className="mx-auto flex h-14 w-full max-w-3xl items-center gap-2 px-4">
                <Link
                    href={dashboard()}
                    prefetch
                    className="mr-auto flex items-center gap-2 font-semibold"
                >
                    <AppLogoIcon className="size-6 text-red-500" />
                    {name}
                </Link>

                <span className="hidden text-sm text-muted-foreground sm:inline">
                    @{auth.user.username}
                </span>

                <Button variant="ghost" size="icon" asChild>
                    <Link href={edit()} prefetch aria-label="Settings">
                        <Settings />
                    </Link>
                </Button>

                <Button variant="outline" size="sm" asChild>
                    <Link
                        href={logout()}
                        as="button"
                        onClick={() => router.flushAll()}
                        data-test="logout-button"
                    >
                        <LogOut />
                        Log out
                    </Link>
                </Button>
            </div>
        </header>
    );
}
```

`resources/js/components/daily-allowance-notice.tsx` (new file)

Create it with this content:

```
import { CalendarCheck, FlaskConical, Sparkles } from 'lucide-react';
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert';
import type { DailyAllowance, Game } from '@/types';

function formatResetTime(resetsAt: string, timezone: string): string {
    return new Intl.DateTimeFormat(undefined, {
        hour: 'numeric',
        minute: '2-digit',
        timeZone: timezone,
        timeZoneName: 'short',
    }).format(new Date(resetsAt));
}

export default function DailyAllowanceNotice({
    allowance,
    games,
    timezone,
}: {
    allowance: DailyAllowance;
    games: Game[] | null;
    timezone: string;
}) {
    if (!allowance.enforced) {
        return (
            <Alert>
```

```
                <FlaskConical />
                <AlertTitle>Daily limits are off</AlertTitle>
                <AlertDescription>
                    Testing mode (REMOVE_LIMITS=true): vote and add games as
                    often as you like. Duplicate titles and one vote per game
                    still apply.
                </AlertDescription>
            </Alert>
        );
    }

    if (allowance.available) {
        return (
            <Alert>
                <Sparkles />
                <AlertTitle>You have one action today</AlertTitle>
                <AlertDescription>
                    Vote for a game you want, or suggest a new one. You can do
                    one or the other each day.
                </AlertDescription>
            </Alert>
        );
    }

    const gameName = games?.find((game) => game.id === allowance.game_id)?.name;
    const spentOn =
        allowance.used_for === 'vote'
            ? gameName
                ? `You voted for ${gameName}`
                : 'You voted'
            : gameName
              ? `You added ${gameName}`
              : 'You added a game';

    return (
        <Alert>
            <CalendarCheck />
            <AlertTitle>{spentOn} today</AlertTitle>
            <AlertDescription>
                Your next vote or new game unlocks at midnight (
                {formatResetTime(allowance.resets_at, timezone)}).
                {allowance.used_for === 'vote' &&
                    ' Changed your mind? Remove today’s vote to get your action back.'}
            </AlertDescription>
        </Alert>
    );
}
```

`resources/js/components/game-list-item.tsx` (new file)

Create it with this content:

```
import { Form } from '@inertiajs/react';
import { memo, useState } from 'react';
import { Check, ThumbsUp, Trash2, Undo2 } from 'lucide-react';
import GameController from '@/actions/App/Http/Controllers/GameController';
import GameVoteController from '@/actions/App/Http/Controllers/GameVoteController';
import MeepleVotes from '@/components/meeple-votes';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import {
    Dialog,
    DialogClose,
    DialogContent,
    DialogDescription,
    DialogFooter,
    DialogTitle,
    DialogTrigger,
} from '@/components/ui/dialog';
```

```
import { Spinner } from '@/components/ui/spinner';
import { cn } from '@/lib/utils';
import type { Game } from '@/types';

function votesLabel(votes: number): string {
    return votes === 1 ? '1 vote' : `${votes} votes`;
}

function RemoveGameDialog({
    game,
    onPendingChange,
}: {
    game: Game;
    onPendingChange: (pending: boolean) => void;
}) {
    return (
        <Dialog>
            <DialogTrigger asChild>
                <Button
                    variant="ghost"
                    size="icon"
                    className="text-muted-foreground hover:text-destructive"
                    aria-label={`Remove ${game.name}`}
                    data-test="remove-game-button"
                >
                    <Trash2 />
                </Button>
            </DialogTrigger>
            <DialogContent>
                <DialogTitle>Remove “{game.name}”?</DialogTitle>
                <DialogDescription>
                    This removes the game for everyone
                    {game.votes > 0 &&
                        ` and discards its ${votesLabel(game.votes)}`}
                    . Anyone who used today’s action on it gets that action
                    back.
                </DialogDescription>
                <Form
                    {...GameController.destroy.form(game.id)}
                    options={{ preserveScroll: true }}
                    onStart={() => onPendingChange(true)}
                    onFinish={() => onPendingChange(false)}
                >
                    {({ processing }) => (
                        <DialogFooter className="gap-2">
                            <DialogClose asChild>
                                <Button variant="secondary">Cancel</Button>
                            </DialogClose>
                            <Button
                                type="submit"
                                variant="destructive"
                                disabled={processing}
                                data-test="confirm-remove-game-button"
                            >
                                {processing && <Spinner />}
                                Remove game
                            </Button>
                        </DialogFooter>
                    )}
                </Form>
            </DialogContent>
        </Dialog>
    );
}

type Props = {
    ref?: React.Ref<HTMLLIElement>;
    game: Game;
    rank: number;
    maxVotes: number;
```

```
    canVote: boolean;
    /** Increments each time this row's votes change, to replay the highlight; null for no highlight. */
    highlightRun: number | null;
    /** Reports when this row's request starts and finishes; the list dims the other rows meanwhile. */
    onPendingChange: (pending: boolean) => void;
};

function GameListItem({
    ref,
    game,
    rank,
    maxVotes,
    canVote,
    highlightRun,
    onPendingChange,
}: Props) {
    const [pending, setPending] = useState(false);
    const reportPending = (isPending: boolean) => {
        setPending(isPending);
        onPendingChange(isPending);
    };

    return (
        <li
            ref={ref}
            data-pending={pending || undefined}
            className={cn(
                'relative flex flex-col gap-3 bg-background p-4 transition-opacity sm:flex-row
↪  sm:items-center',
                game.voted && 'bg-primary/[0.03] dark:bg-primary/[0.06]',
            )}
            style={
                highlightRun === null
                    ? undefined
                    : {
                          animation: `${highlightRun % 2 ? 'row-changed-a' : 'row-changed-b'} 1.8s
↪  ease-out`,
                      }
            }
        >
            <div className="flex min-w-0 flex-1 items-center gap-3">
                <span
                    className="w-6 shrink-0 text-right text-sm text-muted-foreground tabular-nums"
                    aria-hidden="true"
                >
                    {rank}
                </span>
                <div className="min-w-0 flex-1 space-y-1">
                   {/* The badge sits on the title line so every vote bar gets the same track width. */}
                    <div className="flex min-w-0 items-center gap-2">
                        <p className="truncate font-medium" title={game.name}>
                            {game.name}
                        </p>
                        {game.voted && (
                            <Badge variant="secondary">
                                <Check /> Your vote
                            </Badge>
                        )}
                    </div>
                    {/* On phones, end where the vote button ends: leave room for the remove button
↪  (size-9) and its gap. */}
                    <MeepleVotes
                        votes={game.votes}
                        maxVotes={maxVotes}
                        className="mr-11 sm:mr-0"
                    />
                </div>
            </div>

            <div className="flex items-center gap-2 pl-9 sm:pl-0">
```

```
                {game.voted ? (
                    <Form
                        {...GameVoteController.destroy.form(game.id)}
                        options={{ preserveScroll: true }}
                        onStart={() => reportPending(true)}
                        onFinish={() => reportPending(false)}
                        className="flex-1 sm:w-36 sm:flex-none"
                    >
                        {({ processing }) => (
                            <Button
                                type="submit"
                                variant="outline"
                                disabled={processing}
                                className="w-full"
                                data-test="remove-vote-button"
                            >
                                {processing ? <Spinner /> : <Undo2 />}
                                Remove vote
                            </Button>
                        )}
                    </Form>
                ) : (
                    <Form
                        {...GameVoteController.store.form(game.id)}
                        options={{ preserveScroll: true }}
                        onStart={() => reportPending(true)}
                        onFinish={() => reportPending(false)}
                        className="flex-1 sm:w-36 sm:flex-none"
                    >
                        {({ processing }) => (
                            <Button
                                type="submit"
                                disabled={!canVote || processing}
                                title={
                                    canVote
                                        ? undefined
                                        : 'You have used today’s action'
                                }
                                className="w-full"
                                data-test="vote-button"
                            >
                                {processing ? <Spinner /> : <ThumbsUp />}
                                Vote
                            </Button>
                        )}
                    </Form>
                )}
                <RemoveGameDialog game={game} onPendingChange={reportPending} />
            </div>
        </li>
    );
}

/**
 * The server sends new objects for every game after each request, so compare the game's
 * fields rather than its identity: only rows whose data, rank or highlight changed re-render.
 */
export default memo(
    GameListItem,
    (previous, next) =>
        previous.game.id === next.game.id &&
        previous.game.name === next.game.name &&
        previous.game.votes === next.game.votes &&
        previous.game.voted === next.game.voted &&
        previous.rank === next.rank &&
        previous.maxVotes === next.maxVotes &&
        previous.canVote === next.canVote &&
        previous.highlightRun === next.highlightRun &&
        previous.ref === next.ref &&
        previous.onPendingChange === next.onPendingChange,
```

```
);
```

`resources/js/components/meeple-votes.tsx` (new file)

Create it with this content:

```
import Meeple from '@/components/meeple';
import { cn } from '@/lib/utils';

/** Classic board game piece colours, cycled so the bar looks like a row of meeples. */
const colors = [
    'text-red-500',
    'text-blue-500',
    'text-yellow-400',
    'text-green-600',
    'text-orange-500',
    'text-purple-500',
    'text-neutral-800 dark:text-neutral-200',
];

/** About as many meeples as the widest bar can show; any that don't fit stay hidden. */
const maxRendered = 20;

/**
 * A bar chart row: the bar's length is this game's share of the top game's votes, filled with
 * meeples (never more than the votes), with the exact total on the right.
 */
export default function MeepleVotes({
    votes,
    maxVotes,
    className,
}: {
    votes: number;
    maxVotes: number;
    className?: string;
}) {
    const label =
        votes === 0
            ? 'No votes yet'
            : votes === 1
              ? '1 vote'
              : `${votes} votes`;
    const share = maxVotes > 0 ? Math.min(100, (votes / maxVotes) * 100) : 0;

    return (
        <div
            role="img"
            aria-label={label}
            title={label}
            className={cn(
                'flex h-8 items-center gap-2 overflow-hidden rounded-md bg-muted/60 pr-2',
                className,
            )}
        >
            <div className="h-full min-w-0 flex-1">
                {votes === 0 ? (
                    <span className="flex h-full items-center px-2 text-sm text-muted-foreground">
                        No votes yet
                    </span>
                ) : (
                    // One row tall: meeples that don't fit wrap onto a hidden second row,
                    // so the fill only ever shows whole meeples.
                    <div
                        className="vote-bar-fill flex h-full flex-wrap gap-x-0.5 overflow-hidden
↪   rounded-md px-1 transition-[width] duration-500 ease-out"
                        style={{ width: `max(1.75rem, ${share}%)` }}
                    >
                        {Array.from(
                            { length: Math.min(votes, maxRendered) },
```

```
                            (_, index) => (
                                <span
                                    key={index}
                                    className="flex h-full shrink-0 items-center"
                                >
                                    <Meeple
                                        className={cn(
                                            'size-5 drop-shadow-sm sm:size-6',
                                            colors[index % colors.length],
                                        )}
                                    />
                                </span>
                            ),
                        )}
                    </div>
                )}
            </div>
            {votes > 0 && (
                <span className="shrink-0 text-sm font-semibold tabular-nums">
                    {votes}
                </span>
            )}
        </div>
    );
}
```

`resources/js/components/meeple.tsx` (new file)

Create it with this content:

```
import type { SVGProps } from 'react';

/** A classic board game meeple silhouette. */
export default function Meeple(props: SVGProps<SVGSVGElement>) {
    return (
        <svg viewBox="0 0 100 100" aria-hidden="true" {...props}>
            <path
                fill="currentColor"
                d="M50 6c-9 0-15 7-15 15 0 5 2 9 5 12-12 3-27 8-33 14-4 4-1 10 4 10 8 0 16-2 22-4L19
↪  85c-2 5 1 9 6 9h14c4 0 7-2 8-6l3-12 3 12c1 4 4 6 8 6h14c5 0 8-4 6-9L67 53c6 2 14 4 22 4 5 0 8-6
↪  4-10-6-6-21-11-33-14 3-3 5-7 5-12 0-8-6-15-15-15z"
            />
        </svg>
    );
}
```

`resources/js/components/spray-painted-title.tsx` (new file)

Create it with this content:

```
import { cn } from '@/lib/utils';

/** Paint runs under the title, as [left position %, length in rem]. */
const drips: Array<[number, number]> = [
    [9, 1.4],
    [27, 2.4],
    [46, 1.1],
    [63, 2],
    [84, 1.6],
];

export default function SprayPaintedTitle({
    title,
    description,
    className,
}: {
    title: string;
```

```
    description?: string;
    className?: string;
}) {
    return (
        <div
            className={cn(
                'graffiti-wall overflow-hidden rounded-xl border px-5 pt-6 pb-8 sm:px-8',
                className,
            )}
        >
            <h1 className="spray-paint relative inline-block -rotate-2 text-4xl leading-tight
↪   sm:text-6xl">
                {title}
                <span aria-hidden="true">
                    {drips.map(([left, length]) => (
                        <span
                            key={left}
                            className="paint-drip"
                            style={
                                {
                                    left: `${left}%`,
                                    '--drip': `${length}rem`,
                                } as React.CSSProperties
                            }
                        />
                    ))}
                </span>
            </h1>
            {description && (
                <p className="mt-4 max-w-md rounded-md bg-background/80 px-3 py-2 text-sm
↪   text-muted-foreground backdrop-blur-sm">
                    {description}
                </p>
            )}
        </div>
    );
}
```

`resources/js/hooks/use-flip-animation.ts` (new file)

Create it with this content:

```
import { useCallback, useLayoutEffect, useRef } from 'react';

type Key = string | number;

/**
 * Slide list items from their old position to their new one when the list is reordered
 * (the FLIP technique), so a re-sort is visible instead of rows jumping under the cursor.
 *
 * Attach the returned ref callback to each item; it is the same function for a key on every
 * render, so it doesn't make memoized items re-render. Positions are read with offsetTop, so
 * the list should be the items' offset parent (give it `position: relative`).
 */
export function useFlipAnimation<T extends HTMLElement>(order: readonly Key[]) {
    const elements = useRef(new Map<Key, T>());
    const positions = useRef(new Map<Key, number>());
    const refCallbacks = useRef(new Map<Key, (element: T | null) => void>());
    const orderKey = order.join(',');

    useLayoutEffect(() => {
        const reduceMotion = window.matchMedia(
            '(prefers-reduced-motion: reduce)',
        ).matches;
        const next = new Map<Key, number>();

        elements.current.forEach((element, key) => {
            const top = element.offsetTop;
            const previous = positions.current.get(key);
```

```
            next.set(key, top);

            if (reduceMotion || previous === undefined || previous === top) {
                return;
            }

            element.animate(
                [
                    { transform: `translateY(${previous - top}px)` },
                    { transform: 'translateY(0)' },
                ],
                { duration: 300, easing: 'cubic-bezier(0.2, 0, 0, 1)' },
            );
        });

        positions.current = next;
    }, [orderKey]);

    return useCallback((key: Key) => {
        let callback = refCallbacks.current.get(key);

        if (!callback) {
            callback = (element: T | null) => {
                if (element) {
                    elements.current.set(key, element);
                } else {
                    elements.current.delete(key);
                    refCallbacks.current.delete(key);
                }
            };
            refCallbacks.current.set(key, callback);
        }

        return callback;
    }, []);
}
```

resources/js/layouts/app-layout.tsx (edit)

Replace the whole file with:

```
import AppTopBar from '@/components/app-top-bar';

export default function AppLayout({ children }: { children: React.ReactNode }) {
    return (
        <div className="flex min-h-screen flex-col">
            <AppTopBar />
            <main className="mx-auto flex w-full max-w-3xl flex-1 flex-col px-4 py-6">
                {children}
            </main>
        </div>
    );
}
```

resources/js/pages/auth/login.tsx (edit)

Replace the whole file with:

```
import { Form, Head } from '@inertiajs/react';
import InputError from '@/components/input-error';
import PasswordInput from '@/components/password-input';
import TextLink from '@/components/text-link';
import { Button } from '@/components/ui/button';
import { Checkbox } from '@/components/ui/checkbox';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Spinner } from '@/components/ui/spinner';
```

```
import { register } from '@/routes';
import { store } from '@/routes/login';

type Props = {
    status?: string;
};

export default function Login({ status }: Props) {
    return (
        <>
            <Head title="Log in" />

            <Form
                {...store.form()}
                resetOnSuccess={['password']}
                className="flex flex-col gap-6"
            >
                {({ processing, errors }) => (
                    <>
                        <div className="grid gap-6">
                            <div className="grid gap-2">
                                <Label htmlFor="username">Username</Label>
                                <Input
                                    id="username"
                                    type="text"
                                    name="username"
                                    required
                                    autoFocus
                                    tabIndex={1}
                                    autoComplete="username"
                                    autoCapitalize="none"
                                    spellCheck={false}
                                    placeholder="Username"
                                />
                                <InputError message={errors.username} />
                            </div>

                            <div className="grid gap-2">
                                <Label htmlFor="password">Password</Label>
                                <PasswordInput
                                    id="password"
                                    name="password"
                                    required
                                    tabIndex={2}
                                    autoComplete="current-password"
                                    placeholder="Password"
                                />
                                <InputError message={errors.password} />
                            </div>

                            <div className="flex items-center space-x-3">
                                <Checkbox
                                    id="remember"
                                    name="remember"
                                    tabIndex={3}
                                />
                                <Label htmlFor="remember">Remember me</Label>
                            </div>

                            <Button
                                type="submit"
                                className="mt-4 w-full"
                                tabIndex={4}
                                disabled={processing}
                                data-test="login-button"
                            >
                                {processing && <Spinner />}
                                Log in
                            </Button>
                        </div>
```

```
                        <div className="text-center text-sm text-muted-foreground">
                            Don't have an account?{' '}
                            <TextLink href={register()} tabIndex={5}>
                                Sign up
                            </TextLink>
                        </div>
                    </>
                )}
            </Form>

            {status && (
                <div className="mb-4 text-center text-sm font-medium text-green-600">
                    {status}
                </div>
            )}
        </>
    );
}

Login.layout = {
    title: 'Log in to your account',
    description: 'Enter your username and password below to log in',
};
```

`resources/js/pages/auth/register.tsx` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -26,4 +26,11 @@
                 {({ processing, errors }) => (
                     <>
+                        <input
+                            type="hidden"
+                            name="timezone"
+                            value={
+                                Intl.DateTimeFormat().resolvedOptions().timeZone
+                            }
+                        />
                         <div className="grid gap-6">
                             <div className="grid gap-2">
@@ -46,15 +53,17 @@

                                 <div className="grid gap-2">
-                                    <Label htmlFor="email">Email address</Label>
+                                    <Label htmlFor="username">Username</Label>

-                                    <Input
-                                        id="email"
-                                        type="email"
+                                    <Input
+                                        id="username"
+                                        type="text"
                                         required
                                         tabIndex={2}
-                                        autoComplete="email"
-                                        name="email"
-                                        placeholder="email@example.com"
+                                        autoComplete="username"
+                                        autoCapitalize="none"
+                                        spellCheck={false}
+                                        name="username"
+                                        placeholder="Username"
                                     />
-                                    <InputError message={errors.email} />
+                                    <InputError message={errors.username} />
                                 </div>
```

`resources/js/pages/dashboard.tsx` (edit)

Replace the whole file with:

```
import { Head, router } from '@inertiajs/react';
import { AlertCircle, Gamepad2, RotateCw } from 'lucide-react';
import { useState } from 'react';
import AddGameForm from '@/components/add-game-form';
import DailyAllowanceNotice from '@/components/daily-allowance-notice';
import GameListItem from '@/components/game-list-item';
import SprayPaintedTitle from '@/components/spray-painted-title';
import { useFlipAnimation } from '@/hooks/use-flip-animation';
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert';
import { Button } from '@/components/ui/button';
import { Spinner } from '@/components/ui/spinner';
import type { DailyAllowance, Game } from '@/types';

type Props = {
    games: Game[] | null;
    libraryError: string | null;
    allowance: DailyAllowance;
    timezone: string;
};

function LibraryUnavailable({ message }: { message: string }) {
    const [retrying, setRetrying] = useState(false);

    return (
        <Alert variant="destructive">
            <AlertCircle />
            <AlertTitle>Couldn’t load the game library</AlertTitle>
            <AlertDescription>
                <p>{message}</p>
                <Button
                    variant="outline"
                    size="sm"
                    className="mt-2"
                    disabled={retrying}
                    onClick={() =>
                        router.reload({
                            onStart: () => setRetrying(true),
                            onFinish: () => setRetrying(false),
                        })
                    }
                >
                    {retrying ? <Spinner /> : <RotateCw />}
                    Try again
                </Button>
            </AlertDescription>
        </Alert>
    );
}

function EmptyLibrary() {
    return (
        <div className="flex flex-col items-center gap-2 rounded-xl border border-dashed px-6 py-12
↪  text-center">
            <Gamepad2 className="size-8 text-muted-foreground" />
            <p className="font-medium">No games yet</p>
            <p className="max-w-xs text-sm text-muted-foreground">
                Suggest the first game for the office library above.
            </p>
        </div>
    );
}

export default function Dashboard({
    games,
    libraryError,
    allowance,
    timezone,
```

```
}: Props) {
    const maxVotes = Math.max(0, ...(games ?? []).map((game) => game.votes));
    const flipRef = useFlipAnimation<HTMLLIElement>(
        (games ?? []).map((game) => game.id),
    );
    const [busy, setBusy] = useState(false);

    // Highlight rows whose votes changed (or that are new) since the previous list.
    const [previousGames, setPreviousGames] = useState(games);
    const [highlight, setHighlight] = useState({
        ids: [] as number[],
        run: 0,
    });

    if (games !== previousGames) {
        setPreviousGames(games);

        if (games && previousGames) {
            const before = new Map(
                previousGames.map((game) => [game.id, game.votes]),
            );
            const changed = games
                .filter((game) => before.get(game.id) !== game.votes)
                .map((game) => game.id);

            if (changed.length > 0) {
                setHighlight(({ run }) => ({ ids: changed, run: run + 1 }));
            }
        }
    }

    return (
        <>
            <Head title="Game library" />
            <div className="flex flex-col gap-6">
                <SprayPaintedTitle
                    title="Game Library"
                  description="Suggest games for the office library and vote for the ones you want most."
                />

                <DailyAllowanceNotice
                    allowance={allowance}
                    games={games}
                    timezone={timezone}
                />

                <AddGameForm
                    disabled={!allowance.available}
                    limited={allowance.enforced}
                />

                <section aria-labelledby="games-heading" className="grid gap-3">
                    <h2 id="games-heading" className="text-sm font-medium">
                        Most wanted
                        {games && games.length > 0 && (
                            <span className="ml-1 text-muted-foreground">
                                ({games.length})
                            </span>
                        )}
                    </h2>

                    {libraryError !== null || games === null ? (
                        <LibraryUnavailable
                            message={
                                libraryError ??
                                'The game library could not be loaded.'
                            }
                        />
                    ) : games.length === 0 ? (
                        <EmptyLibrary />
```

```
                    ) : (
                        <ol
                            className="games-list relative divide-y overflow-hidden rounded-xl border"
                            data-busy={busy || undefined}
                            aria-busy={busy || undefined}
                        >
                            {games.map((game, index) => (
                                <GameListItem
                                    key={game.id}
                                    ref={flipRef(game.id)}
                                    game={game}
                                    rank={index + 1}
                                    maxVotes={maxVotes}
                                    canVote={allowance.available}
                                    highlightRun={
                                        highlight.ids.includes(game.id)
                                            ? highlight.run
                                            : null
                                    }
                                    onPendingChange={setBusy}
                                />
                            ))}
                        </ol>
                    )}
                </section>
            </div>
        </>
    );
}
```

`resources/js/pages/settings/appearance.tsx` (edit)

Replace the whole file with:

```
import { Head } from '@inertiajs/react';
import AppearanceTabs from '@/components/appearance-tabs';
import Heading from '@/components/heading';

export default function Appearance() {
    return (
        <>
            <Head title="Appearance settings" />

            <h1 className="sr-only">Appearance settings</h1>

            <div className="space-y-6">
                <Heading
                    variant="small"
                    title="Appearance settings"
                    description="Update the appearance settings for your account"
                />
                <AppearanceTabs />
            </div>
        </>
    );
}
```

`resources/js/pages/settings/profile.tsx` (edit)

Replace the whole file with:

```
import { Form, Head, usePage } from '@inertiajs/react';
import ProfileController from '@/actions/App/Http/Controllers/Settings/ProfileController';
import DeleteUser from '@/components/delete-user';
import Heading from '@/components/heading';
import InputError from '@/components/input-error';
import { Button } from '@/components/ui/button';
```

```
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import type { Auth } from '@/types';

type PageProps = {
    auth: Auth;
};

export default function Profile() {
    const { auth } = usePage<PageProps>().props;

    return (
        <>
            <Head title="Profile settings" />

            <h1 className="sr-only">Profile settings</h1>

            <div className="space-y-6">
                <Heading
                    variant="small"
                    title="Profile"
                    description="Update your name and username"
                />

                <Form
                    {...ProfileController.update.form()}
                    options={{
                        preserveScroll: true,
                    }}
                    className="space-y-6"
                >
                    {({ processing, errors }) => (
                        <>
                            <div className="grid gap-2">
                                <Label htmlFor="name">Name</Label>

                                <Input
                                    id="name"
                                    className="mt-1 block w-full"
                                    defaultValue={auth.user.name}
                                    name="name"
                                    required
                                    autoComplete="name"
                                    placeholder="Full name"
                                />

                                <InputError
                                    className="mt-2"
                                    message={errors.name}
                                />
                            </div>

                            <div className="grid gap-2">
                                <Label htmlFor="username">Username</Label>

                                <Input
                                    id="username"
                                    className="mt-1 block w-full"
                                    defaultValue={auth.user.username}
                                    name="username"
                                    required
                                    autoComplete="username"
                                    autoCapitalize="none"
                                    spellCheck={false}
                                    placeholder="Username"
                                />

                                <InputError
                                    className="mt-2"
                                    message={errors.username}
```

```
                                />
                            </div>

                            <div className="flex items-center gap-4">
                                <Button
                                    disabled={processing}
                                    data-test="update-profile-button"
                                >
                                    Save
                                </Button>
                            </div>
                        </>
                    )}
                </Form>
            </div>

            <DeleteUser />
        </>
    );
}
```

`resources/js/pages/settings/security.tsx` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -7,5 +7,4 @@
 import { Button } from '@/components/ui/button';
 import { Label } from '@/components/ui/label';
-import { edit } from '@/routes/security';

 // oxfmt-ignore
@@ -123,11 +122,2 @@
     );
 }
-
-Security.layout = {
-    breadcrumbs: [
-        {
-            title: 'Security settings',
-            href: edit(),
-        },
-    ],
-};
```

`resources/js/pages/welcome.tsx` (edit)

Replace the whole file with:

```
import { Head, Link, usePage } from '@inertiajs/react';
import { Gamepad2, ThumbsUp, CalendarCheck } from 'lucide-react';
import AppLogoIcon from '@/components/app-logo-icon';
import { Button } from '@/components/ui/button';
import { dashboard, login, register } from '@/routes';

const steps = [
    {
        icon: Gamepad2,
        title: 'Suggest',
        description: 'Add a game you'd like to see in the office library.',
    },
    {
        icon: ThumbsUp,
        title: 'Vote',
        description: 'Back the games you want most. Top votes rise first.',
    },
    {
        icon: CalendarCheck,
```

```
        title: 'One a day',
        description: 'Each day you get one action: a vote or a new game.',
    },
];

export default function Welcome() {
    const { auth } = usePage().props;

    return (
        <>
            <Head title="Welcome" />
            <div className="flex min-h-screen flex-col bg-background text-foreground">
              <header className="mx-auto flex w-full max-w-4xl items-center justify-between p-4 md:p-6">
                    <div className="flex items-center gap-2 font-semibold">
                        <AppLogoIcon className="size-6 text-red-500" />
                        GameVote
                    </div>
                    <nav className="flex items-center gap-2">
                        {auth.user ? (
                            <Button asChild>
                                <Link href={dashboard()}>Open library</Link>
                            </Button>
                        ) : (
                            <>
                                <Button variant="ghost" asChild>
                                    <Link href={login()}>Log in</Link>
                                </Button>
                                <Button asChild>
                                    <Link href={register()}>Sign up</Link>
                                </Button>
                            </>
                        )}
                    </nav>
                </header>

                <main className="mx-auto flex w-full max-w-4xl flex-1 flex-col justify-center gap-10 p-4
↪  md:p-6">
                    <div className="max-w-xl space-y-4">
                        <h1 className="text-3xl font-semibold tracking-tight md:text-4xl">
                            Pick the next games for the office library
                        </h1>
                        <p className="text-muted-foreground">
                            Suggest titles, vote for your favourites, and see
                            what the team wants most.
                        </p>
                        <Button size="lg" asChild>
                            <Link href={auth.user ? dashboard() : register()}>
                                {auth.user ? 'Open the library' : 'Get started'}
                            </Link>
                        </Button>
                    </div>

                    <ul className="grid gap-4 sm:grid-cols-3">
                        {steps.map(({ icon: Icon, title, description }) => (
                            <li
                                key={title}
                                className="space-y-2 rounded-xl border p-4"
                            >
                                <Icon className="size-5 text-muted-foreground" />
                                <p className="font-medium">{title}</p>
                                <p className="text-sm text-muted-foreground">
                                    {description}
                                </p>
                            </li>
                        ))}
                    </ul>
                </main>
            </div>
        </>
    );
```

```
}
```

resources/js/types/auth.ts (edit)

Replace the whole file with:

```
export type User = {
    id: number;
    name: string;
    username: string;
    avatar?: string;
    created_at: string;
    updated_at: string;
    [key: string]: unknown;
};

export type Auth = {
    user: User;
};
```

resources/js/types/games.ts (new file)

Create it with this content:

```
export type Game = {
    id: number;
    name: string;
    votes: number;
    voted: boolean;
};

export type DailyAllowance = {
    enforced: boolean;
    available: boolean;
    used_for: 'vote' | 'add_game' | null;
    game_id: number | null;
    resets_at: string;
};
```

resources/js/types/global.d.ts (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -12,5 +12,4 @@
             name: string;
             auth: Auth;
-            sidebarOpen: boolean;
             [key: string]: unknown;
         };
```

resources/js/types/index.ts (edit)

Replace the whole file with:

```
export type * from './auth';
export type * from './navigation';
export type * from './ui';
export type * from './games';
```

resources/js/types/navigation.ts (edit)

Replace the whole file with:

```
import type { InertiaLinkProps } from '@inertiajs/react';
import type { LucideIcon } from 'lucide-react';

export type NavItem = {
    title: string;
    href: NonNullable<InertiaLinkProps['href']>;
    icon?: LucideIcon | null;
    isActive?: boolean;
};
```

`resources/js/types/ui.ts` (edit)

Replace the whole file with:

```
import type { ReactNode } from 'react';

export type FlashToast = {
    type: 'success' | 'info' | 'warning' | 'error';
    message: string;
};

export type AuthLayoutProps = {
    children?: ReactNode;
    name?: string;
    title?: string;
    description?: string;
};
```

`resources/views/app.blade.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -4,7 +4,9 @@
         <meta charset="utf-8">
         <meta name="viewport" content="width=device-width, initial-scale=1">
+        {{-- The app has its own light and dark themes, so browsers shouldn't auto-darken it. --}}
+        <meta name="color-scheme" content="light dark">

         {{-- Inline script to detect system dark mode preference and apply it immediately --}}
-        <script>
+        <script nonce="{{ Vite::cspNonce() }}">
             (function() {
                 const appearance = '{{ $appearance ?? "system" }}';
@@ -24,8 +26,10 @@
             html {
                 background-color: oklch(1 0 0);
+                color-scheme: light;
             }

             html.dark {
                 background-color: oklch(0.145 0 0);
+                color-scheme: dark;
             }
         </style>
```

`vite.config.ts` (edit)

Apply these changes (- remove, + add, other lines are context). Skip the second hunk if you don't keep a `docs/ai-transcript.md`:

```
@@ -17,4 +17,7 @@
                     weights: [400, 500, 600],
                 }),
+                bunny('Rubik Spray Paint', {
+                    weights: [400],
+                }),
```

```
            ],
        }),
@@ -69,4 +72,6 @@
            'resources/js/components/ui/*',
            'resources/views/mail/*',
+           // An exported transcript, kept exactly as generated.
+           'docs/ai-transcript.md',
        ],
        sortTailwindcss: {
```

Tests code

Every test file, A to Z by path, for the registration-only build. New files show their full content; edited kit tests show a diff against the kit, or the full file when most of it changed. Generate each new test with `php artisan make:test <Name>` first, then replace its body. `phpunit.xml` is on the Backend code tab.

`tests/FakeEssenseApi.php` (new file)

Create it by hand (no generator) with this content:

```php
<?php

namespace Tests;

use Illuminate\Http\Client\Request;
use Illuminate\Support\Facades\Http;

/**
 * An in-memory stand-in for the Essense code challenge API that mirrors its real behaviour:
 * form-encoded bodies, `{"error": CODE}` for key problems, HTTP 200 sentinels (`{}`,
 * `{"success": false}`, `{"id": -1}`) for everything else, ids that are never reused,
 * votes that never drop below zero, a list returned in id order rather than by votes, and
 * keys issued by `api-key/generate` that all stop working when the server restarts.
 */
class FakeEssenseApi
{
    public const KEY = 'test-key';

    /** @var array<int, array{id: int, name: string, votes: int}> */
    private array $games = [];

    private int $lastId = 0;

    /** @var list<string> */
    private array $validKeys = [self::KEY];

    /** @var list<string> */
    private array $issuedKeys = [];

    /**
     * @param  list<array{name: string, votes?: int}>  $games
     */
    public static function fake(array $games = []): self
    {
        $api = new self;

        foreach ($games as $game) {
            $id = ++$api->lastId;
            $api->games[$id] = ['id' => $id, 'name' => $game['name'], 'votes' => $game['votes'] ?? 0];
        }

        Http::fake(['essense.test/*' => fn (Request $request) => $api->handle($request)]);

        return $api;
    }

    /**
     * Make every request fail as if the API were unreachable.
     */
    public static function unreachable(): void
    {
        Http::fake(['essense.test/*' => Http::failedConnection()]);
    }

    /**
```

```
     * @return array{id: int, name: string, votes: int}|null
     */
    public function game(int $id): ?array
    {
        return $this->games[$id] ?? null;
    }

    public function idOf(string $name): int
    {
        foreach ($this->games as $game) {
            if ($game['name'] === $name) {
                return $game['id'];
            }
        }

        throw new \OutOfBoundsException("No fake game named [{$name}].");
    }

    public function count(): int
    {
        return count($this->games);
    }

    /**
     * @return list<string>
     */
    public function issuedKeys(): array
    {
        return $this->issuedKeys;
    }

    /**
     * Simulate the API server restarting: every key stops working and the library starts over.
     *
     * @param  list<array{name: string, votes?: int}>  $games  Games a new key's library should contain.
     */
    public function restart(array $games = []): void
    {
        $this->validKeys = [];
        $this->games = [];
        $this->lastId = 0;

        foreach ($games as $game) {
            $id = ++$this->lastId;
            $this->games[$id] = ['id' => $id, 'name' => $game['name'], 'votes' => $game['votes'] ?? 0];
        }
    }

    private function handle(Request $request): mixed
    {
        $data = $request->data();
        $endpoint = trim((string) parse_url($request->url(), PHP_URL_PATH), '/');

        if ($endpoint === 'api-key/generate') {
            $key = bin2hex(random_bytes(16));
            $this->validKeys[] = $this->issuedKeys[] = $key;

            return Http::response(['key' => $key]);
        }

        if (! isset($data['api_key'])) {
            return Http::response(['error' => 'MISSING_KEY'], 400);
        }

        if (! in_array($data['api_key'], $this->validKeys, true)) {
            return Http::response(['error' => 'INVALID_KEY'], 403);
        }

        $id = (int) ($data['id'] ?? 0);
        $exists = isset($this->games[$id]);
```

```
        return Http::response(match ($endpoint) {
            'games/list' => ['games' => array_values($this->games)],
            'games/search' => $exists ? $this->games[$id] : [],
            'games/add' => $this->add((string) ($data['name'] ?? '')),
            'games/remove' => $this->remove($id),
            'games/vote' => $this->adjustVotes($id, 1),
            'games/removeVote' => $this->adjustVotes($id, -1),
            default => throw new \LogicException("Unexpected endpoint [{$endpoint}]."),
        });
    }

    /**
     * @return array{id: int}
     */
    private function add(string $name): array
    {
        if ($name === '') {
            return ['id' => -1];
        }

        $id = ++$this->lastId;
        $this->games[$id] = ['id' => $id, 'name' => $name, 'votes' => 0];

        return ['id' => $id];
    }

    /**
     * @return array{success: bool}
     */
    private function remove(int $id): array
    {
        if (! isset($this->games[$id])) {
            return ['success' => false];
        }

        unset($this->games[$id]);

        return ['success' => true];
    }

    /**
     * @return array{success: bool}
     */
    private function adjustVotes(int $id, int $by): array
    {
        if (! isset($this->games[$id])) {
            return ['success' => false];
        }

        $this->games[$id]['votes'] = max(0, $this->games[$id]['votes'] + $by);

        return ['success' => true];
    }
}
```

`tests/Feature/Auth/AuthenticationTest.php` (edit)

Replace the whole file with:

```
<?php

namespace Tests\Feature\Auth;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\RateLimiter;
use PHPUnit\Framework\Attributes\DataProvider;
use Tests\TestCase;
```

```
class AuthenticationTest extends TestCase
{
    use RefreshDatabase;

    public function test_login_screen_can_be_rendered()
    {
        $response = $this->get(route('login'));

        $response->assertOk();
    }

    public function test_users_can_authenticate_using_the_login_screen()
    {
        $user = User::factory()->create();

        $response = $this->post(route('login.store'), [
            'username' => $user->username,
            'password' => 'password',
        ]);

        $this->assertAuthenticated();
        $response->assertRedirect(route('dashboard', absolute: false));
    }

    public function test_usernames_are_case_insensitive_when_logging_in()
    {
        User::factory()->create(['username' => 'test_user']);

        $this->post(route('login.store'), [
            'username' => 'Test_User',
            'password' => 'password',
        ]);

        $this->assertAuthenticated();
    }

    public function test_users_can_not_authenticate_with_an_email_field()
    {
        $user = User::factory()->create();

        $response = $this->post(route('login.store'), [
            'email' => $user->username,
            'password' => 'password',
        ]);

        $response->assertSessionHasErrors(['username' => 'The username field is required.']);
        $this->assertGuest();
    }

    /**
     * @return array<string, array{string, string}>
     */
    public static function removedEmailRoutes(): array
    {
        return [
            'forgot password form' => ['get', '/forgot-password'],
            'send password reset link' => ['post', '/forgot-password'],
            'reset password' => ['post', '/reset-password'],
            'email verification notice' => ['get', '/email/verify'],
            'resend verification email' => ['post', '/email/verification-notification'],
        ];
    }

    #[DataProvider('removedEmailRoutes')]
    public function test_email_based_routes_are_not_available(string $method, string $uri)
    {
        $user = User::factory()->create();

        $response = $this->actingAs($user)->{$method}($uri);
```

```
        $response->assertNotFound();
    }

    public function test_users_can_not_authenticate_with_invalid_password()
    {
        $user = User::factory()->create();

        $this->post(route('login.store'), [
            'username' => $user->username,
            'password' => 'wrong-password',
        ]);

        $this->assertGuest();
    }

    public function test_users_can_logout()
    {
        $user = User::factory()->create();

        $response = $this->actingAs($user)->post(route('logout'));

        $response->assertRedirect(route('home'));

        $this->assertGuest();
    }

    public function test_users_are_rate_limited()
    {
        $user = User::factory()->create();

        RateLimiter::increment(md5('login'.implode('|', [$user->username, '127.0.0.1'])), amount: 5);

        $response = $this->post(route('login.store'), [
            'username' => $user->username,
            'password' => 'wrong-password',
        ]);

        $response->assertTooManyRequests();
    }
}
```

tests/Feature/Auth/RegistrationTest.php (edit)

Replace the whole file with:

```
<?php

namespace Tests\Feature\Auth;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Laravel\Fortify\Features;
use PHPUnit\Framework\Attributes\DataProvider;
use Tests\TestCase;

class RegistrationTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();

        $this->skipUnlessFortifyHas(Features::registration());
    }

    public function test_registration_screen_can_be_rendered()
    {
```

```
        $response = $this->get(route('register'));

        $response->assertOk();
    }

    public function test_new_users_can_register()
    {
        $response = $this->post(route('register.store'), [
            'name' => 'Test User',
            'username' => 'test_user',
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);

        $this->assertAuthenticated();
        $response->assertRedirect(route('dashboard', absolute: false));
        $this->assertDatabaseHas('users', [
            'name' => 'Test User',
            'username' => 'test_user',
        ]);
    }

    public function test_the_browser_timezone_is_stored_at_registration()
    {
        $this->post(route('register.store'), [
            'name' => 'Test User',
            'username' => 'test_user',
            'timezone' => 'Australia/Sydney',
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);

       $this->assertDatabaseHas('users', ['username' => 'test_user', 'timezone' => 'Australia/Sydney']);
    }

    public function test_the_timezone_defaults_to_the_app_timezone_when_not_sent()
    {
        $this->post(route('register.store'), [
            'name' => 'Test User',
            'username' => 'test_user',
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);

        $this->assertDatabaseHas('users', ['username' => 'test_user', 'timezone' => 'UTC']);
    }

    public function test_an_invalid_timezone_is_rejected()
    {
        $response = $this->post(route('register.store'), [
            'name' => 'Test User',
            'username' => 'test_user',
            'timezone' => 'Mars/Olympus_Mons',
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);

        $response->assertSessionHasErrors(['timezone' => 'The timezone field must be a valid
        ↪  timezone.']);
        $this->assertGuest();
    }

    public function test_usernames_are_stored_in_lowercase()
    {
        $this->post(route('register.store'), [
            'name' => 'Test User',
            'username' => 'Test_User',
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);
```

```
        $this->assertDatabaseHas('users', ['username' => 'test_user']);
    }

    /**
     * @return array<string, array{string, string}>
     */
    public static function invalidUsernames(): array
    {
        return [
            'missing' => ['', 'The username field is required.'],
            'too short' => ['ab', 'The username field must be at least 3 characters.'],
            'too long' => [str_repeat('a', 31), 'The username field must not be greater than 30
            ↪  characters.'],
            'contains a space' => ['test user', 'The username field must only contain letters, numbers,
            ↪  dashes, and underscores.'],
            'contains an at sign' => ['test@example.com', 'The username field must only contain letters,
            ↪  numbers, dashes, and underscores.'],
        ];
    }

    #[DataProvider('invalidUsernames')]
    public function test_registration_requires_a_valid_username(string $username, string $message)
    {
        $response = $this->post(route('register.store'), [
            'name' => 'Test User',
            'username' => $username,
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);

        $response->assertSessionHasErrors(['username' => $message]);
        $this->assertGuest();
    }

    public function test_usernames_must_be_unique_regardless_of_case()
    {
        User::factory()->create(['username' => 'test_user']);

        $response = $this->post(route('register.store'), [
            'name' => 'Test User',
            'username' => 'TEST_USER',
            'password' => 'password',
            'password_confirmation' => 'password',
        ]);

        $response->assertSessionHasErrors(['username' => 'The username has already been taken.']);
        $this->assertGuest();
    }
}
```

`tests/Feature/DashboardTest.php` (edit)

Apply these changes (- remove, + add, other lines are context):

```
@@ -5,4 +5,5 @@
 use App\Models\User;
 use Illuminate\Foundation\Testing\RefreshDatabase;
+use Tests\FakeEssenseApi;
 use Tests\TestCase;

@@ -19,4 +20,5 @@
     public function test_authenticated_users_can_visit_the_dashboard()
     {
+        FakeEssenseApi::fake();
         $user = User::factory()->create();
         $this->actingAs($user);
```

tests/Feature/Games/AddGameTest.php (new file)

Create it with this content:

```php
<?php

use App\Enums\DailyActionType;
use App\Models\DailyAction;
use App\Models\User;
use Tests\FakeEssenseApi;

test('a user can add a game, which spends their daily allowance', function () {
    $api = FakeEssenseApi::fake();
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post(route('games.store'), ['name' => '  The   Legend of Zelda '])
        ->assertSessionHasNoErrors()
        ->assertRedirect(route('dashboard'))
        ->assertInertiaFlash('toast.message', 'Added "The Legend of Zelda" to the library.');

    $gameId = $api->idOf('The Legend of Zelda');
    expect($user->dailyActions()->sole())
        ->type->toBe(DailyActionType::AddGame)
        ->game_id->toBe($gameId);
});

test('duplicate titles are rejected regardless of case and spacing', function () {
    $api = FakeEssenseApi::fake([['name' => 'The Legend of Zelda']]);
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post(route('games.store'), ['name' => 'the  LEGEND of zelda'])
        ->assertSessionHasErrors(['name' => '"The Legend of Zelda" is already in the library.']);

    expect($api->count())->toBe(1)
        ->and($user->dailyActions()->exists())->toBeFalse();
});

test('a user who already voted today cannot also add a game', function () {
    $api = FakeEssenseApi::fake();
    $user = User::factory()->create();
    DailyAction::factory()->for($user)->create();

    $this->actingAs($user)
        ->post(route('games.store'), ['name' => 'Tetris'])
        ->assertSessionHasErrors(['name' => 'You have already used today\'s vote or new game. You can
↪  take another action after midnight.']);

    expect($api->count())->toBe(0);
});

test('the title is validated', function (mixed $name, string $message) {
    FakeEssenseApi::fake();

    $this->actingAs(User::factory()->create())
        ->post(route('games.store'), ['name' => $name])
        ->assertSessionHasErrors(['name' => $message]);
})->with([
    'missing' => ['', 'The game title field is required.'],
    'only whitespace' => ['    ', 'The game title field is required.'],
    'too long' => [str_repeat('a', 101), 'The game title field must not be greater than 100
    ↪  characters.'],
]);

test('the allowance is not spent when the api fails to add the game', function () {
    FakeEssenseApi::unreachable();
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post(route('games.store'), ['name' => 'Tetris'])
```

```
        ->assertSessionHasErrors(['name' => 'The game library is not responding right now. Please try
↪  again in a moment.']);

    expect($user->dailyActions()->exists())->toBeFalse();
});

test('the allowance resets at midnight in the user\'s own timezone', function () {
    $api = FakeEssenseApi::fake();
    $user = User::factory()->create(['timezone' => 'Australia/Sydney']);

    // 23:30 on 20 Oct in Sydney (UTC+11).
    $this->travelTo('2026-10-20 12:30:00');
    $this->actingAs($user)->post(route('games.store'), ['name' =>
    ↪  'Tetris'])->assertSessionHasNoErrors();

    // 23:59 the same Sydney day: still used up.
    $this->travelTo('2026-10-20 12:59:00');
    $this->post(route('games.store'), ['name' => 'Chess'])->assertSessionHasErrors('name');

    // 00:01 on 21 Oct in Sydney, though it is still 20 Oct in UTC.
    $this->travelTo('2026-10-20 13:01:00');
    $this->post(route('games.store'), ['name' => 'Chess'])->assertSessionHasNoErrors();

    expect($api->count())->toBe(2);
});

test('guests cannot add games', function () {
    $api = FakeEssenseApi::fake();

    $this->post(route('games.store'), ['name' => 'Tetris'])->assertRedirect(route('login'));

    expect($api->count())->toBe(0);
});
```

`tests/Feature/Games/ApiKeyTest.php` (new file)

Create it with this content:

```
<?php

use App\Models\ApiCredential;
use App\Models\DailyAction;
use App\Models\User;
use App\Models\Vote;
use App\Services\Essense\ApiKeyProvider;
use App\Services\Essense\EssenseClient;
use Illuminate\Http\Client\Request;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Http;
use Inertia\Testing\AssertableInertia as Assert;
use Tests\FakeEssenseApi;

/**
 * Run without API_KEY_ESSENCE, so the app manages its own key.
 */
function withoutConfiguredKey(): void
{
    config(['services.essense.key' => null]);
}

/**
 * Drop the resolved key, as a new PHP request would.
 */
function startNewRequest(): void
{
    app()->forgetInstance(ApiKeyProvider::class);
    app()->forgetInstance(EssenseClient::class);
}
```

```
test('without API_KEY_ESSENCE the app generates a key and stores it encrypted', function () {
    withoutConfiguredKey();
    $api = FakeEssenseApi::fake([['name' => 'Zelda']]);

    $this->actingAs(User::factory()->create())
        ->get(route('dashboard'))
        ->assertInertia(fn (Assert $page) => $page->where('games.0.name', 'Zelda'));

    [$key] = $api->issuedKeys();
    expect($api->issuedKeys())->toHaveCount(1)
        ->and(ApiCredential::current()->generated_key)->toBe($key)
        ->and(DB::table('api_credentials')->value('generated_key'))->not->toContain($key);
    Http::assertSent(fn (Request $request) => str_ends_with($request->url(), 'games/list') &&
     ↪  $request['api_key'] === $key);
});

test('the stored key is reused by later requests', function () {
    withoutConfiguredKey();
    $api = FakeEssenseApi::fake();
    $user = User::factory()->create();

    $this->actingAs($user)->get(route('dashboard'));
    startNewRequest();
    $this->get(route('dashboard'))->assertInertia(fn (Assert $page) => $page->where('libraryError',
     ↪  null));

    expect($api->issuedKeys())->toHaveCount(1);
});

test('API_KEY_ESSENCE takes precedence and no key is generated', function () {
    $api = FakeEssenseApi::fake();

    $this->actingAs(User::factory()->create())->get(route('dashboard'))->assertOk();

    expect($api->issuedKeys())->toBeEmpty()
        ->and(ApiCredential::current()->generated_key)->toBeNull();
});

test('a rejected key from API_KEY_ESSENCE is not replaced', function () {
    config(['services.essense.key' => 'revoked-key']);
    $api = FakeEssenseApi::fake();

    $this->actingAs(User::factory()->create())
        ->get(route('dashboard'))
        ->assertInertia(fn (Assert $page) => $page
            ->where('libraryError', 'The game library rejected the API key in API_KEY_ESSENCE. Update it
 ↪  in .env, or remove it to let the app get its own key, then restart the app.'));

    expect($api->issuedKeys())->toBeEmpty();
});

test('a generated key that stops working is replaced and the request retried', function () {
    withoutConfiguredKey();
    $api = FakeEssenseApi::fake();
    $user = User::factory()->create();
    $this->actingAs($user)->get(route('dashboard'));

    $api->restart([['name' => 'Chess']]);
    startNewRequest();

    $this->get(route('dashboard'))
        ->assertInertia(fn (Assert $page) => $page
            ->where('libraryError', null)
            ->where('games.0.name', 'Chess'));

    expect($api->issuedKeys())->toHaveCount(2)
        ->and(ApiCredential::current()->generated_key)->toBe($api->issuedKeys()[1]);
});

test('a new library discards local votes and refunds today\'s actions on them', function () {
```

```
    withoutConfiguredKey();
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
    $user = User::factory()->create();
    $this->actingAs($user)->get(route('dashboard'));

    Vote::factory()->for($user)->create(['game_id' => 1]);
    $today = DailyAction::factory()->for($user)->create(['game_id' => 1]);
    $yesterday = DailyAction::factory()->for($user)->create(['action_date' =>
    ↪  now()->subDay()->toDateString(), 'game_id' => 1]);

    // Game id 1 in the new library is a different game, so the old vote must not carry over.
    $api->restart([['name' => 'Chess']]);
    startNewRequest();

    $this->get(route('dashboard'))
        ->assertInertia(fn (Assert $page) => $page
            ->where('games.0.name', 'Chess')
            ->where('games.0.voted', false)
            ->where('allowance.available', true));

    expect(Vote::query()->exists())->toBeFalse()
        ->and($today->fresh())->toBeNull()
        ->and($yesterday->fresh())->not->toBeNull();
});

test('local votes are kept while the key stays the same', function () {
    withoutConfiguredKey();
    FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
    $user = User::factory()->create();
    $this->actingAs($user)->get(route('dashboard'));
    Vote::factory()->for($user)->create(['game_id' => 1]);

    startNewRequest();
    $this->get(route('dashboard'))->assertInertia(fn (Assert $page) => $page->where('games.0.voted',
    ↪  true));

    expect(Vote::query()->count())->toBe(1);
});

test('switching from API_KEY_ESSENCE to a generated key resets local votes', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
    $user = User::factory()->create();
    $this->actingAs($user)->get(route('dashboard'));
    Vote::factory()->for($user)->create(['game_id' => 1]);

    withoutConfiguredKey();
    startNewRequest();
    $this->get(route('dashboard'))->assertOk();

    expect($api->issuedKeys())->toHaveCount(1)
        ->and(Vote::query()->exists())->toBeFalse();
});

test('the page explains when no key can be generated', function () {
    withoutConfiguredKey();
    Http::fake(['essense.test/api-key/generate' => Http::response('', 500)]);

    $this->actingAs(User::factory()->create())
        ->get(route('dashboard'))
        ->assertOk()
        ->assertInertia(fn (Assert $page) => $page
            ->where('games', null)
            ->where('libraryError', 'The game library is not responding right now. Please try again in a
↪   moment.'));
});
```

`tests/Feature/Games/EssenseClientTest.php` (new file)

Create it with this content:

```php
<?php

use App\Services\Essense\EssenseApiException;
use App\Services\Essense\EssenseClient;
use Illuminate\Http\Client\Request;
use Illuminate\Support\Facades\Http;
use Tests\FakeEssenseApi;

test('requests are form encoded and include the api key', function () {
    FakeEssenseApi::fake([['name' => 'Zelda']]);

    app(EssenseClient::class)->vote(1);

    Http::assertSent(fn (Request $request) => $request->url() === 'https://essense.test/games/vote'
        && $request->method() === 'POST'
        && $request->isForm()
        && $request['api_key'] === FakeEssenseApi::KEY
        && (int) $request['id'] === 1);
});

test('games are listed from the library', function () {
    FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 3], ['name' => 'Tetris']]);

    $games = app(EssenseClient::class)->games();

    expect($games)->toHaveCount(2)
        ->and($games[0]->name)->toBe('Zelda')
        ->and($games[0]->votes)->toBe(3);
});

test('a malformed game in the list is reported as an api failure', function () {
    Http::fake(['essense.test/*' => Http::response(['games' => [['id' => 1, 'name' => null, 'votes' =>
    ↪  0]]])]);

    app(EssenseClient::class)->games();
})->throws(EssenseApiException::class, 'malformed game');

test('finding an unknown game returns null', function () {
    FakeEssenseApi::fake();

    expect(app(EssenseClient::class)->find(999))->toBeNull();
});

test('a rejected add throws instead of returning the sentinel id', function () {
    Http::fake(['essense.test/*' => Http::response(['id' => -1])]);

    app(EssenseClient::class)->add('Zelda');
})->throws(EssenseApiException::class, 'could not add that game');

test('voting for an unknown game reports failure', function () {
    FakeEssenseApi::fake();

    expect(app(EssenseClient::class)->vote(999))->toBeFalse();
});

test('api error codes are surfaced on the exception', function () {
    Http::fake(['essense.test/*' => Http::response(['error' => 'INVALID_KEY'], 403)]);

    try {
        app(EssenseClient::class)->games();
        $this->fail('Expected an EssenseApiException.');
    } catch (EssenseApiException $exception) {
        expect($exception->errorCode)->toBe('INVALID_KEY');
    }
});

test('reads are retried once when the api is unreachable', function () {
    FakeEssenseApi::unreachable();

    expect(fn () => app(EssenseClient::class)->games())->toThrow(EssenseApiException::class, 'could not
↪  be reached');
```

```
    Http::assertSentCount(2);
});

test('votes are not retried so a vote is never counted twice', function () {
    FakeEssenseApi::unreachable();

    expect(fn () => app(EssenseClient::class)->vote(1))->toThrow(EssenseApiException::class);

    Http::assertSentCount(1);
});
```

tests/Feature/Games/GameLibraryTest.php (new file)

Create it with this content:

```
<?php

use App\Models\DailyAction;
use App\Models\User;
use App\Models\Vote;
use Illuminate\Support\Facades\Http;
use Inertia\Testing\AssertableInertia as Assert;
use Tests\FakeEssenseApi;

test('games are listed most voted first with ties ordered by name', function () {
    FakeEssenseApi::fake([
        ['name' => 'Tetris', 'votes' => 1],
        ['name' => 'zelda', 'votes' => 4],
        ['name' => 'Asteroids', 'votes' => 1],
        ['name' => 'Chess', 'votes' => 0],
    ]);

    $this->actingAs(User::factory()->create())
        ->get(route('dashboard'))
        ->assertOk()
        ->assertInertia(fn (Assert $page) => $page
            ->component('dashboard')
            ->where('games.0.name', 'zelda')
            ->where('games.1.name', 'Asteroids')
            ->where('games.2.name', 'Tetris')
            ->where('games.3.name', 'Chess')
            ->where('libraryError', null));
});

test('games the user voted for are marked', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1], ['name' => 'Tetris']]);
    $user = User::factory()->create();
    Vote::factory()->for($user)->create(['game_id' => $api->idOf('Zelda')]);

    $this->actingAs($user)
        ->get(route('dashboard'))
        ->assertInertia(fn (Assert $page) => $page
            ->where('games.0.name', 'Zelda')
            ->where('games.0.voted', true)
            ->where('games.1.voted', false));
});

test('the page still loads with a friendly error when the api is unreachable', function () {
    FakeEssenseApi::unreachable();

    $this->actingAs(User::factory()->create())
        ->get(route('dashboard'))
        ->assertOk()
        ->assertInertia(fn (Assert $page) => $page
            ->where('games', null)
            ->where('libraryError', 'The game library is not responding right now. Please try again in a
↪  moment.'));
});
```

```
test('a rejected api key from .env is explained instead of reported as an outage', function () {
    Http::fake(['essense.test/*' => Http::response(['error' => 'INVALID_KEY'], 403)]);

    $this->actingAs(User::factory()->create())
        ->get(route('dashboard'))
        ->assertOk()
        ->assertInertia(fn (Assert $page) => $page
            ->where('libraryError', 'The game library rejected the API key in API_KEY_ESSENCE. Update it
↪  in .env, or remove it to let the app get its own key, then restart the app.'));
});

test('the allowance shows what the user spent today and when it resets', function () {
    $this->travelTo('2026-10-20 15:00:00');
    FakeEssenseApi::fake();
    $user = User::factory()->create(['timezone' => 'Australia/Sydney']);
    DailyAction::factory()->for($user)->create(['action_date' => '2026-10-21', 'game_id' => 7]);

    $this->actingAs($user)
        ->get(route('dashboard'))
        ->assertInertia(fn (Assert $page) => $page
            ->where('allowance.available', false)
            ->where('allowance.used_for', 'vote')
            ->where('allowance.game_id', 7)
            ->where('allowance.resets_at', '2026-10-22T00:00:00+11:00'));
});
```

`tests/Feature/Games/RemoveGameTest.php` (new file)

Create it with this content:

```
<?php

use App\Enums\DailyActionType;
use App\Models\DailyAction;
use App\Models\User;
use App\Models\Vote;
use Tests\FakeEssenseApi;

test('any user can remove a game, which discards its votes', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 2], ['name' => 'Tetris', 'votes' => 1]]);
    Vote::factory()->count(2)->create(['game_id' => 1]);
    Vote::factory()->create(['game_id' => 2]);

    $this->actingAs(User::factory()->create())
        ->delete(route('games.destroy', 1))
        ->assertRedirect(route('dashboard'))
        ->assertInertiaFlash('toast.message', 'Game removed from the library.');

    expect($api->game(1))->toBeNull()
        ->and(Vote::query()->pluck('game_id')->all())->toBe([2]);
});

test('removing a game refunds anyone who spent today\'s allowance on it', function () {
    FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
    $adder = User::factory()->create();
    $voter = User::factory()->create();
    DailyAction::factory()->for($adder)->addedGame()->create(['game_id' => 1]);
    DailyAction::factory()->for($voter)->create(['type' => DailyActionType::Vote, 'game_id' => 1]);

    $this->actingAs(User::factory()->create())->delete(route('games.destroy', 1));

    expect(DailyAction::query()->exists())->toBeFalse();
});

test('removing a game does not refund actions from earlier days', function () {
    FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
    DailyAction::factory()->create(['action_date' => now()->subDay()->toDateString(), 'game_id' => 1]);
```

```
    $this->actingAs(User::factory()->create())->delete(route('games.destroy', 1));

    expect(DailyAction::query()->count())->toBe(1);
});

test('removing a game that is already gone tells the user', function () {
    FakeEssenseApi::fake();

    $this->actingAs(User::factory()->create())
        ->delete(route('games.destroy', 99))
        ->assertInertiaFlash('toast.message', 'That game is no longer in the library. The list has been
↪   refreshed.');
});

test('votes are kept when the api fails to remove the game', function () {
    FakeEssenseApi::unreachable();
    Vote::factory()->create(['game_id' => 1]);

    $this->actingAs(User::factory()->create())
        ->delete(route('games.destroy', 1))
        ->assertInertiaFlash('toast.message', 'The game library is not responding right now. Please try
↪   again in a moment.');

    expect(Vote::query()->count())->toBe(1);
});
```

tests/Feature/Games/RemoveLimitsTest.php (new file)

Create it with this content:

```
<?php

use App\Models\User;
use App\Models\Vote;
use Illuminate\Foundation\Http\Middleware\PreventRequestForgery;
use Inertia\Testing\AssertableInertia as Assert;
use Tests\FakeEssenseApi;

beforeEach(function () {
    config(['games.remove_limits' => true]);
});

test('with limits removed a user can add games and vote as often as they like', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda'], ['name' => 'Tetris']]);
    $user = User::factory()->create();

    $this->actingAs($user)->post(route('games.store'), ['name' => 'Chess'])->assertSessionHasNoErrors();
    $this->post(route('games.store'), ['name' => 'Go'])->assertSessionHasNoErrors();
    $this->post(route('games.vote.store', 1))->assertInertiaFlash('toast.type', 'success');
    $this->post(route('games.vote.store', 2))->assertInertiaFlash('toast.type', 'success');

    expect($api->count())->toBe(4)
        ->and($api->game(1)['votes'])->toBe(1)
        ->and($api->game(2)['votes'])->toBe(1)
        ->and($user->dailyActions()->exists())->toBeFalse();
});

test('duplicate titles are still rejected with limits removed', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda']]);

    $this->actingAs(User::factory()->create())
        ->post(route('games.store'), ['name' => 'ZELDA'])
        ->assertSessionHasErrors(['name' => '"Zelda" is already in the library.']);

    expect($api->count())->toBe(1);
});

test('a user still gets one vote per game with limits removed', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
```

```
    $user = User::factory()->create();
    Vote::factory()->for($user)->create(['game_id' => 1]);

    $this->actingAs($user)
        ->post(route('games.vote.store', 1))
        ->assertInertiaFlash('toast.message', 'You have already voted for this game.');

    expect($api->game(1)['votes'])->toBe(1);
});

test('the page shows that limits are off', function () {
    FakeEssenseApi::fake();

    $this->actingAs(User::factory()->create())
        ->get(route('dashboard'))
        ->assertInertia(fn (Assert $page) => $page
            ->where('allowance.enforced', false)
            ->where('allowance.available', true));
});

test('the setting is ignored in production', function () {
    app()->instance('env', 'production');
    // CSRF checks are only skipped in the testing environment.
    $this->withoutMiddleware(PreventRequestForgery::class);
    $api = FakeEssenseApi::fake([['name' => 'Zelda'], ['name' => 'Tetris']]);
    $user = User::factory()->create();

    $this->actingAs($user)->post(route('games.vote.store', 1));
    $this->post(route('games.vote.store', 2))
        ->assertInertiaFlash('toast.message', 'You have already used today\'s vote or new game. You can
↪  take another action after midnight.');

    expect($api->game(2)['votes'])->toBe(0);
});
```

`tests/Feature/Games/SeedGamesCommandTest.php` (new file)

Create it with this content:

```
<?php

use App\Console\Commands\SeedGames;
use Tests\FakeEssenseApi;

test('it adds ten games with random vote counts by default', function () {
    $api = FakeEssenseApi::fake();

    $this->artisan('games:seed')
        ->expectsOutputToContain('Added 10 games to the library.')
        ->assertSuccessful();

    expect($api->count())->toBe(10);

    foreach (range(1, 10) as $id) {
        expect($api->game($id)['name'])->toBeIn(SeedGames::TITLES)
            ->and($api->game($id)['votes'])->toBeGreaterThanOrEqual(0)->toBeLessThanOrEqual(15);
    }
});

test('the number of games and the vote ceiling can be chosen', function () {
    $api = FakeEssenseApi::fake();

    $this->artisan('games:seed', ['--count' => 3, '--max-votes' => 0])->assertSuccessful();

    expect($api->count())->toBe(3)
        ->and(array_sum(array_map(fn (int $id) => $api->game($id)['votes'], [1, 2, 3])))->toBe(0);
});

test('titles already in the library are not added again', function () {
```

```
    $titles = array_slice(SeedGames::TITLES, 0, count(SeedGames::TITLES) - 2);
    $api = FakeEssenseApi::fake(array_map(fn (string $title) => ['name' => strtoupper($title)],
    ↪  $titles));

    $this->artisan('games:seed', ['--count' => 5])
        ->expectsOutputToContain('Only 2 new titles were available')
        ->assertSuccessful();

    expect($api->count())->toBe(count(SeedGames::TITLES));
});

test('invalid options are rejected without touching the library', function (array $options) {
    $api = FakeEssenseApi::fake();

    $this->artisan('games:seed', $options)->assertExitCode(SeedGames::INVALID);

    expect($api->count())->toBe(0);
})->with([
    'zero games' => [['--count' => 0]],
    'more games than titles' => [['--count' => count(SeedGames::TITLES) + 1]],
    'not a number' => [['--count' => 'lots']],
    'negative votes' => [['--max-votes' => -1]],
]);

test('it reports a friendly error when the api is unreachable', function () {
    FakeEssenseApi::unreachable();

    $this->artisan('games:seed')
        ->expectsOutputToContain('Seeding stopped: The game library could not be reached.')
        ->assertFailed();
});

test('in production it does nothing unless forced', function () {
    $api = FakeEssenseApi::fake();
    app()->instance('env', 'production');

    $this->artisan('games:seed', ['--no-interaction' => true])
        ->expectsOutputToContain('Command cancelled.')
        ->assertFailed();

    expect($api->count())->toBe(0);

    $this->artisan('games:seed', ['--force' => true])->assertSuccessful();

    expect($api->count())->toBe(10);
});
```

tests/Feature/Games/VoteTest.php (new file)

Create it with this content:

```
<?php

use App\Enums\DailyActionType;
use App\Models\DailyAction;
use App\Models\User;
use App\Models\Vote;
use Tests\FakeEssenseApi;

test('a user can vote for a game, which spends their daily allowance', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 2]]);
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post(route('games.vote.store', 1))
        ->assertRedirect(route('dashboard'))
        ->assertInertiaFlash('toast.message', 'Your vote was counted.');

    expect($api->game(1)['votes'])->toBe(3)
```

```
        ->and($user->votes()->sole()->game_id)->toBe(1)
        ->and($user->dailyActions()->sole()->type)->toBe(DailyActionType::Vote);
});

test('a user can only vote once per day', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda'], ['name' => 'Tetris']]);
    $user = User::factory()->create();

    $this->actingAs($user)->post(route('games.vote.store', 1));
    $this->post(route('games.vote.store', 2))
        ->assertInertiaFlash('toast.message', 'You have already used today\'s vote or new game. You can
↪  take another action after midnight.');

    expect($api->game(2)['votes'])->toBe(0);
});

test('a user can vote again the next day', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda'], ['name' => 'Tetris']]);
    $user = User::factory()->create();

    $this->travelTo('2026-10-20 09:00:00');
    $this->actingAs($user)->post(route('games.vote.store', 1));

    $this->travelTo('2026-10-21 09:00:00');
    $this->post(route('games.vote.store', 2))->assertInertiaFlash('toast.type', 'success');

    expect($api->game(2)['votes'])->toBe(1);
});

test('a user cannot vote for the same game twice, even on another day', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
    $user = User::factory()->create();
    Vote::factory()->for($user)->create(['game_id' => 1]);

    $this->actingAs($user)
        ->post(route('games.vote.store', 1))
        ->assertInertiaFlash('toast.message', 'You have already voted for this game.');

    expect($api->game(1)['votes'])->toBe(1)
        ->and($user->dailyActions()->exists())->toBeFalse();
});

test('voting for a game that no longer exists does not spend the allowance', function () {
    FakeEssenseApi::fake();
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post(route('games.vote.store', 99))
        ->assertInertiaFlash('toast.message', 'That game is no longer in the library. The list has been
↪  refreshed.');

    expect($user->votes()->exists())->toBeFalse()
        ->and($user->dailyActions()->exists())->toBeFalse();
});

test('a failed vote is rolled back so the allowance is not spent', function () {
    FakeEssenseApi::unreachable();
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post(route('games.vote.store', 1))
        ->assertInertiaFlash('toast.message', 'The game library is not responding right now. Please try
↪  again in a moment.');

    expect($user->votes()->exists())->toBeFalse()
        ->and($user->dailyActions()->exists())->toBeFalse();
});

test('removing a vote cast today gives the allowance back', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda'], ['name' => 'Tetris']]);
```

```
    $user = User::factory()->create();
    $this->actingAs($user)->post(route('games.vote.store', 1));

    $this->delete(route('games.vote.destroy', 1))
        ->assertInertiaFlash('toast.message', 'Your vote was removed. You can vote or add a game again
↪  today.');

    expect($api->game(1)['votes'])->toBe(0)
        ->and($user->votes()->exists())->toBeFalse();

    $this->post(route('games.vote.store', 2))->assertInertiaFlash('toast.type', 'success');
});

test('removing a vote from an earlier day does not refund today\'s action', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1], ['name' => 'Tetris']]);
    $user = User::factory()->create();
    Vote::factory()->for($user)->create(['game_id' => 1]);
    DailyAction::factory()->for($user)->create(['action_date' => now()->subDay()->toDateString(),
↪  'game_id' => 1]);
    DailyAction::factory()->for($user)->addedGame()->create(['game_id' => 2]);

    $this->actingAs($user)
        ->delete(route('games.vote.destroy', 1))
        ->assertInertiaFlash('toast.message', 'Your vote was removed.');

    expect($api->game(1)['votes'])->toBe(0)
        ->and($user->dailyActions()->count())->toBe(2);
});

test('a user cannot remove a vote they did not cast', function () {
    $api = FakeEssenseApi::fake([['name' => 'Zelda', 'votes' => 1]]);
    Vote::factory()->create(['game_id' => 1]);

    $this->actingAs(User::factory()->create())
        ->delete(route('games.vote.destroy', 1))
        ->assertInertiaFlash('toast.message', 'You have not voted for this game.');

    expect($api->game(1)['votes'])->toBe(1)
        ->and(Vote::query()->count())->toBe(1);
});

test('a vote is kept when the api fails to remove it', function () {
    FakeEssenseApi::unreachable();
    $user = User::factory()->create();
    Vote::factory()->for($user)->create(['game_id' => 1]);

    $this->actingAs($user)
        ->delete(route('games.vote.destroy', 1))
        ->assertInertiaFlash('toast.message', 'The game library is not responding right now. Please try
↪  again in a moment.');

    expect($user->votes()->exists())->toBeTrue();
});
```

tests/Feature/SecurityHardeningTest.php (new file)

Create it with this content:

```
<?php

use App\Models\User;

test('one ip address is limited across many usernames', function () {
    User::factory()->create(['username' => 'real_user']);

    foreach (range(1, 20) as $attempt) {
        $this->post(route('login.store'), ['username' => "guess_{$attempt}", 'password' =>
↪  'wrong-password']);
    }
```

```
    $this->post(route('login.store'), ['username' => 'real_user', 'password' => 'password'])
        ->assertTooManyRequests();

    $this->assertGuest();
});

test('a different ip address is not affected by another ip being limited', function () {
    foreach (range(1, 20) as $attempt) {
        $this->post(route('login.store'), ['username' => "guess_{$attempt}", 'password' =>
        ↪  'wrong-password']);
    }

    User::factory()->create(['username' => 'real_user']);

    $this->withServerVariables(['REMOTE_ADDR' => '198.51.100.9'])
        ->post(route('login.store'), ['username' => 'real_user', 'password' => 'password']);

    $this->assertAuthenticated();
});

test('an unexpected appearance cookie falls back to the system theme', function () {
    $response = $this->withUnencryptedCookie('appearance', "x';alert(1)//\\")
        ->get(route('login'));

    $response->assertOk();
    expect($response->getContent())
        ->toContain("const appearance = 'system';")
        ->not->toContain('alert(1)');
});

test('a known appearance cookie is used', function () {
    $this->withUnencryptedCookie('appearance', 'dark')
        ->get(route('login'))
        ->assertSee("const appearance = 'dark';", escape: false);
});

test('the content security policy allows only scripts carrying the request nonce', function () {
    config(['app.content_security_policy' => true]);

    $response = $this->get(route('login'));

    $policy = $response->headers->get('Content-Security-Policy');
    expect($policy)->toContain("default-src 'self'")
        ->toContain("object-src 'none'")
        ->toContain("frame-ancestors 'self'");

    preg_match("/script-src 'self' 'nonce-([^']+)'/", (string) $policy, $match);
    expect($match)->toHaveCount(2);

    preg_match_all('/<script\b[^>]*>/', $response->getContent(), $scripts);
    $executable = array_filter($scripts[0], fn (string $tag) => ! str_contains($tag,
    ↪  'application/json'));

    expect($executable)->not->toBeEmpty();
    foreach ($executable as $tag) {
        expect($tag)->toContain('nonce="'.$match[1].'"');
    }
});

test('no content security policy is sent when it is turned off', function () {
    config(['app.content_security_policy' => false]);

    $this->get(route('login'))->assertHeaderMissing('Content-Security-Policy');
});
```

`tests/Feature/Settings/ProfileUpdateTest.php` (edit)

Replace the whole file with:

```php
<?php

namespace Tests\Feature\Settings;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class ProfileUpdateTest extends TestCase
{
    use RefreshDatabase;

    public function test_profile_page_is_displayed()
    {
        $user = User::factory()->create();

        $response = $this
            ->actingAs($user)
            ->get(route('profile.edit'));

        $response->assertOk();
    }

    public function test_profile_information_can_be_updated()
    {
        $user = User::factory()->create();

        $response = $this
            ->actingAs($user)
            ->patch(route('profile.update'), [
                'name' => 'Test User',
                'username' => 'test_user',
            ]);

        $response
            ->assertSessionHasNoErrors()
            ->assertRedirect(route('profile.edit'));

        $user->refresh();

        $this->assertSame('Test User', $user->name);
        $this->assertSame('test_user', $user->username);
    }

    public function test_username_is_stored_in_lowercase_when_updated()
    {
        $user = User::factory()->create();

        $this
            ->actingAs($user)
            ->patch(route('profile.update'), [
                'name' => $user->name,
                'username' => 'Test_User',
            ]);

        $this->assertSame('test_user', $user->refresh()->username);
    }

    public function test_user_can_keep_their_current_username()
    {
        $user = User::factory()->create(['username' => 'test_user']);

        $response = $this
            ->actingAs($user)
            ->patch(route('profile.update'), [
                'name' => 'Test User',
                'username' => 'test_user',
            ]);

        $response->assertSessionHasNoErrors();
```

```
    }

    public function test_username_cannot_be_changed_to_another_users_username()
    {
        User::factory()->create(['username' => 'taken_name']);
        $user = User::factory()->create(['username' => 'test_user']);

        $response = $this
            ->actingAs($user)
            ->from(route('profile.edit'))
            ->patch(route('profile.update'), [
                'name' => $user->name,
                'username' => 'Taken_Name',
            ]);

        $response
            ->assertSessionHasErrors(['username' => 'The username has already been taken.'])
            ->assertRedirect(route('profile.edit'));

        $this->assertSame('test_user', $user->refresh()->username);
    }

    public function test_username_must_only_contain_letters_numbers_dashes_and_underscores()
    {
        $user = User::factory()->create(['username' => 'test_user']);

        $response = $this
            ->actingAs($user)
            ->patch(route('profile.update'), [
                'name' => $user->name,
                'username' => 'test user',
            ]);

        $response->assertSessionHasErrors(['username' => 'The username field must only contain letters,
        ↪  numbers, dashes, and underscores.']);
        $this->assertSame('test_user', $user->refresh()->username);
    }

    public function test_user_can_delete_their_account()
    {
        $user = User::factory()->create();

        $response = $this
            ->actingAs($user)
            ->delete(route('profile.destroy'), [
                'password' => 'password',
            ]);

        $response
            ->assertSessionHasNoErrors()
            ->assertRedirect(route('home'));

        $this->assertGuest();
        $this->assertNull($user->fresh());
    }

    public function test_correct_password_must_be_provided_to_delete_account()
    {
        $user = User::factory()->create();

        $response = $this
            ->actingAs($user)
            ->from(route('profile.edit'))
            ->delete(route('profile.destroy'), [
                'password' => 'wrong-password',
            ]);

        $response
            ->assertSessionHasErrors('password')
            ->assertRedirect(route('profile.edit'));
```

```
        $this->assertNotNull($user->fresh());
    }
}
```

tests/Feature/TrustedProxiesTest.php (new file)

Create it with this content:

```
<?php

use App\Providers\AppServiceProvider;
use Illuminate\Http\Middleware\TrustProxies;

/**
 * Re-run the provider so it reads the trusted proxies setting set by the test.
 */
function trustProxies(?string $setting): void
{
    config(['app.trusted_proxies' => $setting]);
    (new AppServiceProvider(app()))->boot();
}

afterEach(function () {
    TrustProxies::flushState();
});

$proxyHeaders = [
    'X-Forwarded-Proto' => 'https',
    'X-Forwarded-Host' => 'gamevote.example.com',
    'X-Forwarded-Port' => '443',
];

test('behind a trusted proxy, generated urls use the forwarded https host', function () use
↪  ($proxyHeaders) {
    trustProxies('*');

    $this->get(route('dashboard'), $proxyHeaders)
        ->assertRedirect('https://gamevote.example.com/login');
});

test('forwarded headers are ignored when no proxy is trusted', function () use ($proxyHeaders) {
    trustProxies(null);
    $untrusted = config('app.url').'/login';

    $this->get(route('dashboard'), $proxyHeaders)
        ->assertRedirect($untrusted);
});

test('a list of proxy addresses only trusts those addresses', function () use ($proxyHeaders) {
    trustProxies('10.0.0.1, 10.0.0.2');
    $untrusted = config('app.url').'/login';

    $this->get(route('dashboard'), $proxyHeaders)
        ->assertRedirect($untrusted);

    $this->withServerVariables(['REMOTE_ADDR' => '10.0.0.2'])
        ->get(route('dashboard'), $proxyHeaders)
        ->assertRedirect('https://gamevote.example.com/login');
});
```

tests/Pest.php (check)

`laravel new --pest` usually generates this file. Make sure it matches:

```
<?php
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

/*
|--------------------------------------------------------------------------
| Test Case
|--------------------------------------------------------------------------
|
| The closure you provide to your test functions is always bound to a specific PHPUnit test
| case class. By default, that class is "PHPUnit\Framework\TestCase". Of course, you may
| need to change it using the "pest()" function to bind different classes or traits.
|
*/

pest()->extend(TestCase::class)
    ->use(RefreshDatabase::class)
    ->in('Feature');

/*
|--------------------------------------------------------------------------
| Expectations
|--------------------------------------------------------------------------
|
| When you're writing tests, you often need to check that values meet certain conditions. The
| "expect()" function gives you access to a set of "expectations" methods that you can use
| to assert different things. Of course, you may extend the Expectation API at any time.
|
*/

expect()->extend('toBeOne', function () {
    return $this->toBe(1);
});

/*
|--------------------------------------------------------------------------
| Functions
|--------------------------------------------------------------------------
|
| While Pest is very powerful out-of-the-box, you may have some testing code specific to your
| project that you don't want to repeat in every file. Here you can also expose helpers as
| global functions to help you to reduce the number of lines of code in your test files.
|
*/

function something()
{
    // ..
}
```

`tests/TestCase.php` (edit)

Replace the whole file with:

```
<?php

namespace Tests;

use Illuminate\Foundation\Testing\TestCase as BaseTestCase;
use Illuminate\Support\Facades\Http;
use Laravel\Fortify\Features;

abstract class TestCase extends BaseTestCase
{
    protected function setUp(): void
    {
        parent::setUp();

        Http::preventStrayRequests();
    }
```

```
    protected function skipUnlessFortifyHas(string $feature, ?string $message = null): void
    {
        if (! Features::enabled($feature)) {
            $this->markTestSkipped($message ?? "Fortify feature [{$feature}] is not enabled.");
        }
    }
}
```

Docker code

None of these files has an artisan generator, so you write all of them by hand. The shell scripts need to be executable: `chmod +x docker/*.sh`.

`.dockerignore` (new file)

Create it with this content:

```
.git
.github
.env
.env.*
!.env.example
node_modules
vendor
public/build
public/hot
public/fonts-manifest.dev.json
bootstrap/ssr
bootstrap/cache/*.php
database/*.sqlite
storage/logs/*
storage/framework/cache/data/*
storage/framework/sessions/*
storage/framework/views/*
storage/pail
.phpunit.cache
resources/js/actions
resources/js/routes
resources/js/wayfinder
docker-compose.yml
Dockerfile
```

`.railway/railway.ts` (new file)

Create it with this content. Change `github('webanoid/gamevote', …)` if your repo lives elsewhere:

```
import {
    defineRailway,
    github,
    preserve,
    project,
    service,
    volume,
} from 'railway/iac';

/**
 * GameVote on Railway: one service running nginx and PHP-FPM together (the Dockerfile's
 * final `railway` stage), with SQLite and the generated APP_KEY on a persistent volume.
 * See docs/deployment.md#railway.
 *
 * Preview with `railway config plan`; apply with `railway config apply`.
 */
export default defineRailway(() => {
    // 512 MB: the most Railway's Free and Trial plans allow, and far more than SQLite needs here.
    const data = volume('gamevote-data', { sizeMB: 512 });

    const app = service('gamevote', {
        source: github('webanoid/gamevote', { branch: 'main' }),
        build: { builder: 'DOCKERFILE', dockerfilePath: 'Dockerfile' },

        // Laravel's health route; nginx exempts it from HTTP basic auth.
        healthcheck: '/up',
        healthcheckTimeout: 120,
```

```
        // SQLite lives on the volume, and a volume attaches to one service: one replica only.
        replicas: 1,
        deploy: {
            restartPolicyType: 'ON_FAILURE',
            restartPolicyMaxRetries: 5,
            requiredMountPath: '/data',
            // Sleep when idle (Railway "Serverless") so a demo stays within the free credit.
            // The first request after a sleep wakes it and can be slow or return a 502.
            sleepApplication: true,
        },
        volumeMounts: { '/data': data },

        env: {
            PORT: '8080',
            APP_NAME: 'GameVote',
            APP_ENV: 'production',
            APP_DEBUG: 'false',
            APP_URL: 'https://${{RAILWAY_PUBLIC_DOMAIN}}',
            DB_CONNECTION: 'sqlite',
            DB_DATABASE: '/data/database.sqlite',
            LOG_CHANNEL: 'stderr',
            // Railway terminates HTTPS at its proxy.
            TRUSTED_PROXIES: '*',
            SESSION_SECURE_COOKIE: 'true',

            // Optional, set in the Railway dashboard; preserve() leaves existing values alone.
            // Without API_KEY_ESSENCE the app generates its own key and stores it on the volume.
            API_KEY_ESSENCE: preserve(),
            HTTP_AUTH_USER: preserve(),
            HTTP_AUTH_PASSWORD: preserve(),
        },
    });

    return project('gamevote', { resources: [data, app] });
});
```

`Dockerfile` (new file)

Create it with this content:

```
# syntax=docker/dockerfile:1

ARG PHP_VERSION=8.5
ARG NODE_VERSION=22
ARG NGINX_VERSION=1.30

FROM node:${NODE_VERSION}-trixie-slim AS node

# ------------------------------------------------------------------------
# PHP 8.5 FPM with the extensions Laravel needs (SQLite and OPcache are built in)
# ------------------------------------------------------------------------
FROM php:${PHP_VERSION}-fpm AS php-base

RUN apt-get update \
    && apt-get install -y --no-install-recommends git unzip libicu-dev libzip-dev sqlite3 \
    && docker-php-ext-install -j"$(nproc)" intl zip pcntl \
    && rm -rf /var/lib/apt/lists/*

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html

# ------------------------------------------------------------------------
# PHP + Node toolchain (Wayfinder's Vite plugin shells out to artisan)
# ------------------------------------------------------------------------
FROM php-base AS toolchain

COPY --from=node /usr/local/bin/node /usr/local/bin/node
```

```
COPY --from=node /usr/local/lib/node_modules /usr/local/lib/node_modules
RUN ln -s /usr/local/lib/node_modules/npm/bin/npm-cli.js /usr/local/bin/npm \
    && ln -s /usr/local/lib/node_modules/npm/bin/npx-cli.js /usr/local/bin/npx

# ------------------------------------------------------------------------
# Development: source is bind-mounted, see docker-compose.yml
# ------------------------------------------------------------------------
FROM toolchain AS dev

ARG UID=1000
ARG GID=1000

RUN mv "$PHP_INI_DIR/php.ini-development" "$PHP_INI_DIR/php.ini" \
    && mkdir -p /var/www/html/node_modules \
    && chown "$UID:$GID" /var/www/html/node_modules

ENV HOME=/tmp COMPOSER_HOME=/tmp/composer

COPY docker/dev-entrypoint.sh /usr/local/bin/dev-entrypoint
ENTRYPOINT ["dev-entrypoint"]
CMD ["sh", "-c", "php artisan serve --host=0.0.0.0 --port=8000 & exec npm run dev"]

# ------------------------------------------------------------------------
# Build: install production dependencies and compile front-end assets
# ------------------------------------------------------------------------
FROM toolchain AS build

COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --no-interaction --prefer-dist

COPY package.json package-lock.json .npmrc ./
RUN npm ci

COPY . .
RUN composer dump-autoload --optimize --no-dev \
    && npm run build \
    && rm -rf node_modules

# ------------------------------------------------------------------------
# Production PHP-FPM image: no Node, no dev dependencies
# ------------------------------------------------------------------------
FROM php-base AS production

RUN mv "$PHP_INI_DIR/php.ini-production" "$PHP_INI_DIR/php.ini"

COPY --from=build --chown=www-data:www-data /var/www/html /var/www/html
COPY docker/entrypoint.sh /usr/local/bin/entrypoint

# /data holds SQLite and the generated keys. Mount persistent storage there: the sqlite-data
# volume in docker-compose.yml, or a Railway volume (Railway rejects the VOLUME instruction).
RUN mkdir -p /data && chown www-data:www-data /data

EXPOSE 9000
ENTRYPOINT ["entrypoint"]
CMD ["php-fpm"]

# ------------------------------------------------------------------------
# nginx: serves public/ (including built assets) and proxies PHP to the app
# ------------------------------------------------------------------------
FROM nginx:${NGINX_VERSION}-alpine AS web

COPY docker/nginx.conf /etc/nginx/conf.d/default.conf
COPY --chmod=755 docker/nginx-http-auth.sh /docker-entrypoint.d/40-http-auth.sh
COPY --from=build /var/www/html/public /var/www/html/public

EXPOSE 80

# ------------------------------------------------------------------------
# Railway: nginx and PHP-FPM in one container. Railway runs one container per
# service and always builds the last stage of this file; docker-compose builds
```

```
# the web and app targets above instead.
# ------------------------------------------------------------------------
FROM production AS railway

# Debian's nginx logs to files; send them to the container output so Railway shows them.
RUN apt-get update \
    && apt-get install -y --no-install-recommends nginx \
    && rm -rf /var/lib/apt/lists/* /etc/nginx/sites-enabled/default \
    && ln -sf /dev/stdout /var/log/nginx/access.log \
    && ln -sf /dev/stderr /var/log/nginx/error.log

COPY docker/nginx.conf /etc/nginx/conf.d/default.conf
RUN sed -i 's/app:9000/127.0.0.1:9000/g' /etc/nginx/conf.d/default.conf \
    && printf '[www]\nlisten = 127.0.0.1:9000\n' > /usr/local/etc/php-fpm.d/zz-railway.conf

COPY --chmod=755 docker/nginx-http-auth.sh /usr/local/bin/nginx-http-auth
COPY --chmod=755 docker/railway-start.sh /usr/local/bin/railway-start

# The PHP image stops with SIGQUIT; Railway (and this script) use SIGTERM.
STOPSIGNAL SIGTERM
EXPOSE 8080
ENTRYPOINT ["railway-start"]
CMD []
```

`docker-compose.yml` (new file)

Create it with this content:

```
services:
    # Production-like stack: `docker compose up --build` -> http://localhost:8000
    # nginx serves static files and proxies PHP requests to the app (PHP-FPM)
    web:
        build:
            context: .
            target: web
        ports:
            - '${APP_PORT:-8000}:80'
        environment:
            # Optional HTTP basic auth for the whole site; off when the password is empty.
            HTTP_AUTH_USER: ${HTTP_AUTH_USER:-}
            HTTP_AUTH_PASSWORD: ${HTTP_AUTH_PASSWORD:-}
        depends_on:
            app:
                condition: service_healthy

    app:
        build:
            context: .
            target: production
        healthcheck:
            test:
                - CMD
                - php
                - -r
                - "exit(@fsockopen('127.0.0.1', 9000) ? 0 : 1);"
            interval: 5s
            timeout: 3s
            retries: 3
            start_period: 30s
        environment:
            APP_NAME: GameVote
            APP_ENV: production
            APP_DEBUG: 'false'
            APP_URL: http://localhost:${APP_PORT:-8000}
            APP_KEY: ${DOCKER_APP_KEY:-}
            DB_CONNECTION: sqlite
            DB_DATABASE: /data/database.sqlite
            LOG_CHANNEL: stderr
            API_KEY_ESSENCE: ${API_KEY_ESSENCE:-}
```

```
        volumes:
            - sqlite-data:/data

    # Development with hot reload: `docker compose --profile dev up dev`
    # App on http://localhost:8000, Vite dev server on http://localhost:5173
    dev:
        profiles: [dev]
        build:
            context: .
            target: dev
            args:
                UID: ${UID:-1000}
                GID: ${GID:-1000}
        user: '${UID:-1000}:${GID:-1000}'
        environment:
            # Makes laravel-vite-plugin listen on 0.0.0.0 and advertise localhost
            LARAVEL_SAIL: '1'
        # Local machine only: the dev server runs with APP_DEBUG=true and Vite serves source files.
        ports:
            - '127.0.0.1:${APP_PORT:-8000}:8000'
            - '127.0.0.1:5173:5173'
        volumes:
            - .:/var/www/html
            - node-modules:/var/www/html/node_modules

volumes:
    sqlite-data:
    node-modules:
```

`docker/dev-entrypoint.sh` (new file)

Create it with this content:

```
#!/bin/sh
set -e

cd /var/www/html

[ -f .env ] || cp .env.example .env
[ -d vendor ] || composer install --no-interaction
[ -f node_modules/.package-lock.json ] || npm ci
grep -q '^APP_KEY=base64:' .env || php artisan key:generate
[ -f database/database.sqlite ] || touch database/database.sqlite
php artisan migrate --force

exec "$@"
```

`docker/entrypoint.sh` (new file)

Create it with this content:

```
#!/bin/sh
set -e

cd /var/www/html

DB_DATABASE="${DB_DATABASE:-/data/database.sqlite}"
export DB_DATABASE

# Generate and persist an APP_KEY on first boot if one wasn't provided.
if [ -z "$APP_KEY" ]; then
    if [ ! -f /data/app.key ]; then
        php artisan key:generate --show > /data/app.key
        chown www-data:www-data /data/app.key
    fi
    APP_KEY="$(cat /data/app.key)"
    export APP_KEY
```

```
fi

if [ ! -f "$DB_DATABASE" ]; then
    touch "$DB_DATABASE"
fi
chown www-data:www-data "$DB_DATABASE" "$(dirname "$DB_DATABASE")"

php artisan migrate --force
php artisan optimize
chown -R www-data:www-data storage bootstrap/cache

exec "$@"
```

docker/nginx-http-auth.sh (new file)

Create it with this content:

```
#!/bin/sh
# Runs before nginx starts: from /docker-entrypoint.d/ in the docker-compose nginx image,
# and from railway-start.sh in the Railway image.
# When HTTP_AUTH_PASSWORD is set, protect the whole site with HTTP basic auth;
# otherwise leave it open (the default, so `docker-compose up` works without setup).
set -eu

snippet=/etc/nginx/snippets/http-auth.conf
password_file=/etc/nginx/.htpasswd
mkdir -p /etc/nginx/snippets

if [ -z "${HTTP_AUTH_PASSWORD:-}" ]; then
    : > "$snippet"
    rm -f "$password_file"
    echo "$0: HTTP_AUTH_PASSWORD is not set; HTTP basic auth is off"
    exit 0
fi

user="${HTTP_AUTH_USER:-gamevote}"
case "$user" in
    *:*) echo "$0: HTTP_AUTH_USER must not contain ':'" >&2; exit 1 ;;
esac

# Hash with SHA-512 crypt, reading the password from stdin so it never appears in a process list.
# The Railway image has PHP; the Alpine nginx image has BusyBox mkpasswd.
hash_password() {
    if command -v php >/dev/null 2>&1; then
        php -r 'echo crypt(stream_get_contents(STDIN), "\$6\$" . bin2hex(random_bytes(8)));'
    else
        mkpasswd -m sha512 -P 0
    fi
}

hash=$(printf '%s' "$HTTP_AUTH_PASSWORD" | hash_password)
printf '%s:%s\n' "$user" "$hash" > "$password_file"

# nginx workers run as "nginx" on Alpine and "www-data" on Debian.
group=$(id -gn nginx 2>/dev/null || echo www-data)
chown "root:$group" "$password_file"
chmod 640 "$password_file"

# $basic_auth_realm is "off" for /up (see nginx.conf), so health checks never need the password.
cat > "$snippet" <<'CONF'
auth_basic $basic_auth_realm;
auth_basic_user_file /etc/nginx/.htpasswd;
CONF

echo "$0: HTTP basic auth is on for user '$user'"
```

`docker/nginx.conf` (new file)

Create it with this content:

```
# Basic auth realm per request: off for /up, Laravel's health check (used by Railway).
map $request_uri $basic_auth_realm {
    ~^/up(\?.*)?$ off;
    default "GameVote";
}

# HSTS only for requests a TLS-terminating proxy (Railway) reports as https; empty means not sent.
map $http_x_forwarded_proto $hsts_header {
    https "max-age=31536000";
    default "";
}

server {
    listen 80;
    listen [::]:80;
    server_name _;
    root /var/www/html/public;

    # Don't advertise the nginx version. The Content-Security-Policy comes from Laravel
    # (it needs a per-request nonce); these headers apply to every response.
    server_tokens off;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "camera=(), microphone=(), geolocation=(), payment=(), usb=()" always;
    add_header Strict-Transport-Security $hsts_header always;

    # HTTP basic auth when HTTP_AUTH_PASSWORD is set; empty otherwise (see nginx-http-auth.sh).
    include /etc/nginx/snippets/http-auth.conf;

    index index.php;
    charset utf-8;
    client_max_body_size 20M;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    # Fingerprinted Vite assets can be cached forever
    location /build/assets/ {
        # add_header here replaces the server-level headers, so repeat the one that matters for assets.
        add_header Cache-Control "public, max-age=31536000, immutable";
        add_header X-Content-Type-Options "nosniff";
        access_log off;
        try_files $uri =404;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt  { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ ^/index\.php(/|$) {
        fastcgi_pass app:9000;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_hide_header X-Powered-By;

        # Laravel's response headers (Content-Security-Policy, the asset preload Link header and
        # cookies) outgrow the default 4-8 KB buffer, which nginx answers with a 502.
        fastcgi_buffer_size 32k;
        fastcgi_buffers 16 16k;
    }

    location ~ /\.(?!well-known).* {
        deny all;
    }
```

```
}
```

`docker/railway-start.sh` (new file)

Create it with this content:

```
#!/bin/bash
# Railway runs one container per service, so this image runs PHP-FPM and nginx together
# (docker-compose runs them as separate web and app containers instead).
set -euo pipefail

# Same first-boot setup as the docker-compose app container: APP_KEY, SQLite file,
# migrations and caches. Its final `exec "$@"` runs `true`, handing control back here.
entrypoint true
nginx-http-auth

# Listen on Railway's PORT (default 8080) so its proxy can route to us.
port="${PORT:-8080}"
sed -i -E "s/listen (\[::\]:)?[0-9]+;/listen \1${port};/" /etc/nginx/conf.d/default.conf

php-fpm --nodaemonize &
fpm_pid=$!
nginx -g 'daemon off;' &
nginx_pid=$!

# QUIT is the graceful stop for both: requests already in progress finish first.
stop() {
    kill -QUIT "$fpm_pid" "$nginx_pid" 2>/dev/null || true
    wait "$fpm_pid" "$nginx_pid" 2>/dev/null || true
}

# Stop both on shutdown; if either exits on its own, stop the other so Railway restarts us.
trap 'stop; exit 0' TERM INT QUIT
status=0
wait -n "$fpm_pid" "$nginx_pid" || status=$?
stop
exit "$status"
```